

Neural Networks

Lecture 2 - Working with Neural Networks

Peter Lehoczký

(slides based on materials by *Igor Jánoš* and contributors)

Agenda

1. Back-propagation Algorithm with Neural Network
2. Machine Learning Project
 - Goals, Metrics, Datasets, Splits, Models
 - Model-centric, Data-centric Approach
3. NN Training Techniques
4. Optimizers

1 Back-propagation with NN

Quick Recap

Backpropagation Basics (Last Lecture)

Chain Rule

- Two differentiable functions

$$h(x), \quad g(y)$$

$$h'(x) = \frac{dh(x)}{dx}, \quad g'(y) = \frac{dg(y)}{dy}$$

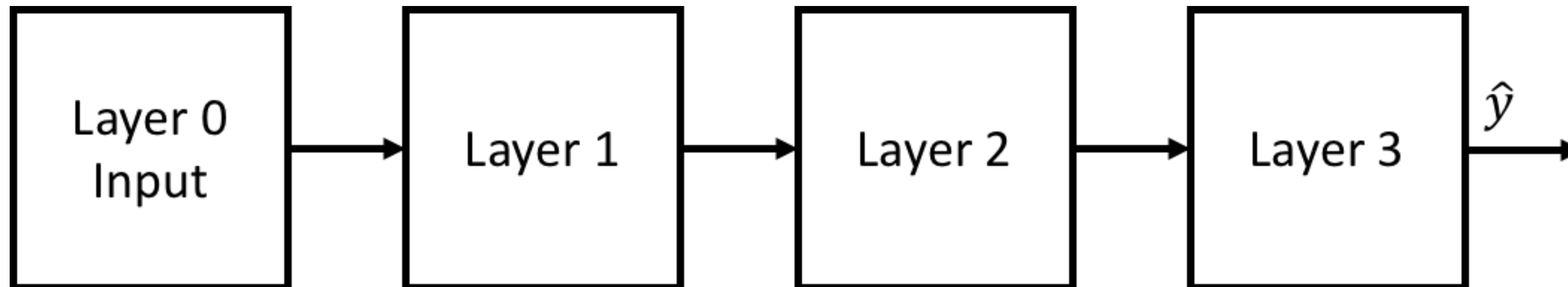
- Compound function

$$f(y) = h(g(y))$$

$$f'(y) = h'(g(y))g'(y)$$

Artificial Neural Network

- Layer ~ Function
- Network ~ Compound Function



Forward Pass - Vectorized Form

Multiple Samples

$$\mathcal{X} = \{x^{(1)}, x^{(2)}, \dots, x^{(m)}\} \quad \mathcal{Y} = \{y^{(1)}, y^{(2)}, \dots, y^{(m)}\}$$

$$\begin{pmatrix} | & | & & | \\ z^{(1)} & z^{(2)} & \dots & z^{(m)} \\ | & | & & | \end{pmatrix} = \begin{pmatrix} - & w_1 & - \\ - & w_2 & - \\ - & w_3 & - \end{pmatrix} \begin{pmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

$$\textcolor{red}{Z} = W\textcolor{red}{X} + b$$

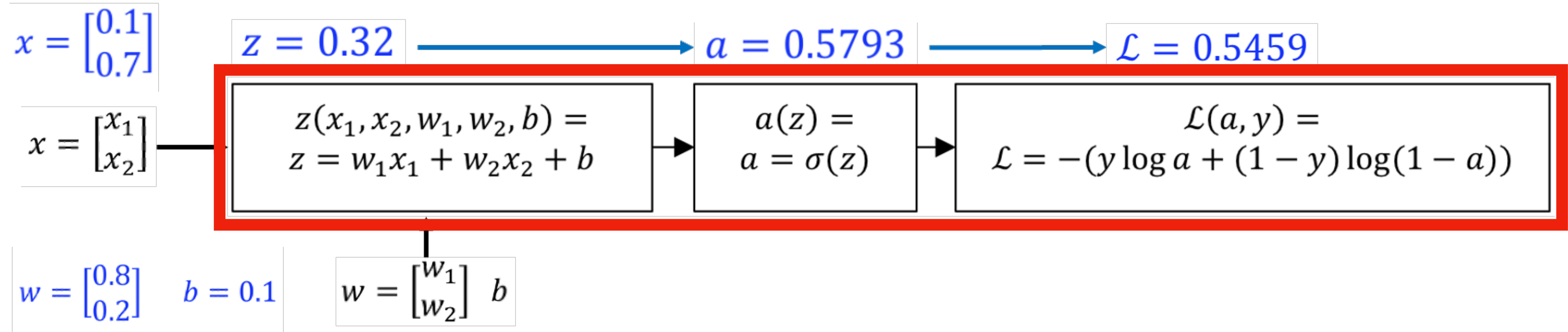
$$\textcolor{red}{A} = g(\textcolor{red}{Z})$$

Example - Logistic Regression

Backward Pass



$$\mathcal{X} = \{(x_1 = 0.1, x_2 = 0.7) \rightarrow (y = 1)\}$$



$$\nabla w = \begin{bmatrix} -0.04207 \\ -0.29447 \end{bmatrix}$$

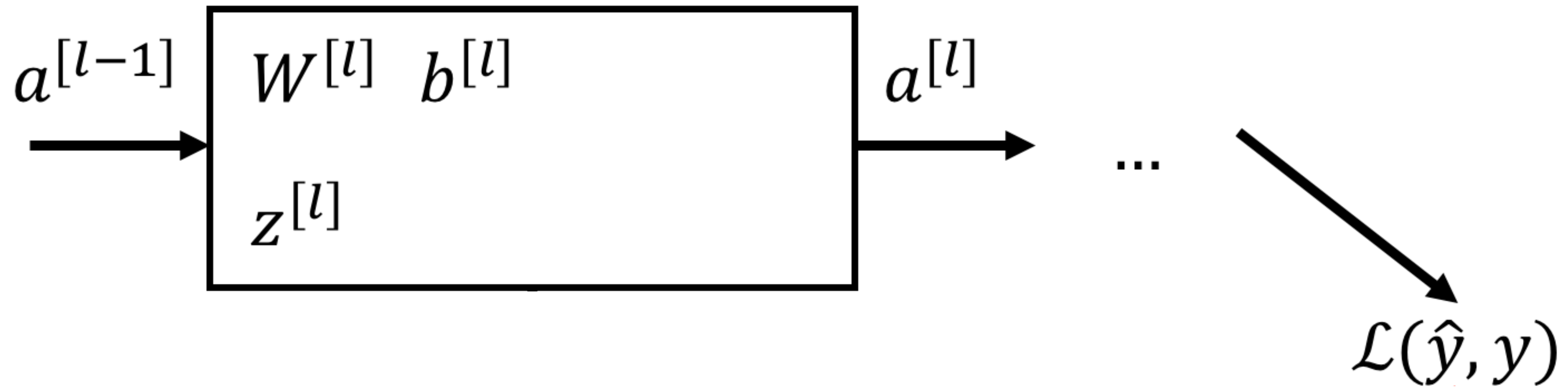
$$\nabla b = -0.4207$$

Back-propagation with Neural Network

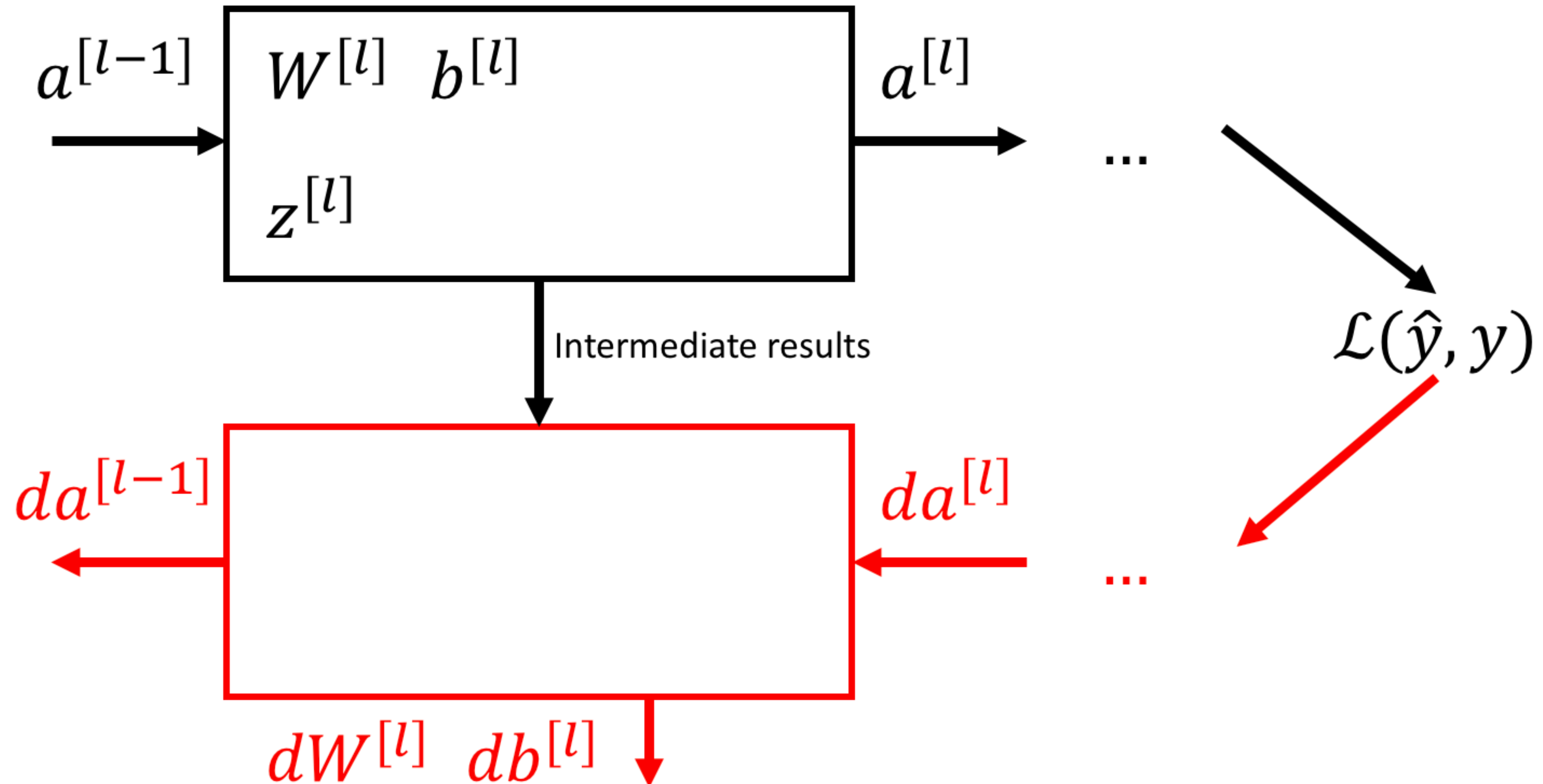
- Simplify notation

$$\frac{\partial \mathcal{L}(\hat{y}, y)}{\partial \mathbf{W}^{[1]}} = \text{“d}\mathbf{W}^{[1]}\text{”}$$

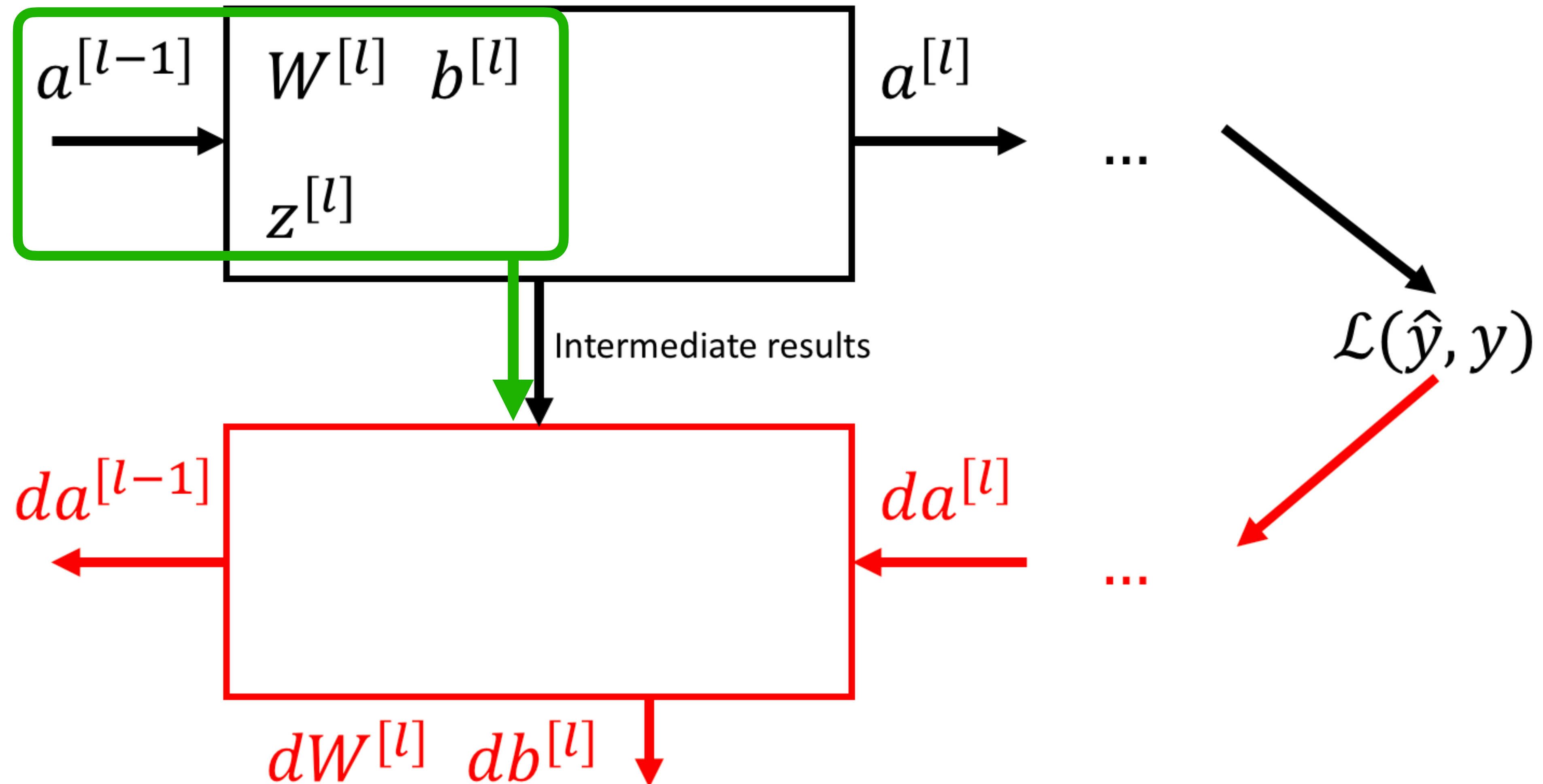
Back-propagation with Neural Network



Back-propagation with Neural Network

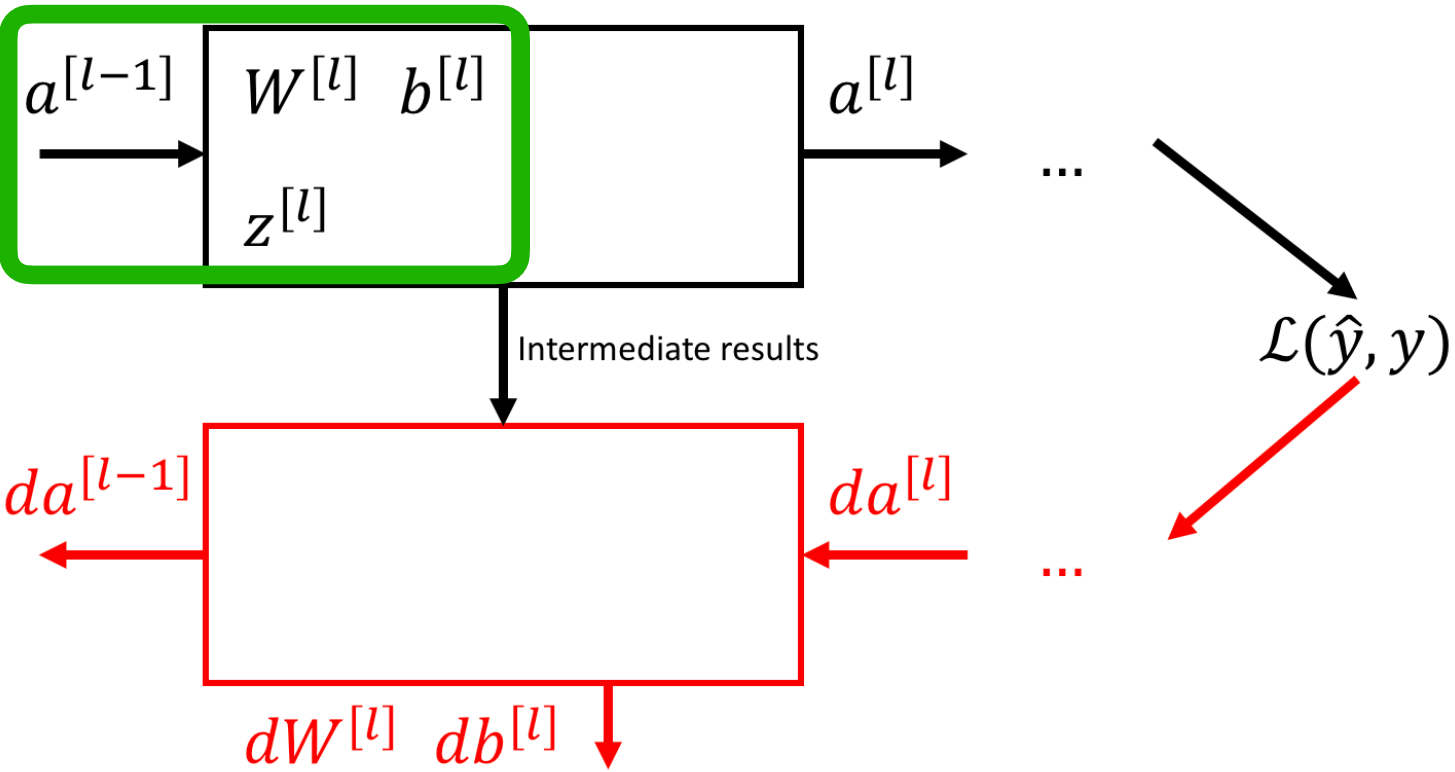
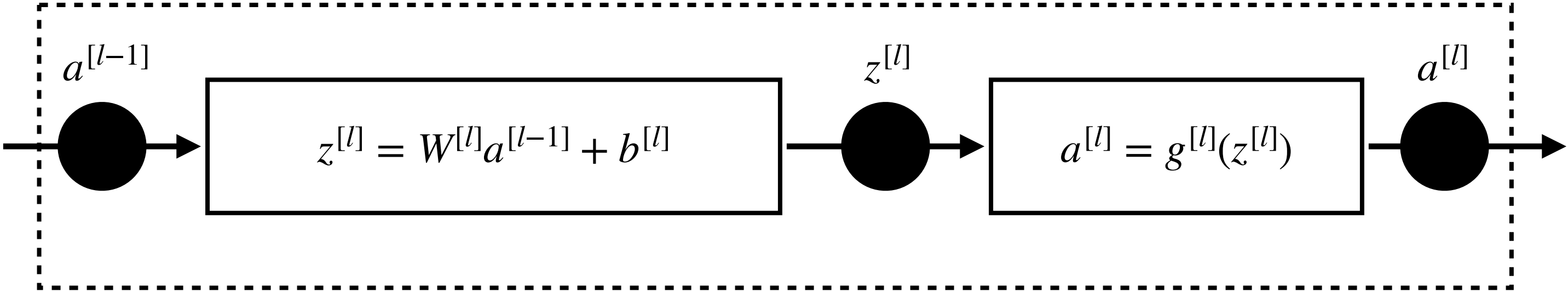


Back-propagation with Neural Network



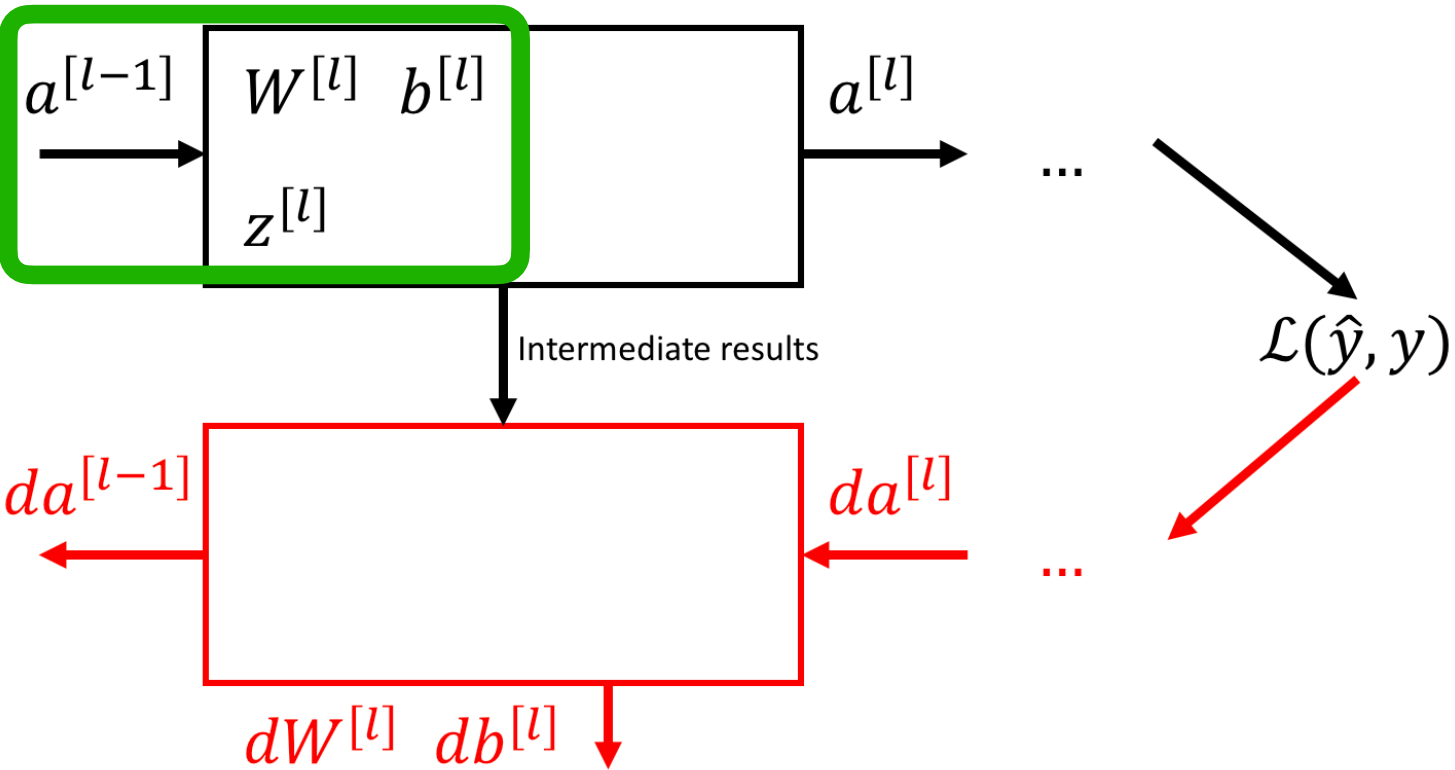
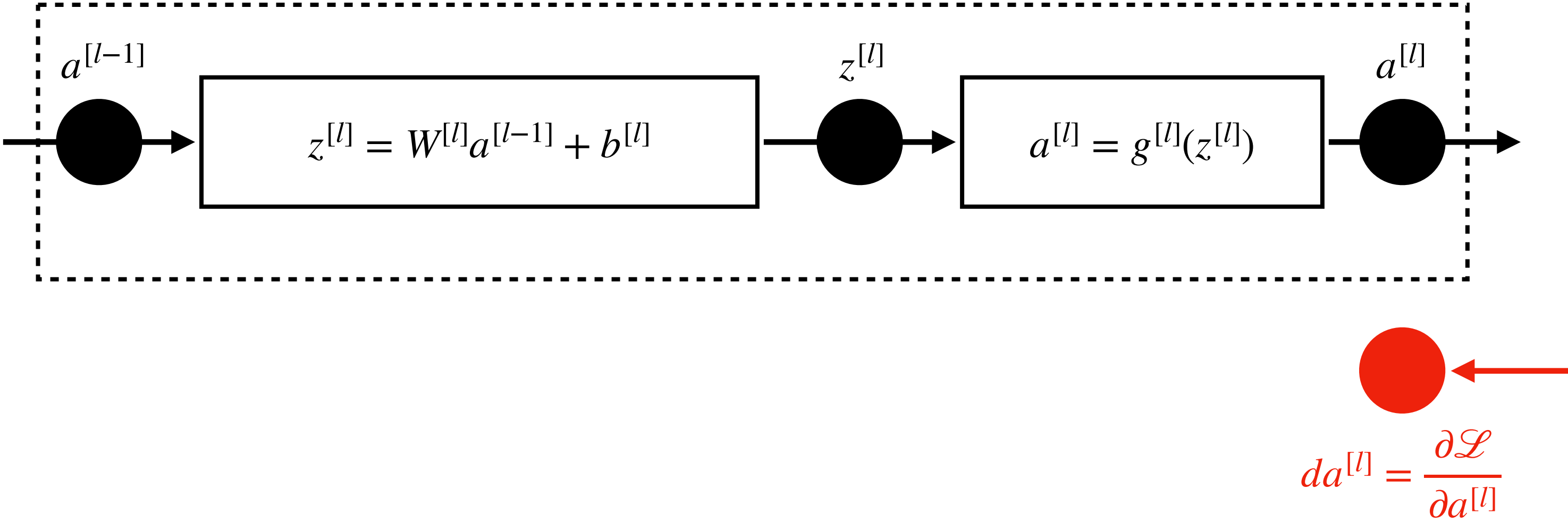
Back-propagation with Neural Network

Backward Pass



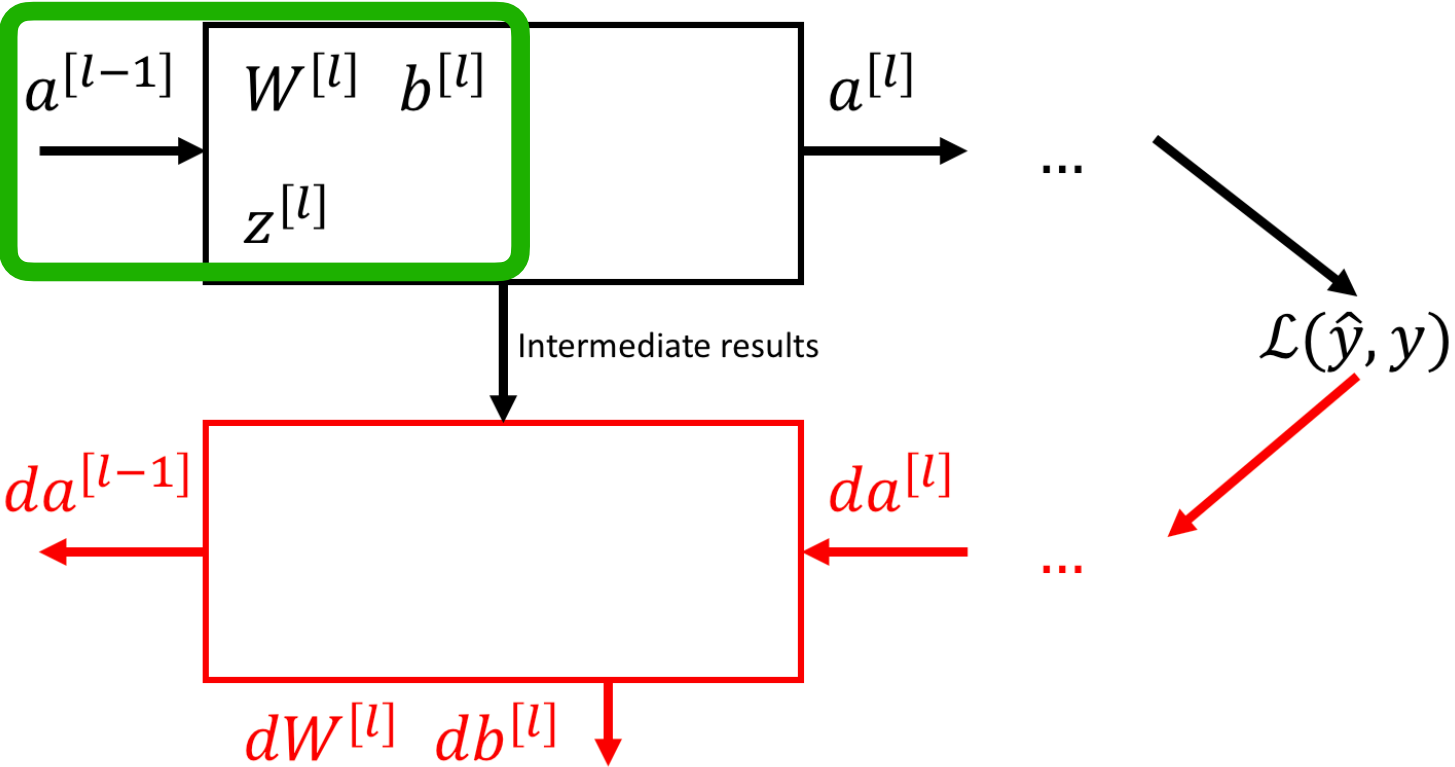
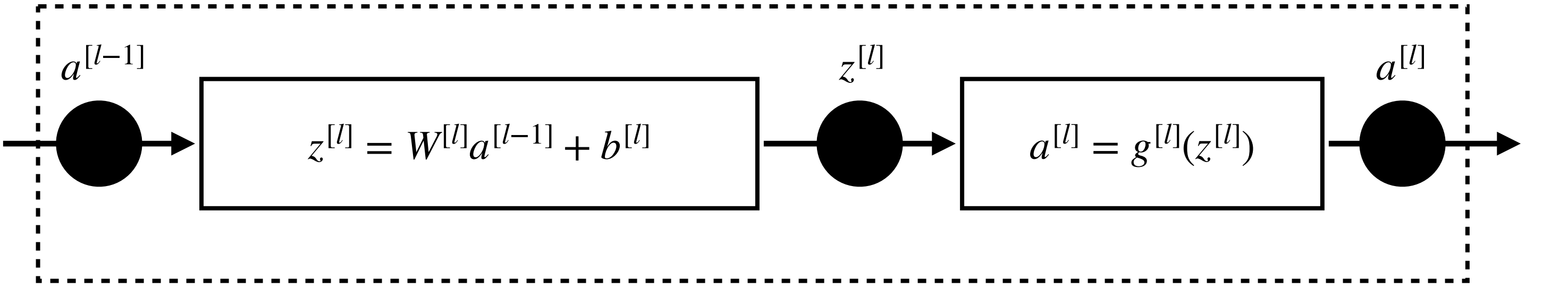
Back-propagation with Neural Network

Backward Pass



Back-propagation with Neural Network

Backward Pass



$$dz^{[l]} = \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$
$$da^{[l]} = \frac{\partial \mathcal{L}}{\partial a^{[l]}}$$

$$dz^{[l]} = g^{[l]'}(z^{[l]}) \odot da^{[l]}$$

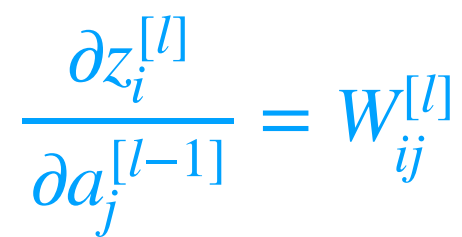
Inner function derivative

Outer function derivative

Element-wise multiplication

$$\begin{bmatrix} | \\ dz^{[l]} \\ (n_l \times 1) \\ | \end{bmatrix} = \begin{bmatrix} | \\ g^{[l]'}(z^{[l]}) \\ (n_l \times 1) \\ | \end{bmatrix} \odot \begin{bmatrix} | \\ da^{[l]} \\ (n_l \times 1) \\ | \end{bmatrix}$$

Backward Pass



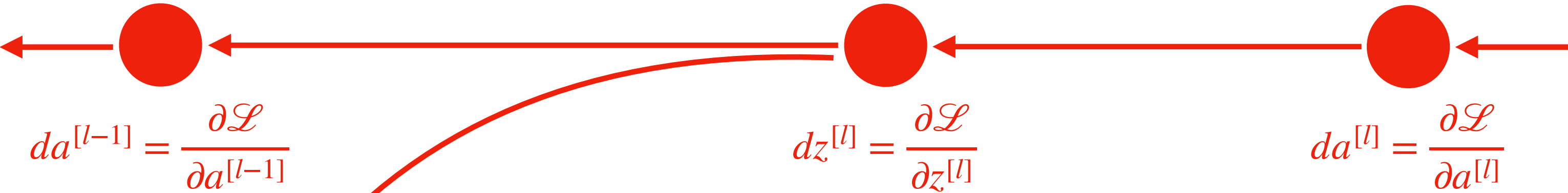
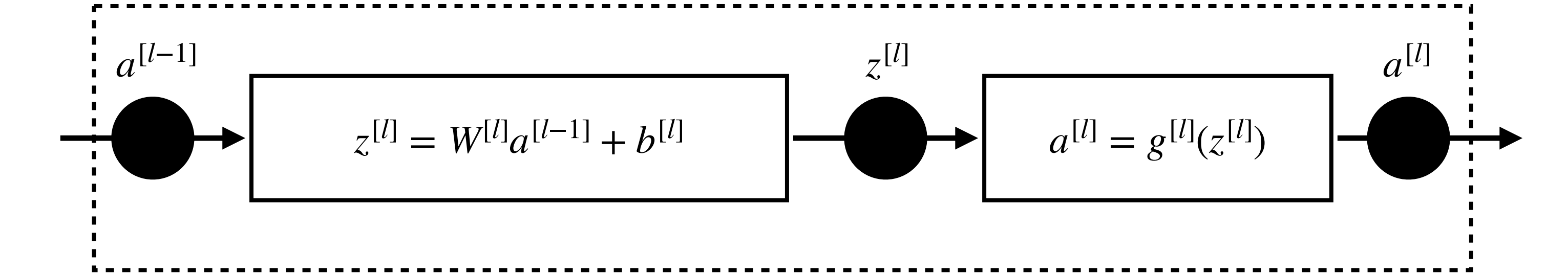
$$da^{[l-1]} = \underbrace{W^{[l]T}}_{\text{weight}} \underbrace{dz^{[l]}}_{\text{activation}}$$

Outer function derivative



Back-propagation with Neural Network

Backward Pass



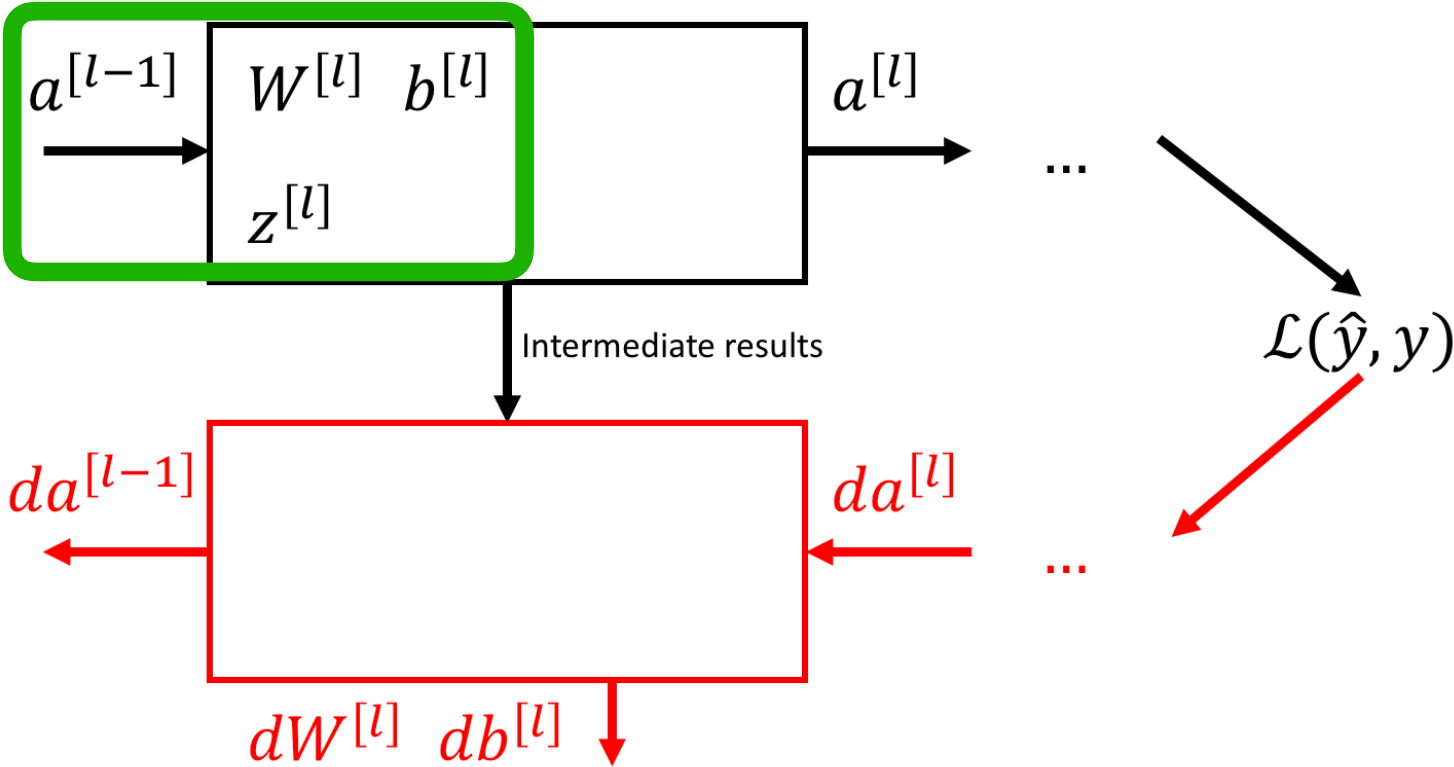
$$db^{[l]} = \frac{\partial \mathcal{L}}{\partial b^{[l]}} = \left(\frac{\partial z^{[l]}}{\partial b^{[l]}} \right)^T \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

$$\frac{\partial z_i^{[l]}}{\partial b_k^{[l]}} = \begin{cases} 1 & i = k \\ 0 & i \neq k \end{cases} \Rightarrow \frac{\partial z^{[l]}}{\partial b^{[l]}} = I_{n_l}$$

$$db^{[l]} = \mathbf{1} \cdot dz^{[l]}$$

Inner function derivative
(1 = identity matrix)

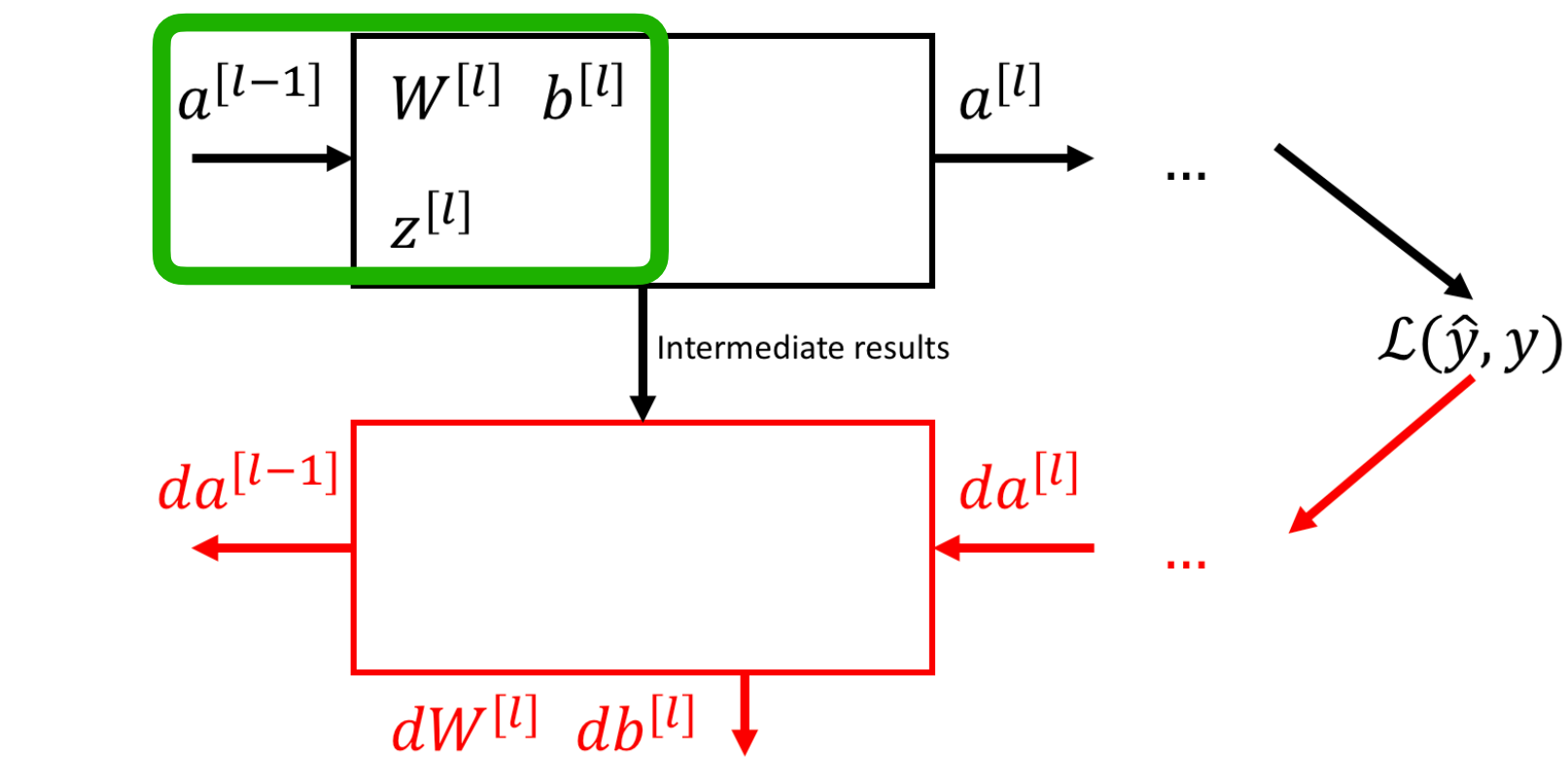
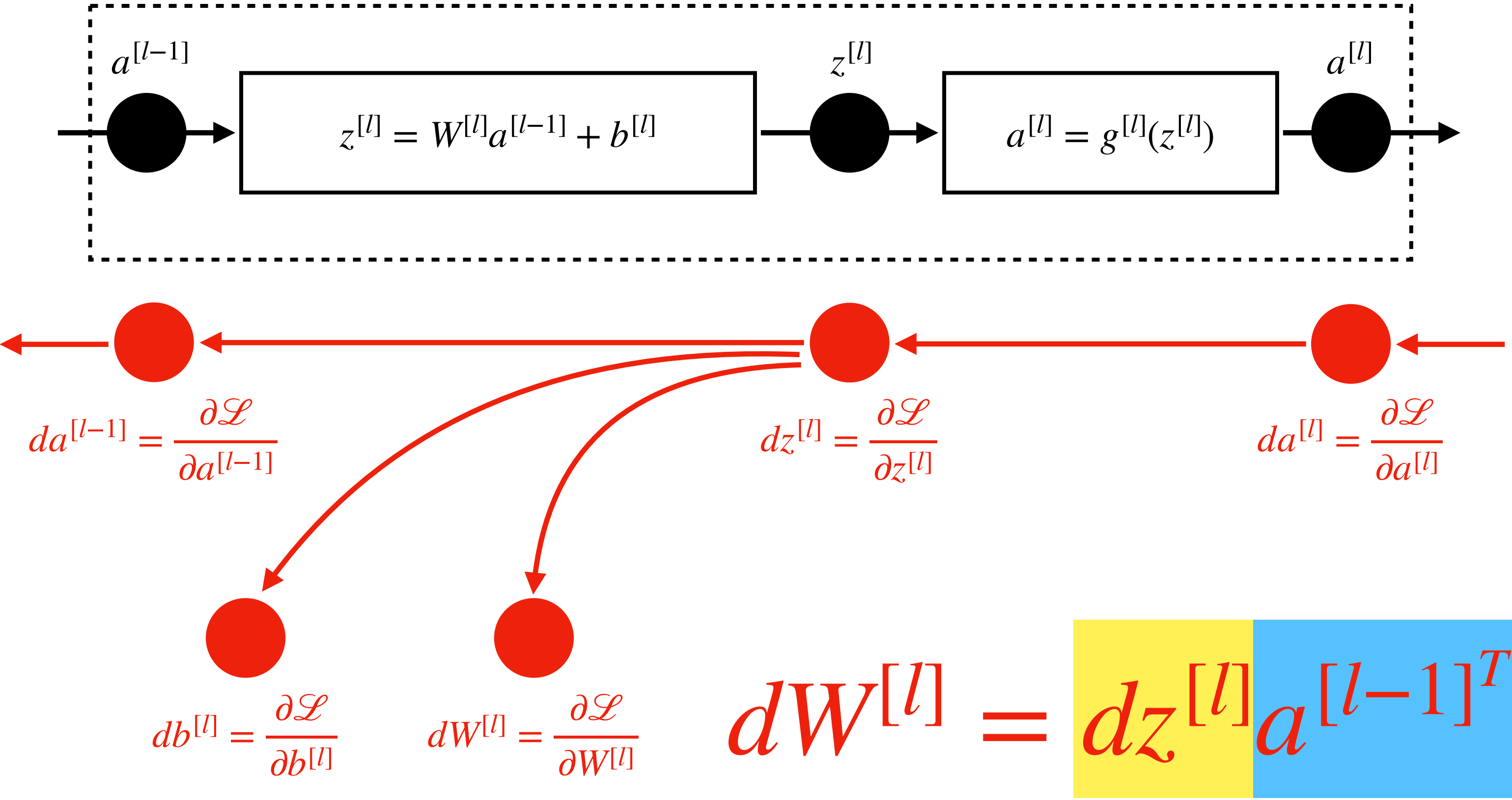
Outer function derivative



$$dz^{[l]} = g^{[l]'}(z^{[l]}) \odot da^{[l]}$$
$$da^{[l-1]} = W^{[l]T} dz^{[l]}$$

Back-propagation with Neural Network

Backward Pass

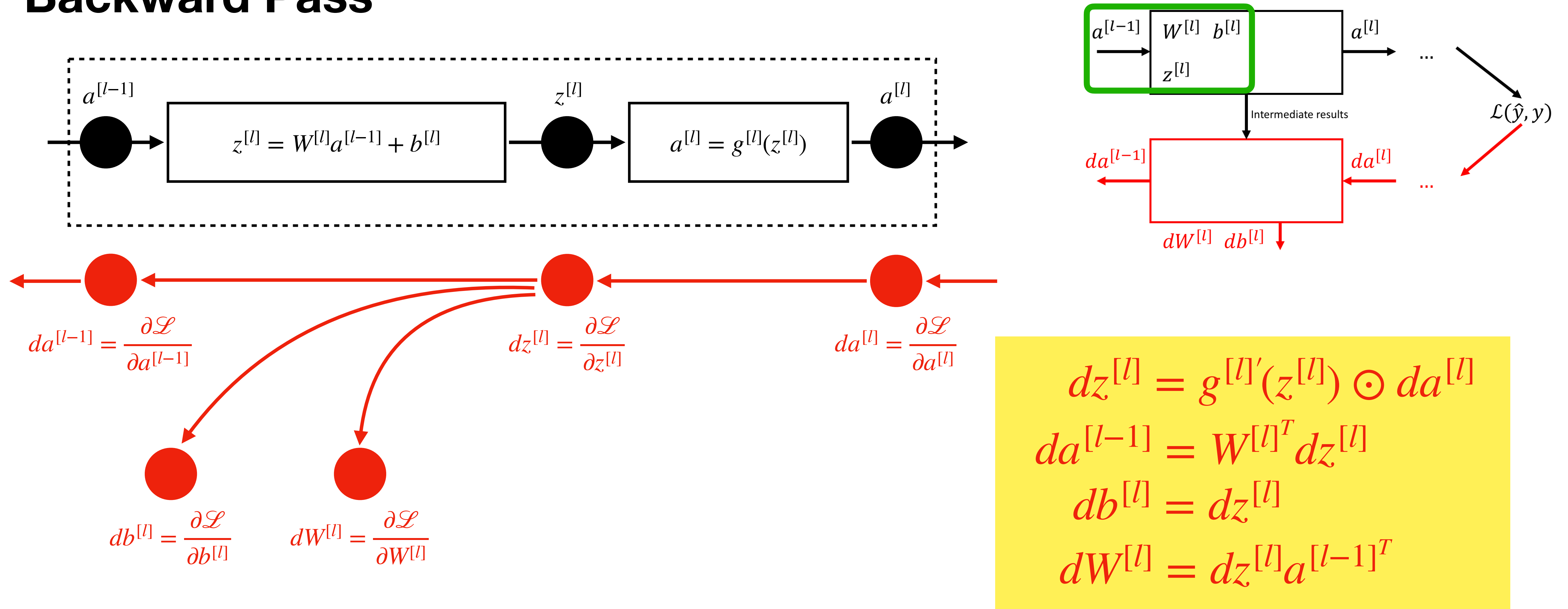


$$dz^{[l]} = g^{[l]'}(z^{[l]}) \odot da^{[l]}$$
$$da^{[l-1]} = W^{[l]T} dz^{[l]}$$
$$db^{[l]} = dz^{[l]}$$

$$\frac{\partial z_k^{[l]}}{\partial W_{ij}^{[l]}} = \begin{cases} a_j^{[l-1]} & k = i \\ 0 & k \neq i \end{cases} \Rightarrow \frac{\partial \mathcal{L}}{\partial W_{ij}^{[l]}} = \sum_{k=1}^{n_l} \frac{\partial \mathcal{L}}{\partial z_k^{[l]}} \frac{\partial z_k^{[l]}}{\partial W_{ij}^{[l]}} = \frac{\partial \mathcal{L}}{\partial z_i^{[l]}} a_j^{[l-1]} = dz_i^{[l]} a_j^{[l-1]}$$

Back-propagation with Neural Network

Backward Pass

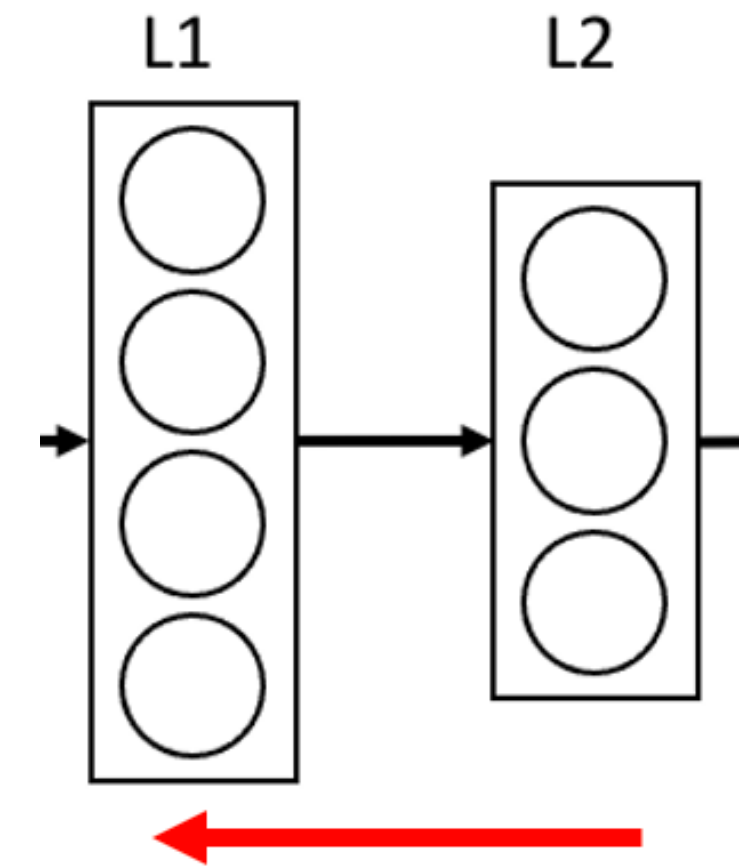


Back-propagation with Neural Network

Shapes

$$dz^{[2]} = g^{[2]'}(z^{[2]}) \odot da^{[2]}$$

(3×1)



$$dz^{[l]} = g^{[l]'}(z^{[l]}) \odot da^{[l]}$$

$$da^{[l-1]} = W^{[l]T} dz^{[l]}$$

$$db^{[l]} = dz^{[l]}$$

$$dW^{[l]} = dz^{[l]} a^{[l-1]T}$$

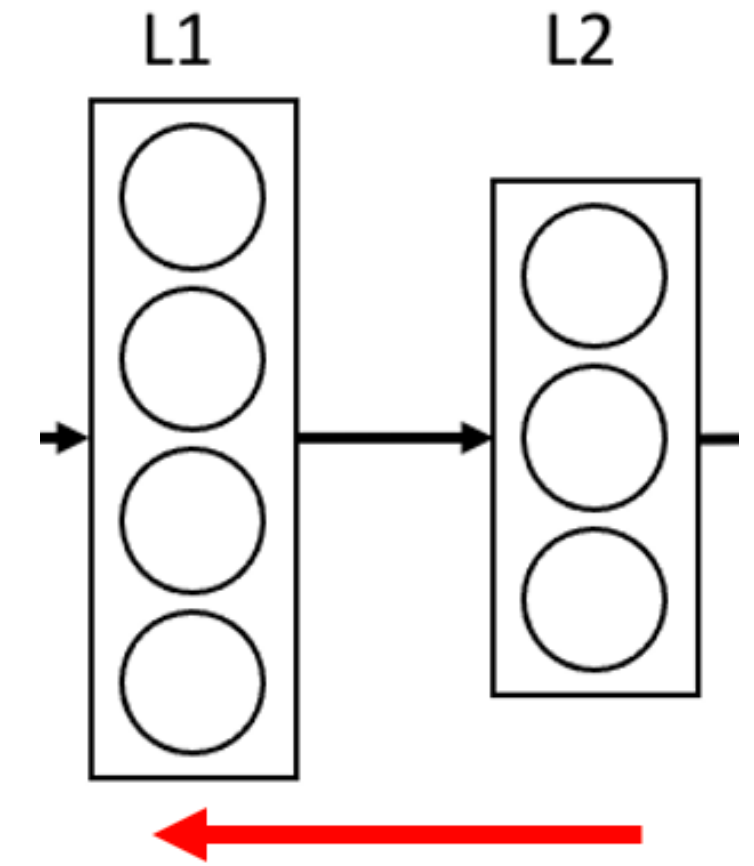
Back-propagation with Neural Network

Shapes

$$dz^{[2]} = g^{[2]'}(z^{[2]}) \odot da^{[2]}$$

$$da^{[1]} = W^{[2]T} dz^{[2]}$$

$$\begin{bmatrix} | \\ da^{[1]} \\ (4 \times 1) \\ | \end{bmatrix} = \begin{bmatrix} W^{[2]T} \\ (4 \times 3) \end{bmatrix} \begin{bmatrix} | \\ dz^{[2]} \\ (3 \times 1) \\ | \end{bmatrix}$$



$$\begin{aligned} dz^{[l]} &= g^{[l]'}(z^{[l]}) \odot da^{[l]} \\ da^{[l-1]} &= W^{[l]T} dz^{[l]} \\ db^{[l]} &= dz^{[l]} \\ dW^{[l]} &= dz^{[l]} a^{[l-1]T} \end{aligned}$$

Back-propagation with Neural Network

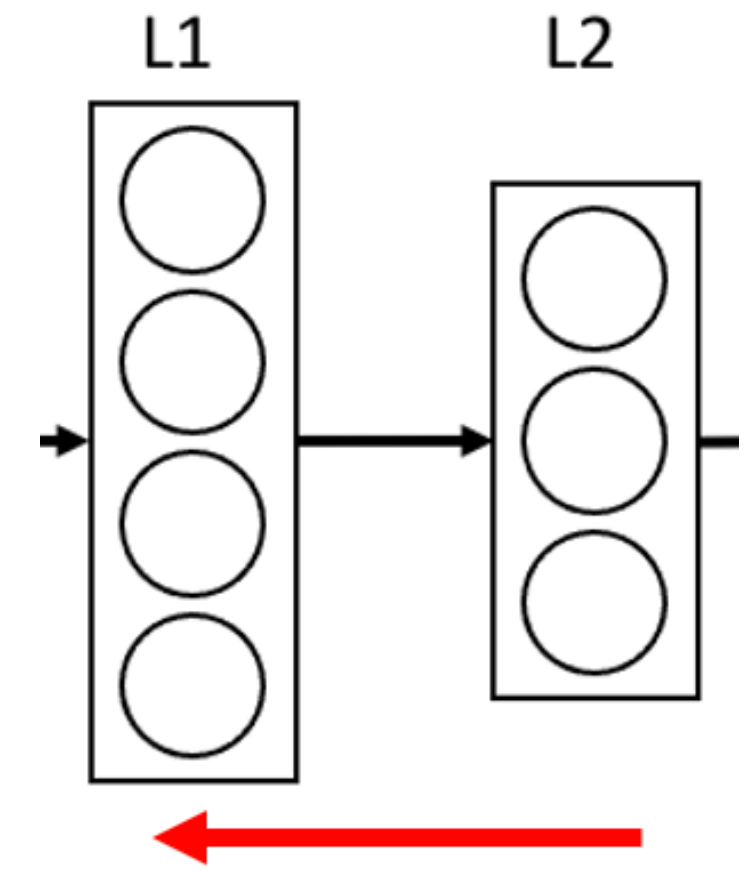
Shapes

$$dz^{[2]} = g^{[2]'}(z^{[2]}) \odot da^{[2]}$$

$$da^{[1]} = W^{[2]T} dz^{[2]}$$

$$db^{[2]} = dz^{[2]}$$

$$\begin{bmatrix} | \\ db^{[2]} \\ (3 \times 1) \\ | \end{bmatrix} = \begin{bmatrix} | \\ dz^{[2]} \\ (3 \times 1) \\ | \end{bmatrix}$$



$$\begin{aligned} dz^{[l]} &= g^{[l]'}(z^{[l]}) \odot da^{[l]} \\ da^{[l-1]} &= W^{[l]T} dz^{[l]} \\ db^{[l]} &= dz^{[l]} \\ dW^{[l]} &= dz^{[l]} a^{[l-1]T} \end{aligned}$$

Back-propagation with Neural Network

Shapes

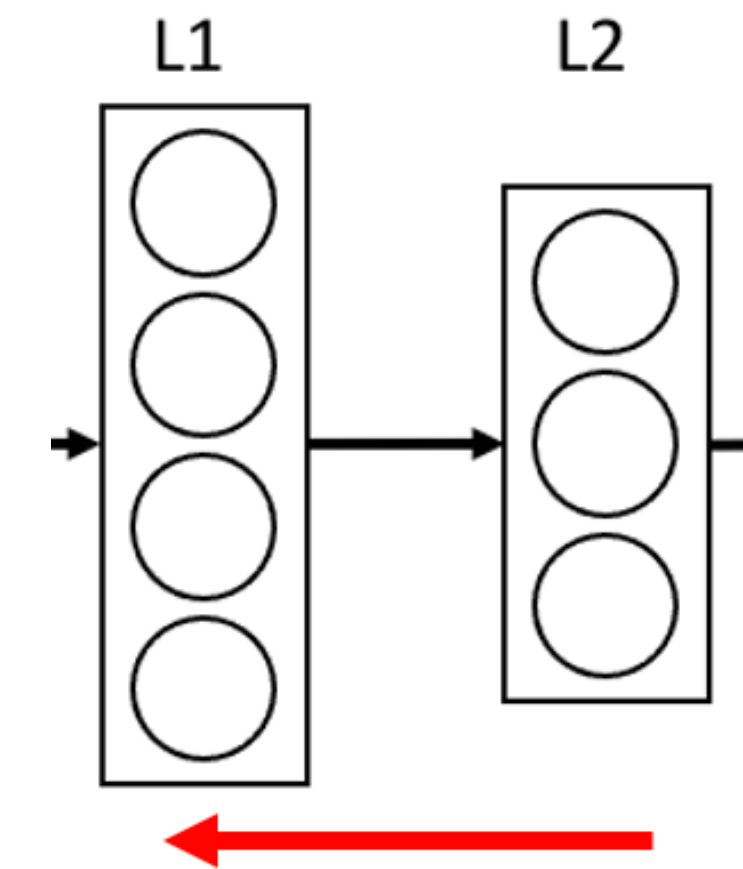
$$dz^{[2]} = g^{[2]'}(z^{[2]}) \odot da^{[2]}$$

$$da^{[1]} = W^{[2]T} dz^{[2]}$$

$$db^{[2]} = dz^{[2]}$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$\begin{bmatrix} dW^{[2]} \\ (3 \times 4) \end{bmatrix} = \begin{bmatrix} | \\ dz^{[2]} \\ (3 \times 1) \\ | \end{bmatrix} \begin{bmatrix} - & a^{[1]T} & - \\ (1 \times 4) & & \end{bmatrix}$$



$$dz^{[l]} = g^{[l]'}(z^{[l]}) \odot da^{[l]}$$

$$da^{[l-1]} = W^{[l]T} dz^{[l]}$$

$$db^{[l]} = dz^{[l]}$$

$$dW^{[l]} = dz^{[l]} a^{[l-1]T}$$

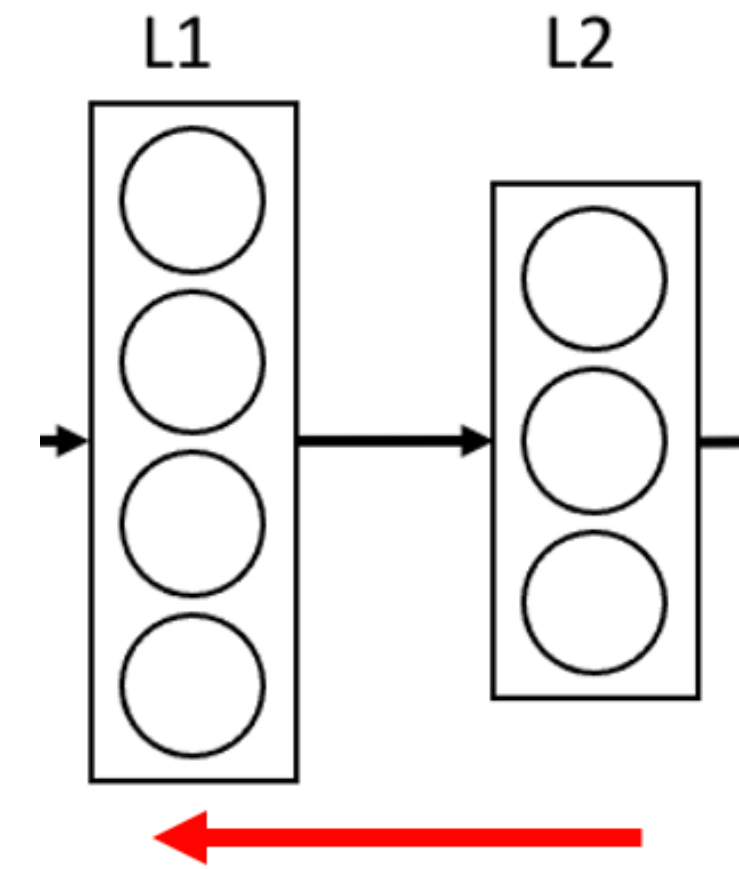
Back-propagation with Neural Network

Shapes - Vectorized

$$dZ^{[2]} = g^{[2]'}(Z^{[2]}) \odot dA^{[2]}$$

$(3 \times m)$

m - batch size



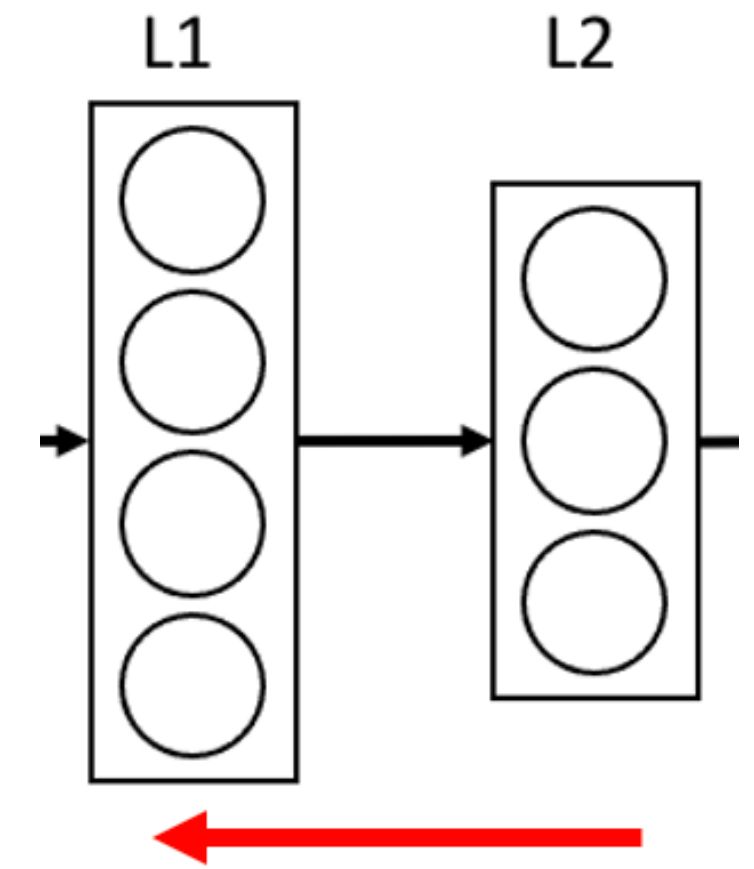
$$\begin{aligned}dZ^{[l]} &= g^{[l]'}(Z^{[l]}) \odot dA^{[l]} \\dA^{[l-1]} &= W^{[l]T} dZ^{[l]} \\db^{[l]} &= \text{mean}(dZ^{[l]}) \\dW^{[l]} &= \text{mean}(dZ^{[l]} A^{[l-1]T})\end{aligned}$$

Back-propagation with Neural Network

Shapes - Vectorized

$$dZ^{[2]} = g^{[2]'}(Z^{[2]}) \odot dA^{[2]}$$

$$dA^{[1]} = W^{[2]T} dZ^{[2]}$$



$$\begin{bmatrix} \dots & | & \dots \\ \dots & dA^{[1]} & \dots \\ & (4 \times m) & \\ & | & \end{bmatrix} = \begin{bmatrix} W^{[2]T} \\ (4 \times 3) \end{bmatrix} \begin{bmatrix} \dots & | & \dots \\ \dots & dZ^{[2]} & \dots \\ & (3 \times m) & \\ & | & \end{bmatrix}$$

$$\begin{aligned} dZ^{[l]} &= g^{[l]'}(Z^{[l]}) \odot dA^{[l]} \\ dA^{[l-1]} &= W^{[l]T} dZ^{[l]} \\ db^{[l]} &= \text{mean}(dZ^{[l]}) \\ dW^{[l]} &= \text{mean}(dZ^{[l]} A^{[l-1]T}) \end{aligned}$$

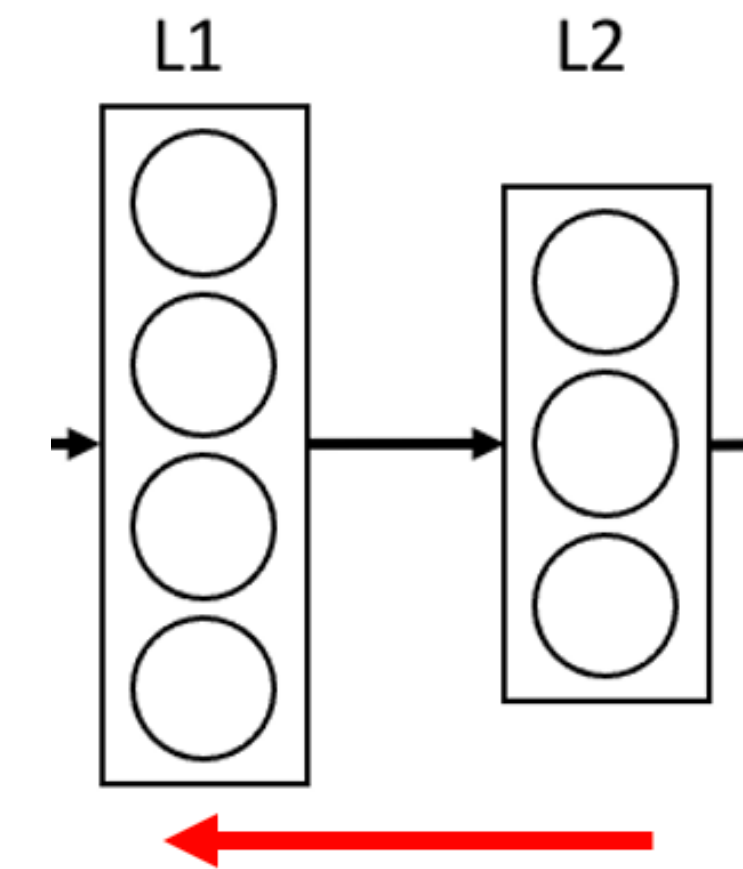
Back-propagation with Neural Network

Shapes - Vectorized

$$dZ^{[2]} = g^{[2]'}(Z^{[2]}) \odot dA^{[2]}$$

$$dA^{[1]} = W^{[2]T} dZ^{[2]}$$

$$db^{[2]} = \text{mean}(dZ^{[2]})$$



$$\begin{bmatrix} | \\ db^{[2]} \\ (3 \times 1) \\ | \end{bmatrix} = \frac{1}{m} \begin{bmatrix} \dots & | & \dots \\ \dots & dZ^{[2]} & \dots \\ | \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ (m \times 1) \end{bmatrix}$$

m

$$\begin{aligned} dZ^{[l]} &= g^{[l]'}(Z^{[l]}) \odot dA^{[l]} \\ dA^{[l-1]} &= W^{[l]T} dZ^{[l]} \\ db^{[l]} &= \text{mean}(dZ^{[l]}) \\ dW^{[l]} &= \text{mean}(dZ^{[l]} A^{[l-1]T}) \end{aligned}$$

Back-propagation with Neural Network

Shapes - Vectorized

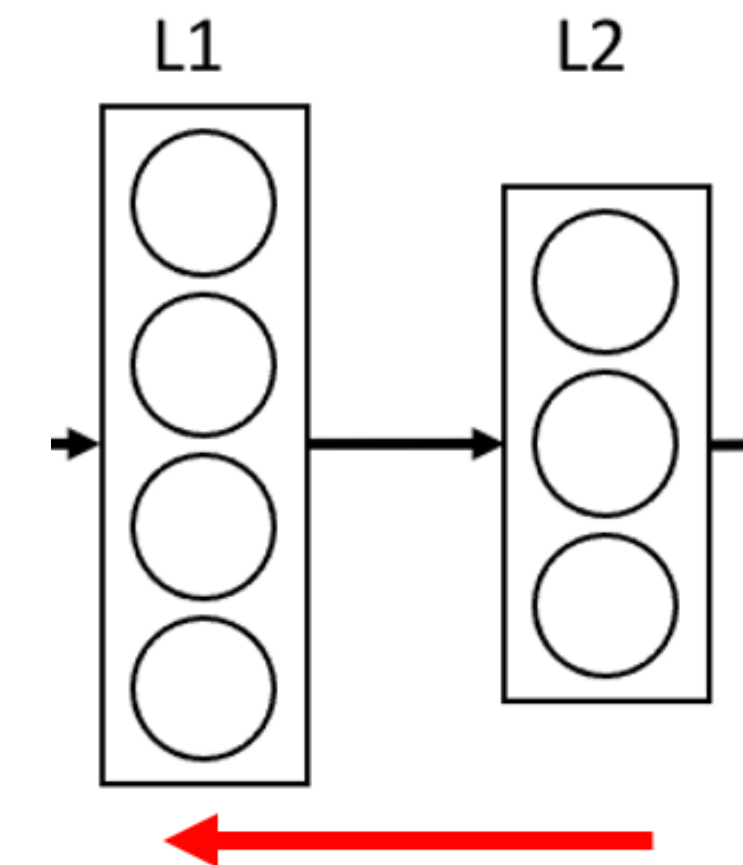
$$dZ^{[2]} = g^{[2]'}(Z^{[2]}) \odot dA^{[2]}$$

$$dA^{[1]} = W^{[2]T} dZ^{[2]}$$

$$db^{[2]} = \text{mean}(dZ^{[2]})$$

$$dW^{[2]} = \text{mean}(dZ^{[2]} A^{[1]T}) m$$

$$\begin{bmatrix} dW^{[2]} \\ (3 \times 4) \end{bmatrix} = \frac{1}{m} \begin{bmatrix} \dots & | & \dots \\ \dots & dZ^{[2]} & \dots \\ & (3 \times m) & \\ & | & \end{bmatrix} \begin{bmatrix} - \\ \dots \\ \dots \\ \dots \end{bmatrix} \left. \begin{matrix} A^{[1]T} \\ (m \times 4) \end{matrix} \right\} m$$



$$\begin{aligned} dZ^{[l]} &= g^{[l]'}(Z^{[l]}) \odot dA^{[l]} \\ dA^{[l-1]} &= W^{[l]T} dZ^{[l]} \\ db^{[l]} &= \text{mean}(dZ^{[l]}) \\ dW^{[l]} &= \text{mean}(dZ^{[l]} A^{[l-1]T}) \end{aligned}$$

Back-propagation with Neural Network

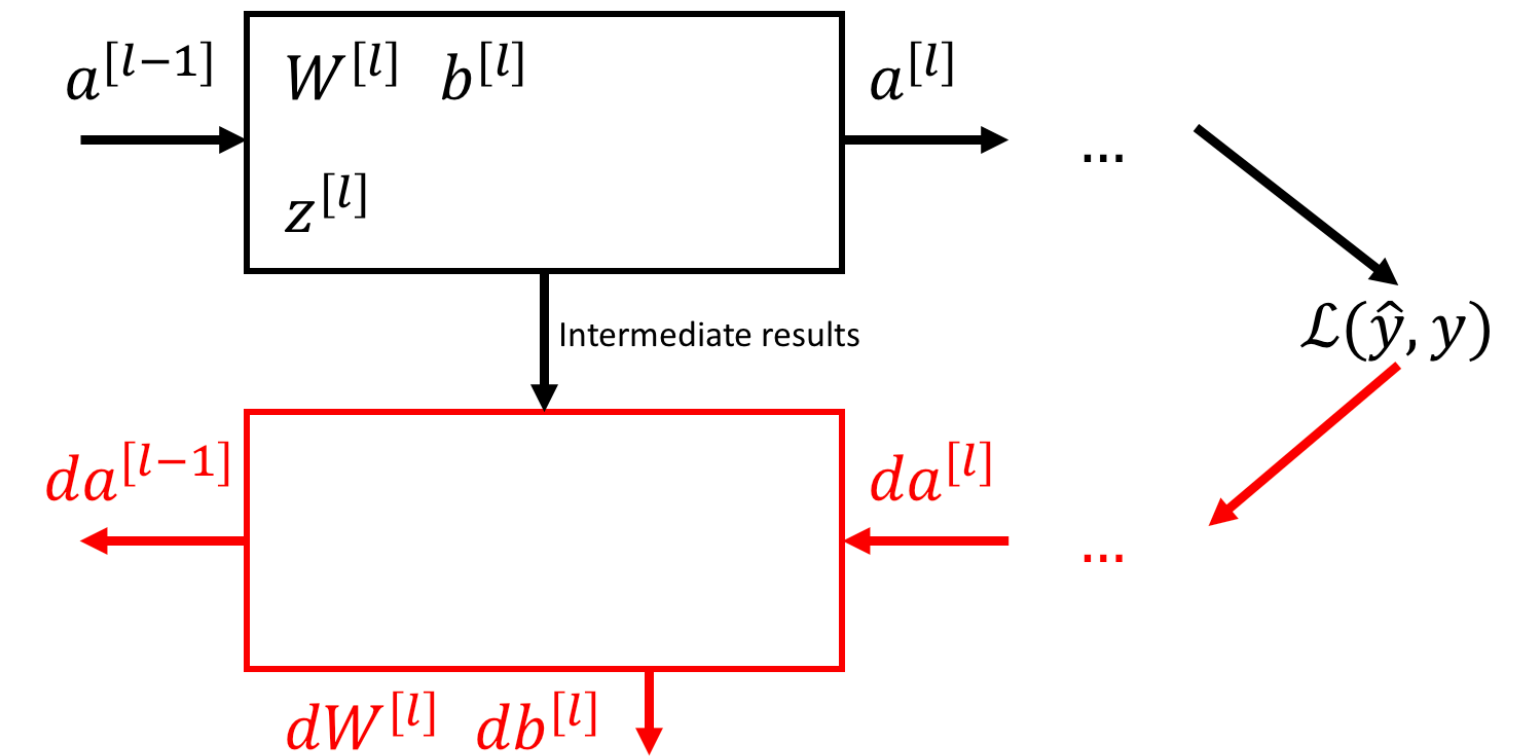
Shapes - Vectorized

```
def backward(self, da, a_prev, intermediate_results):
```

```
    W, b, z = intermediate_results    # From forward pass
    _, m = da.shape                    # Get number of samples - m
```

```
    # Find activation gradient for previous layer
    dz = np.multiply(self.activation.derivative(z), da)
    da_prev = np.matmul(W.T, dz)
```

```
    # Find gradients for weights and biases
    db = (1.0/m) * np.sum(dz, axis=1, keepdims=True)
    dW = (1.0/m) * np.matmul(dz, a_prev.T)
```



$$\begin{aligned}dZ^{[l]} &= g^{[l]'}(Z^{[l]}) \odot dA^{[l]} \\dA^{[l-1]} &= W^{[l]T} dZ^{[l]} \\db^{[l]} &= \text{mean}(dZ^{[l]}) \\dW^{[l]} &= \text{mean}(dZ^{[l]} A^{[l-1]T})\end{aligned}$$

2 Machine Learning Project

Machine Learning Project

Goal

- **What** is it that we need to accomplish ?

Machine Learning Project

Goal

- **What** is it that we need to accomplish ?

Identify breed of cats



Machine Learning Project

Goal

- **What** is it that we need to accomplish ?

Identify breed of cats

Classification



Machine Learning Project

Goal

- **What** is it that we need to accomplish ?

Identify breed of cats

Classification

Binary ?

Multi-class ?



Machine Learning Project

Metrics

- **How can we tell** if we are doing **better or worse** ?
- Metrics and Loss functions are not the same !

Machine Learning Project

Metrics

- **How can we tell** if we are doing **better or worse** ?
- Metrics and Loss functions are not the same !
 - We need a single quantity that can be put on a scale

Machine Learning Project

Metrics

- **How can we tell** if we are doing **better or worse** ?
- Metrics and Loss functions are not the same !
 - We need a single quantity that can be put on a scale

	Real True	Real False
Predicted True	TP	FP
Predicted False	FN	TN

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Machine Learning Project

Metrics

- **How can we tell** if we are doing **better or worse** ?
- Metrics and Loss functions are not the same !
 - We need a single quantity that can be put on a scale

	Real True	Real False
Predicted True	TP	FP
Predicted False	FN	TN

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Machine Learning Project

Metrics

- **How can we tell** if we are doing **better or worse** ?
- Metrics and Loss functions are not the same !
 - If multiple aspects are important, we need some kind of **compound score**

$$Precision = \frac{TP}{TP + FP}$$

	Real True	Real False
Predicted True	TP	FP
Predicted False	FN	TN

Machine Learning Project

Metrics

- **How can we tell** if we are doing **better or worse** ?
- Metrics and Loss functions are not the same !
 - If multiple aspects are important, we need some kind of **compound score**

	Real True	Real False
Predicted True	TP	FP
Predicted False	FN	TN

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Machine Learning Project

Metrics

- **How can we tell** if we are doing **better or worse** ?
- Metrics and Loss functions are not the same !
 - If multiple aspects are important, we need some kind of **compound score**

	Real True	Real False
Predicted True	TP	FP
Predicted False	FN	TN

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 \text{ Score} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

(harmonic mean of precision and recall)

Machine Learning Project

Metrics

- **How can we tell** if we are doing **better or worse** ?
- Metrics and Loss functions are not the same !
 - If multiple aspects are important, we need some kind of **compound score**
 - use **custom metrics** when standard ones don't match real costs
 - keep them simple, clearly defined, reproducible

	Real True	Real False
Predicted True	TP	FP
Predicted False	FN	TN

Machine Learning Project

Data

- **Validation Set** is of paramount importance !
- Represents the **real use-case**
- Can be difficult to obtain

Machine Learning Project

Data

- **Validation Set** is of paramount importance !
- Represents the **real use-case**
- Can be **difficult to obtain**

Make sure your validation data
**matches real-world conditions and
is labeled accurately and
consistently.**

Machine Learning Project

Data

- **Train Set** should be as close to the validation set as possible
- **Can** be different !
 - Downloaded from Internet
 - Different task
 - Synthetic
 - ...
- Cheap / Cheaper to obtain

Machine Learning Project

Data

- **Test Set** is typically not public
- Contains real data of your customers
 - Used for evaluation upon delivery

Machine Learning Project

Data



Training Data

Machine Learning Project

Data



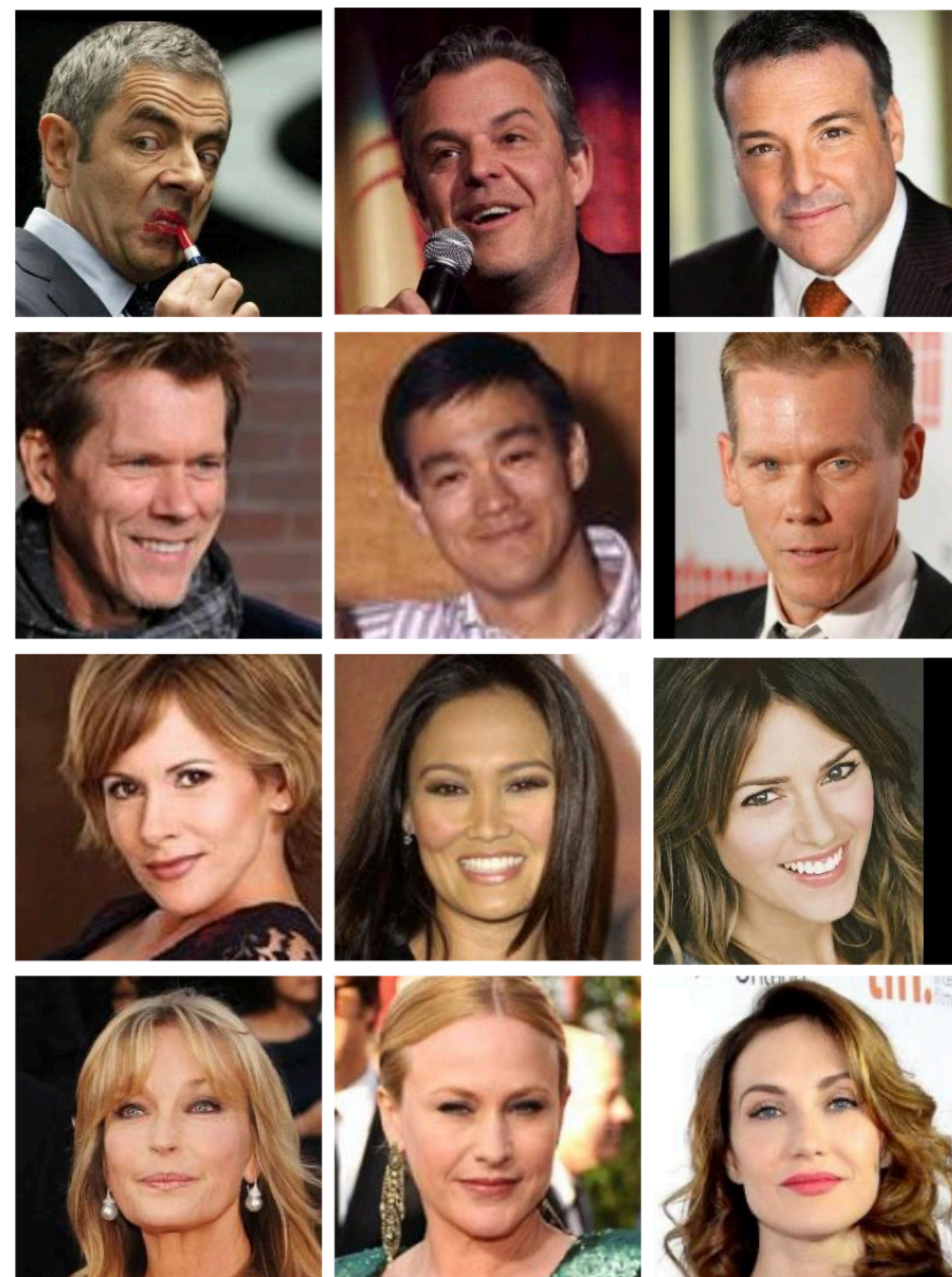
Training Data



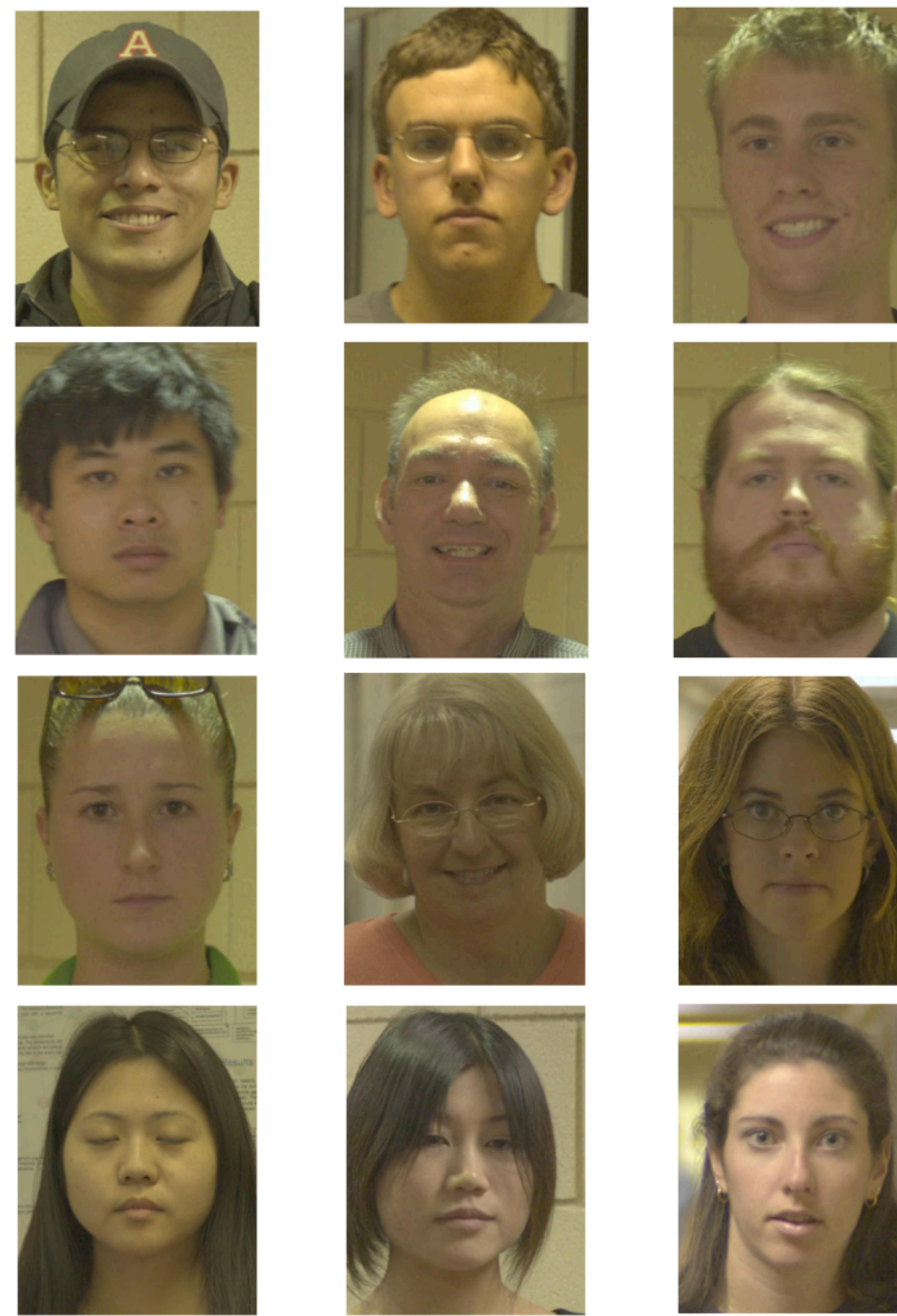
Customer Data

Machine Learning Project

Data - Domain Adaptation



(a) CASIA-Webface



(b) GBU



(c) IJB-A

Machine Learning Project

Data

- There is **no single best solution**
- Each and every use-case is different
- Trained model will perform better when production data is close to validation data
- During the training, validation set prevents overfitting on train set

Machine Learning Project

Model Architecture

- Any model with **sufficient capacity** should be able to learn **on train set**
- Depending on the task and how much data you have
 - Fully connected model
 - Convolutional model
 - Transformer-based model
 - ...
- Start with a small model, increase if necessary

Machine Learning Project

Training Process

- Decide which **training paradigm** is most suitable

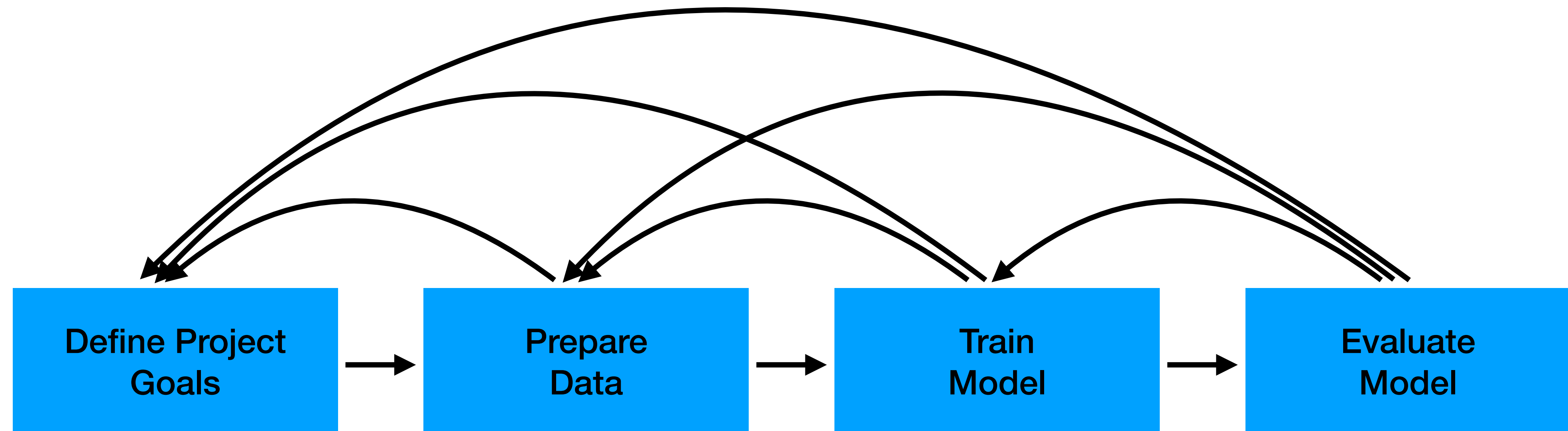
Machine Learning Project

Training Process

- Decide which **training paradigm** is most suitable
 1. Supervised learning
 2. Unsupervised learning
 3. Weakly/semi supervised learning
 4. Self-supervised learning
 5. Reinforcement learning

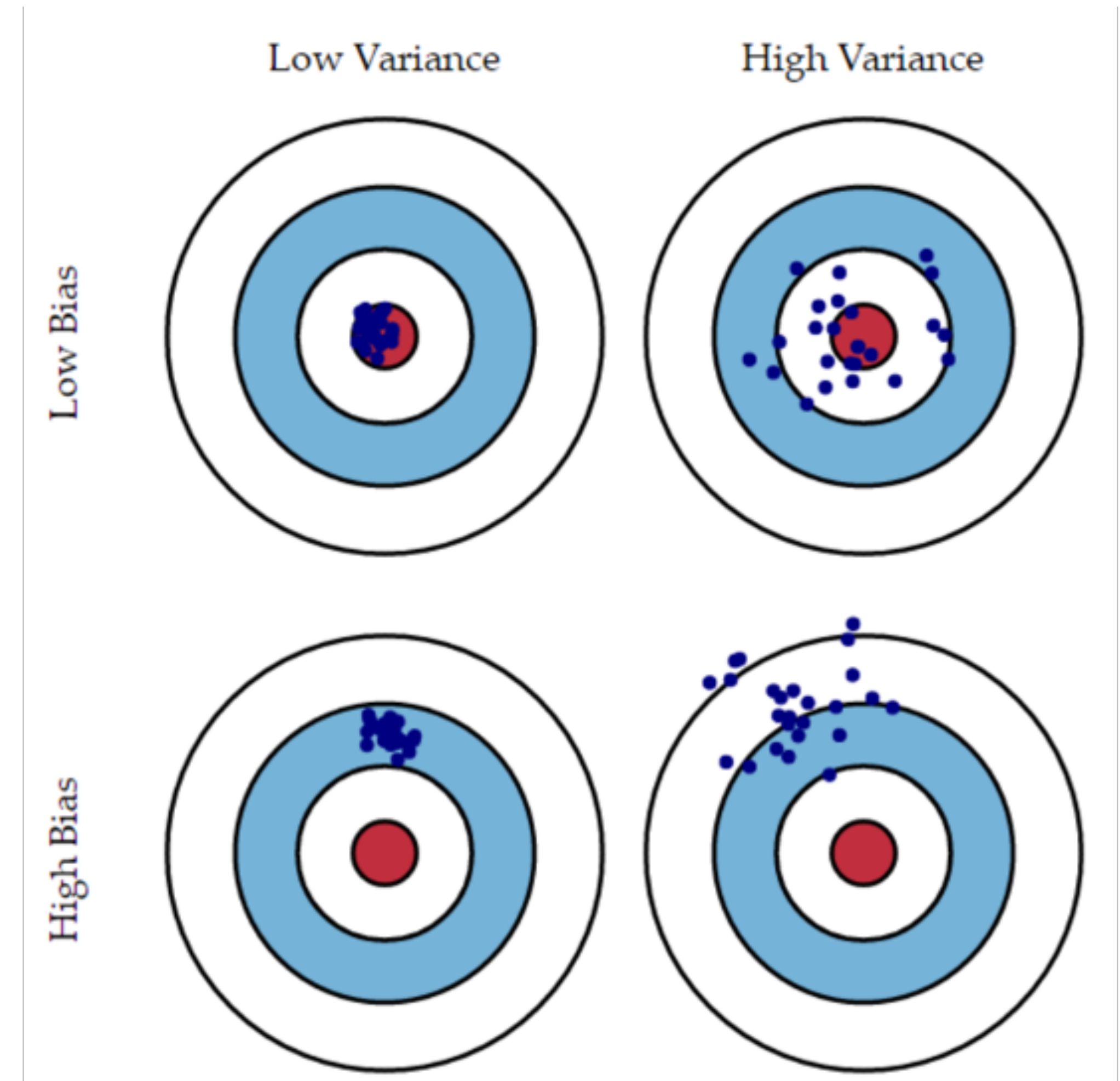
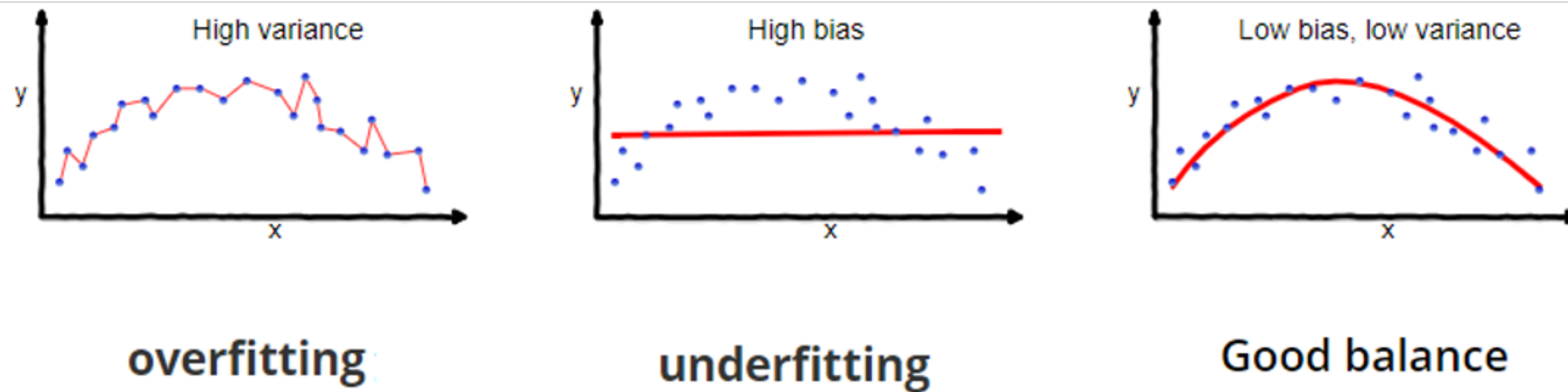
Machine Learning Project

Life Cycle



Machine Learning Project

Bias / Variance Trade-Off



Machine Learning Project

Bias / Variance

- Cat / Dog
 - Human Error ~ 0%
 - **Optimal Error**



Train Set Error	1 %	13 %	15 %	0.5 %
Validation Set Error	15 %	14 %	30 %	0.7 %
	High Variance	High Bias	High Bias High Variance	Low Bias Low Variance

Machine Learning Project

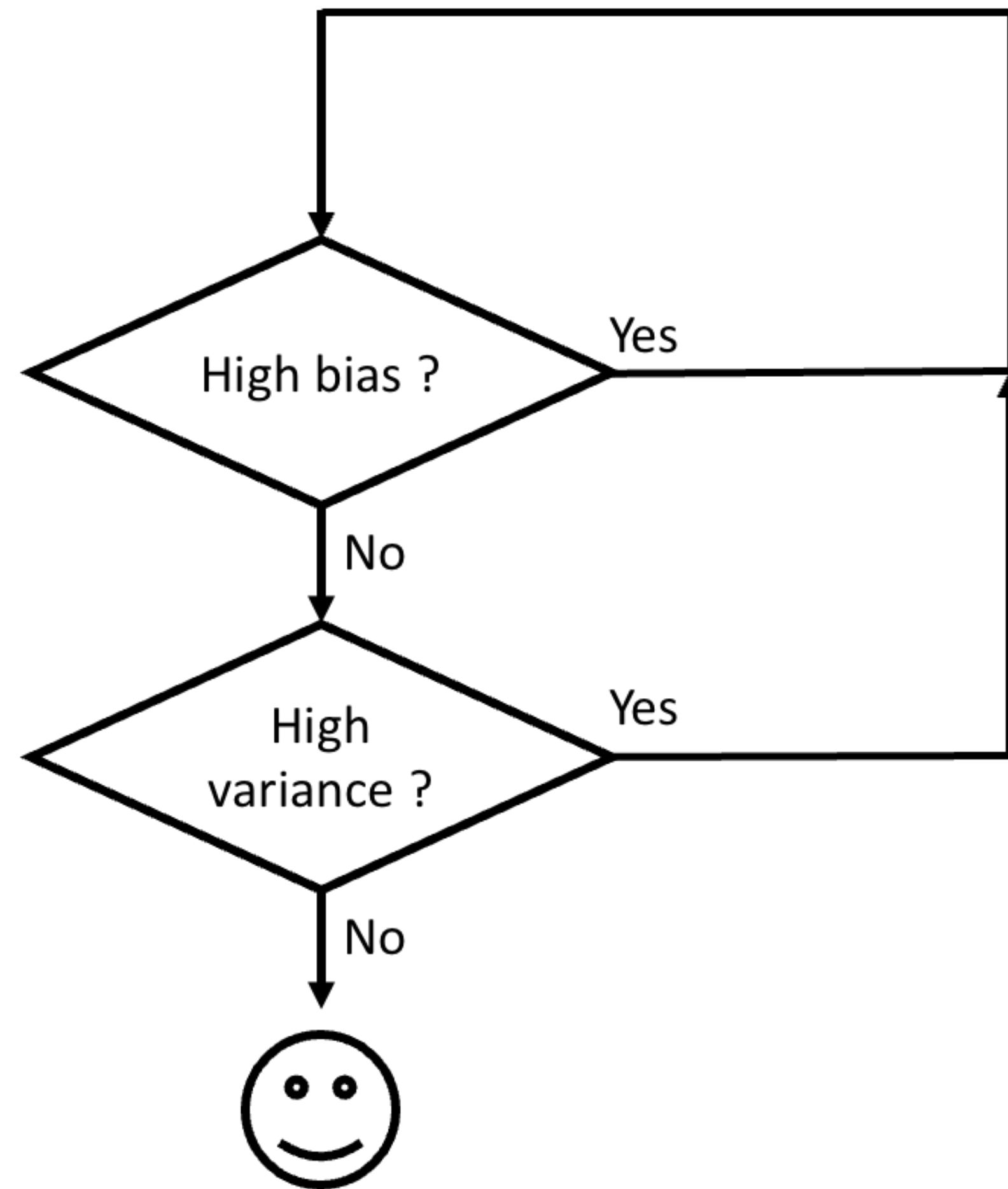
Bias / Variance

- Muffin / Chihuahua
 - Human Error ~ 12%
 - Optimal Error



Train Set Error		13 %	15 %	
Validation Set Error		14 %	30 %	
		Low Bias Low Variance	Low Bias High Variance	

Machine Learning Project

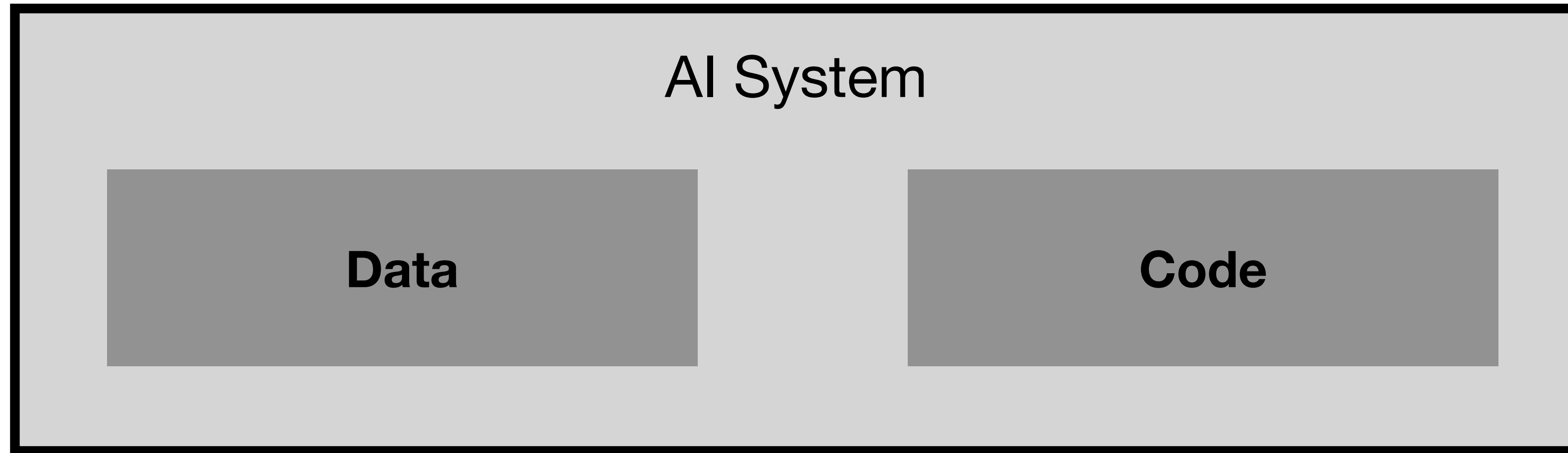


- Bigger model
- More suitable architecture
- Train longer
- Different hyper-parameters

- More data
- Regularization
- More suitable architecture

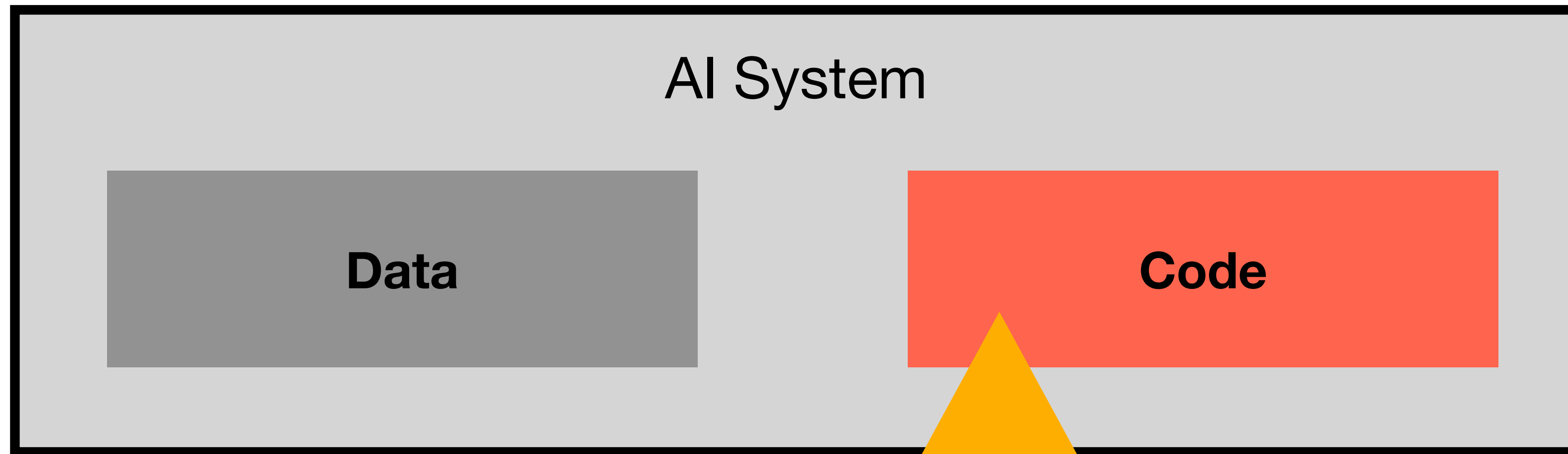
Machine Learning Project

Model Centric vs. Data Centric



Machine Learning Project

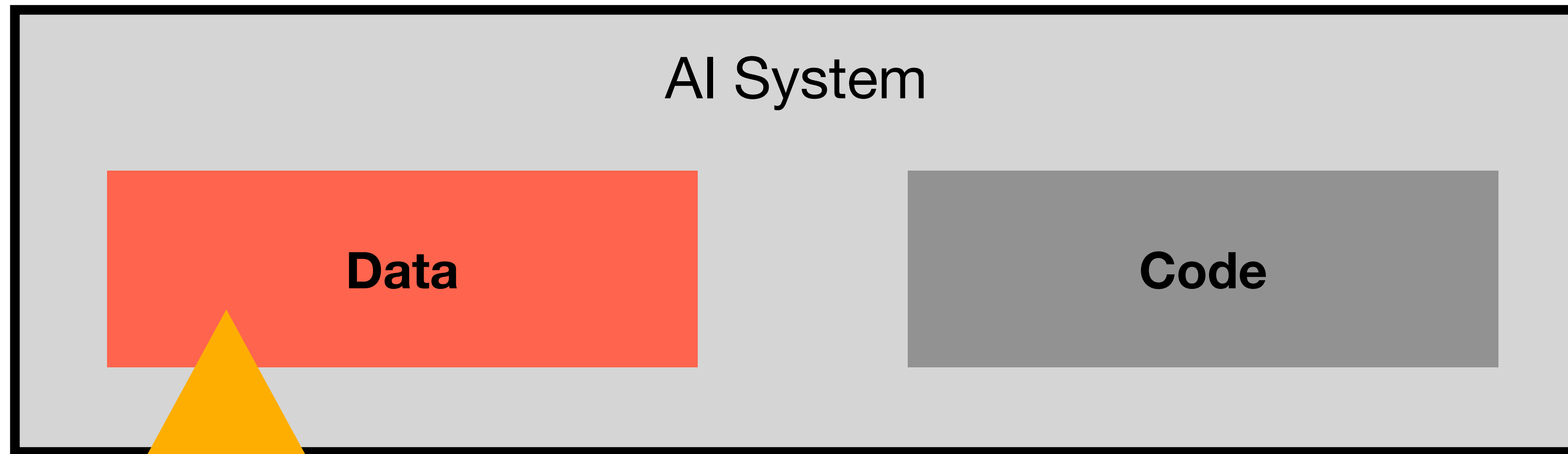
Model Centric vs. Data Centric



How can we change the code (model)
to improve the performance ?

Machine Learning Project

Model Centric vs. Data Centric



How can we change the data
(inputs and labels)
to improve the performance ?

Machine Learning Project

Summary

- We need to have a **clear goal**
- We need proper **metrics** to tell us which solution is better
- **Validation data represents the intended use-case**
- Be aware of differences between train and validation sets
- Start with a smaller model
- Improve in iterations
- Consider improving both - model and data

3 NN Training Techniques

Training Techniques

Paradigms

1. Supervised learning
2. Unsupervised learning
3. Weakly/semi supervised learning
4. Self-supervised learning
5. Reinforcement learning

Paradigms

Supervised Learning

- For each input x_i we have ground truth annotation y_i
- Loss - measures error between predicted $\hat{y}_i = f(x_i)$ and y_i
- Regression, Classification

Paradigms

Unsupervised Learning

- We only have inputs x_i with no annotations
- Learns patterns and structure from unlabeled data
- Self-organizing Maps
- Principal Component Analysis (PCA)
- Clustering
- Auto-Encoder

Paradigms

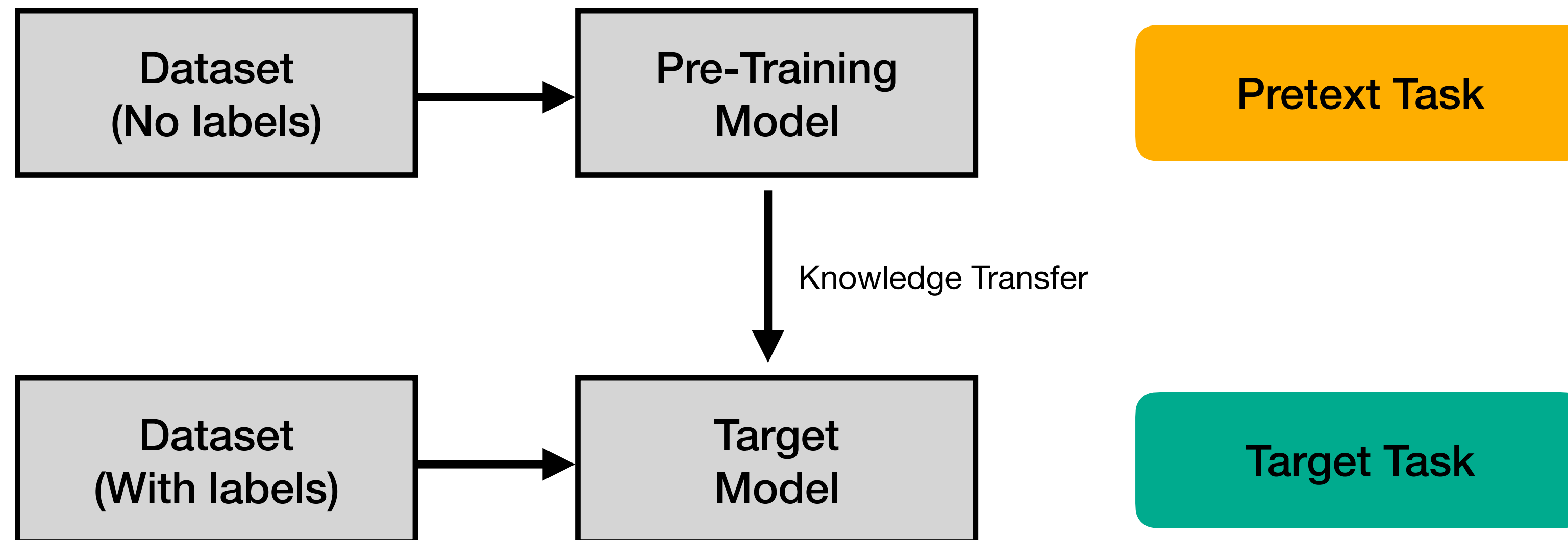
Weakly Supervised Learning

- We have **small amount** of data x_i **with labels** y_i
- We have **large amount** of data x_i with **annotations of lower quality/no annotations**
 - E.g. for segmentation we only have bounding boxes
 - Noisy data - unreliable labels
 - Pseudo-labels - annotations produced by different model (not humans)
- Many different training strategies - alternating supervised/other ...

Paradigms

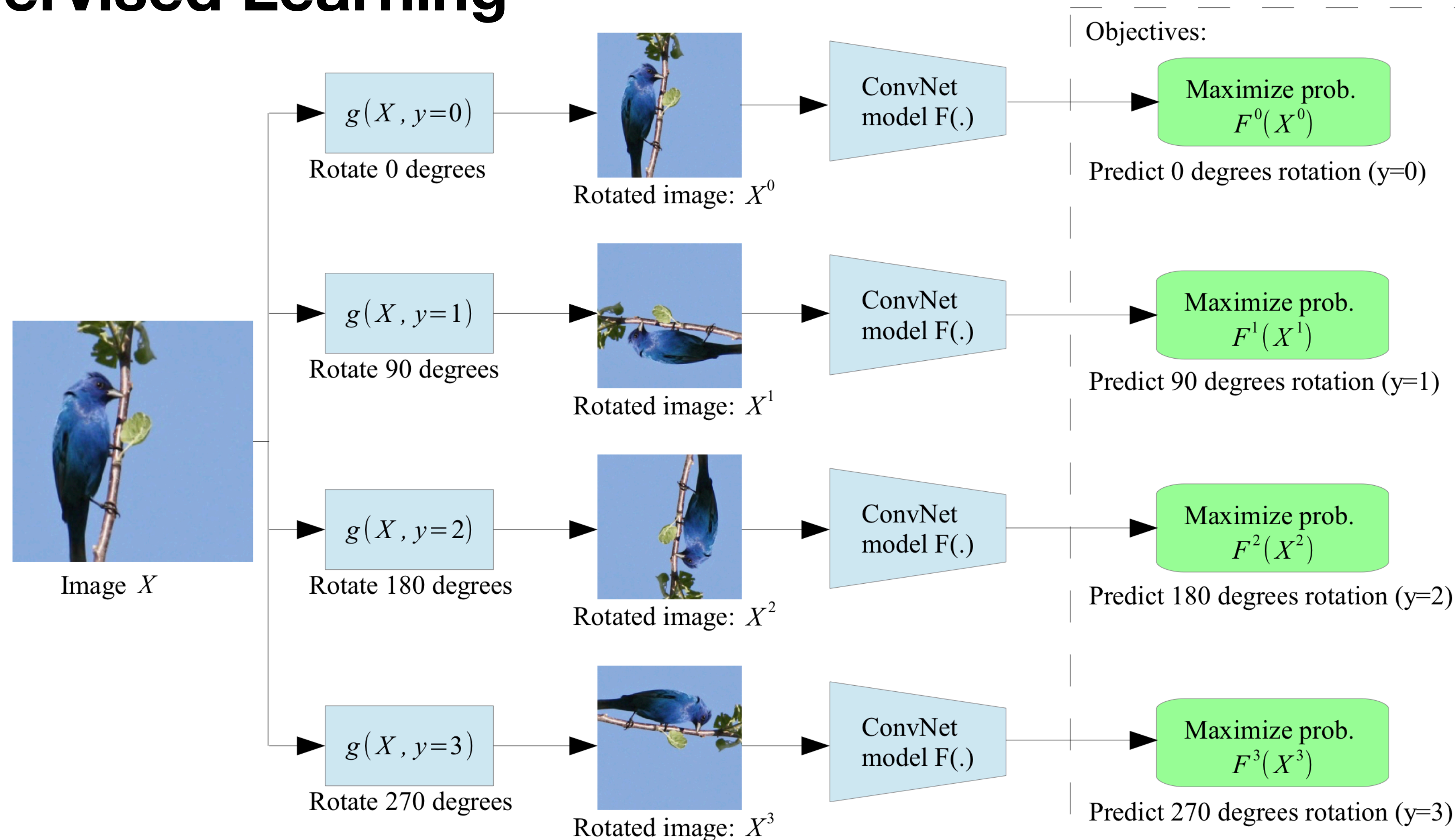
Self-Supervised Learning

- We have **large amount** of data x_i with **no annotations**
- We have **small amount** of data x_i **with labels** y_i



Paradigms

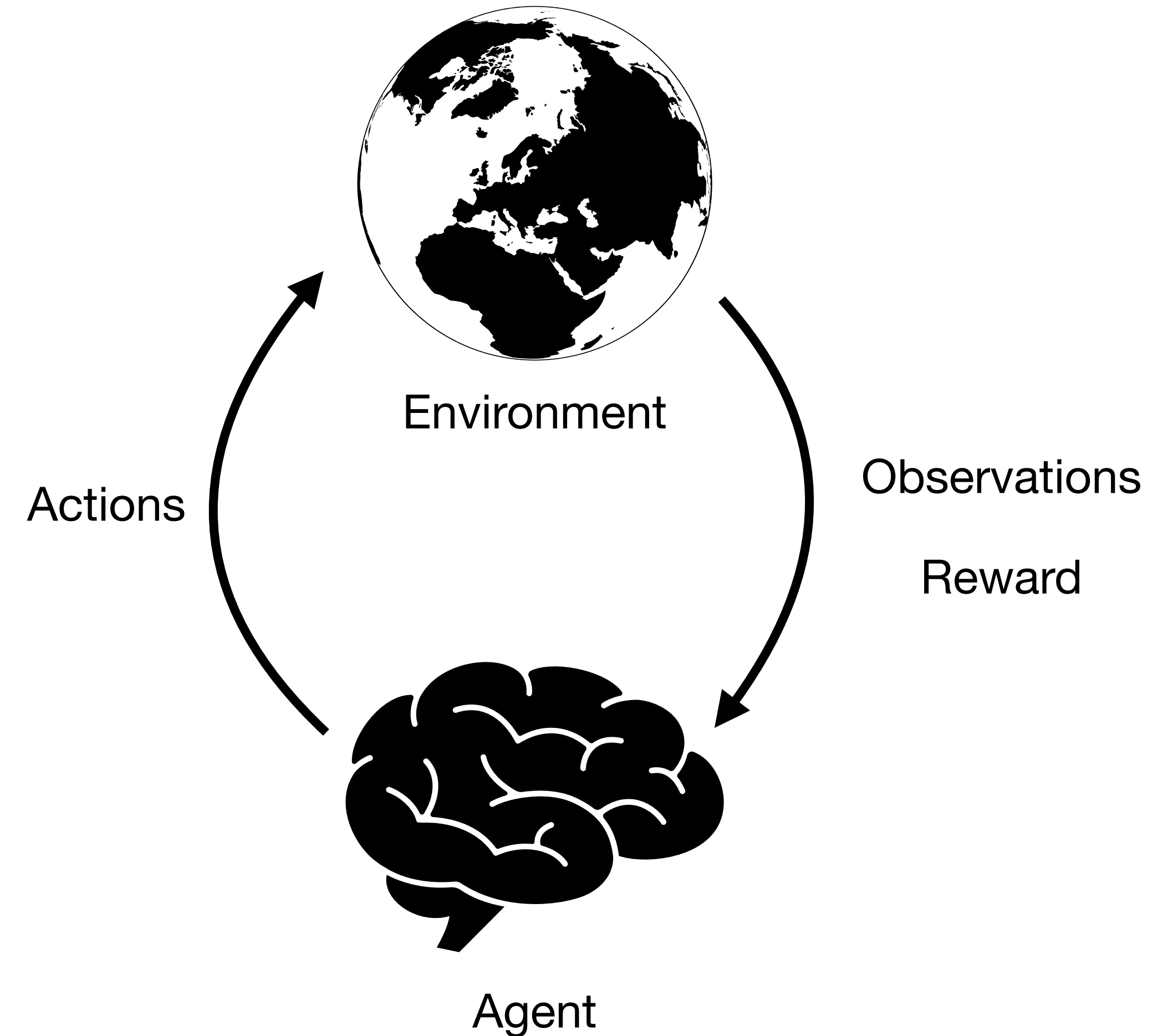
Self-Supervised Learning



Paradigms

Reinforcement Learning

- Agent learns through **interaction** with the Environment
- Goal is to maximize the **total reward**



Training Techniques

Training Loop - Definition of Terms

- **Gradient descent** - algorithm, iterate in small steps until convergence
- **Dataset** - too big to fit in memory, broken down into smaller mini-batches
- **(Mini-)Batch** - subset of train set, contains m training samples
- **Iteration** - one step of computation
 - forward pass, backward pass, update parameters
 - computed on mini-batches (vectorized computation)
- **Epoch** - one complete pass of iterations (over all mini-batches from train set)

Training Techniques

Batch Size

1. Stochastic descent

- batch size = 1

2. Mini-batch descent

- batch size > 1

3. Batch descent

- batch size = $M = |\mathcal{X}|$

	Batch Size	Steps / Epoch
Batch descent	M	1
Mini-batch descent	e.g., BS = 128	M / BS
Stochastic descent	1	M

Training Techniques

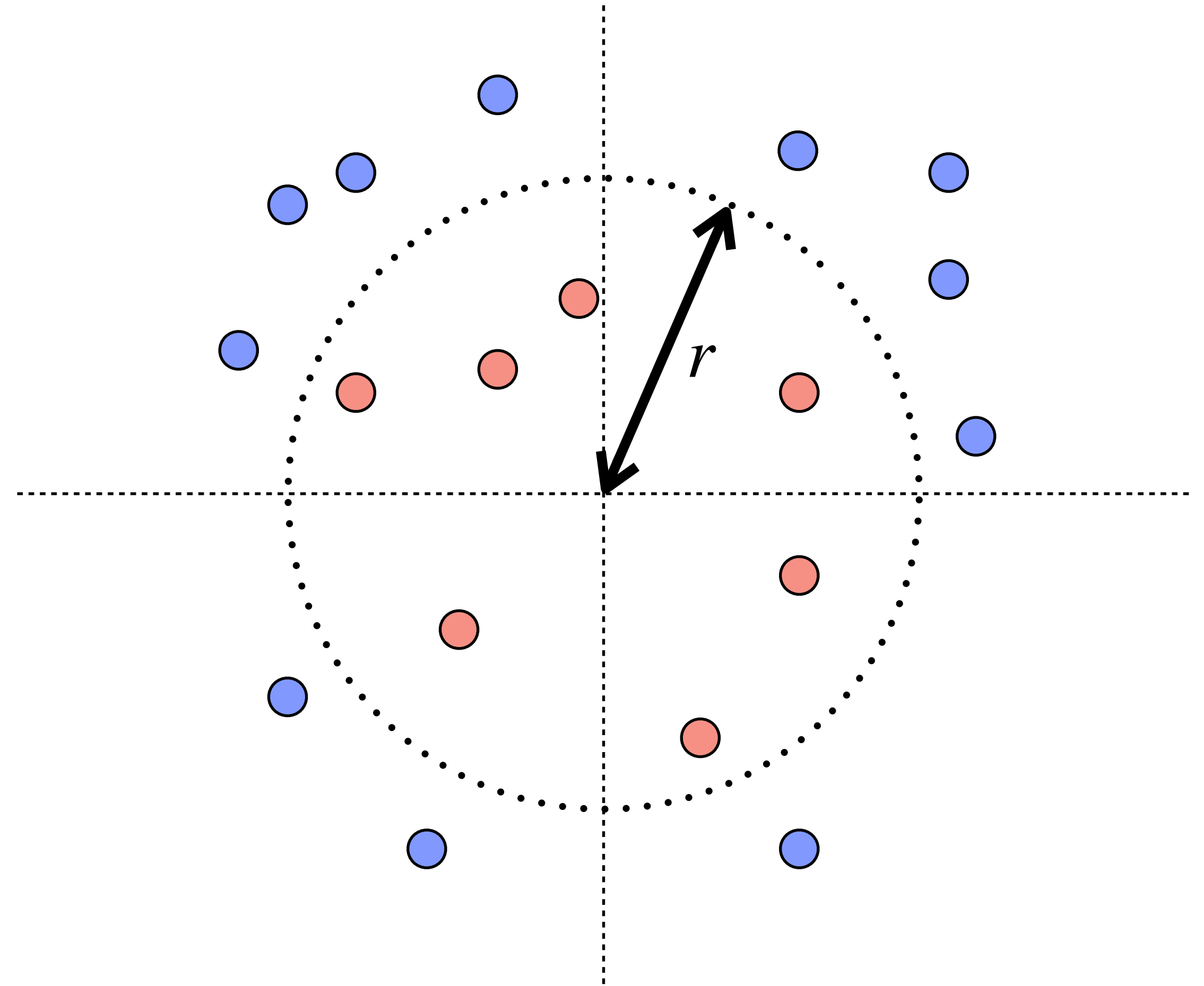
Batch Size

- Training - **Samples in Mini-batch must be chosen randomly**
- Validation - Order of samples does not matter

Example Toy Project

Example Toy Project

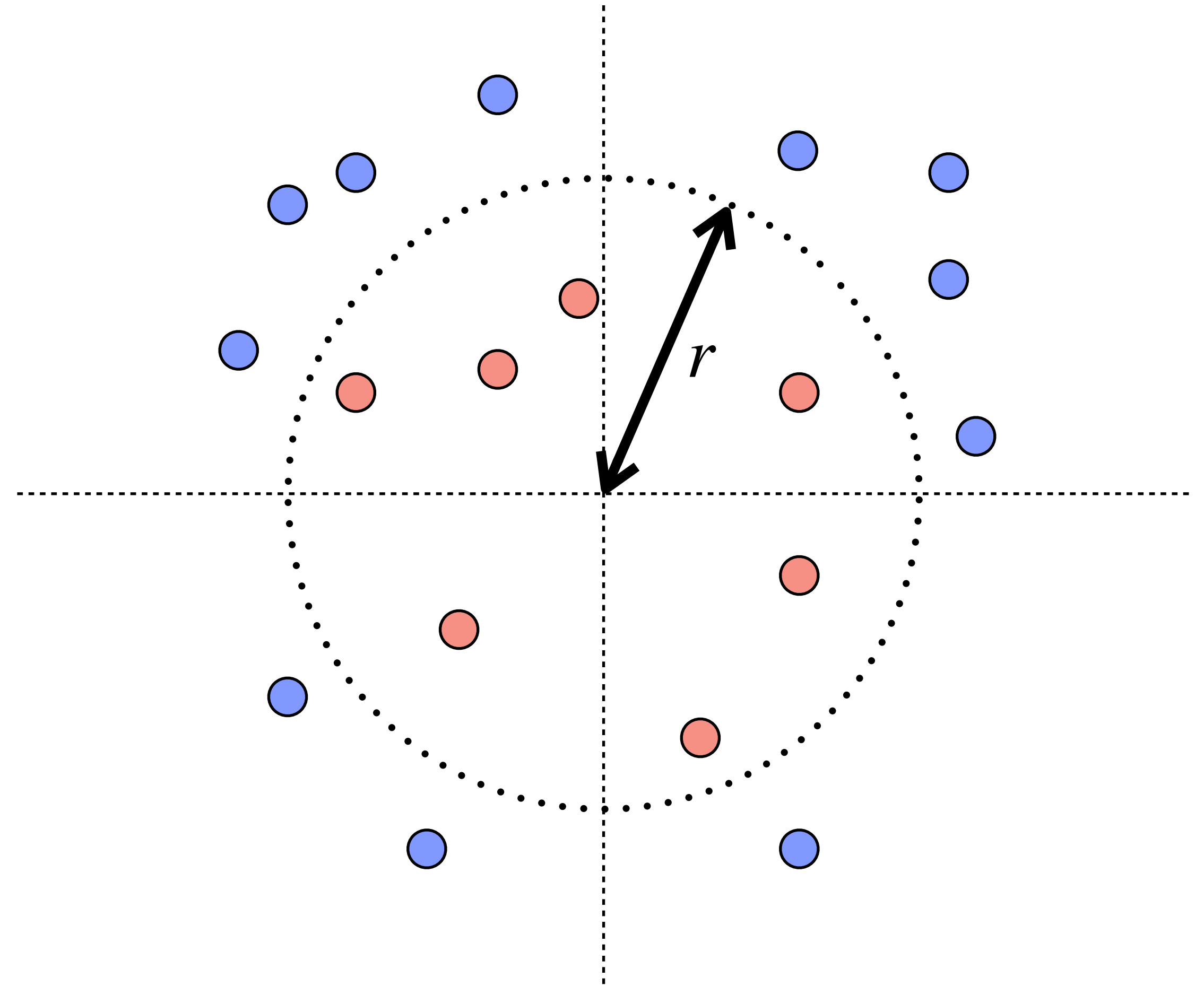
- **Goal** - What are we going to do ?



Example Toy Project

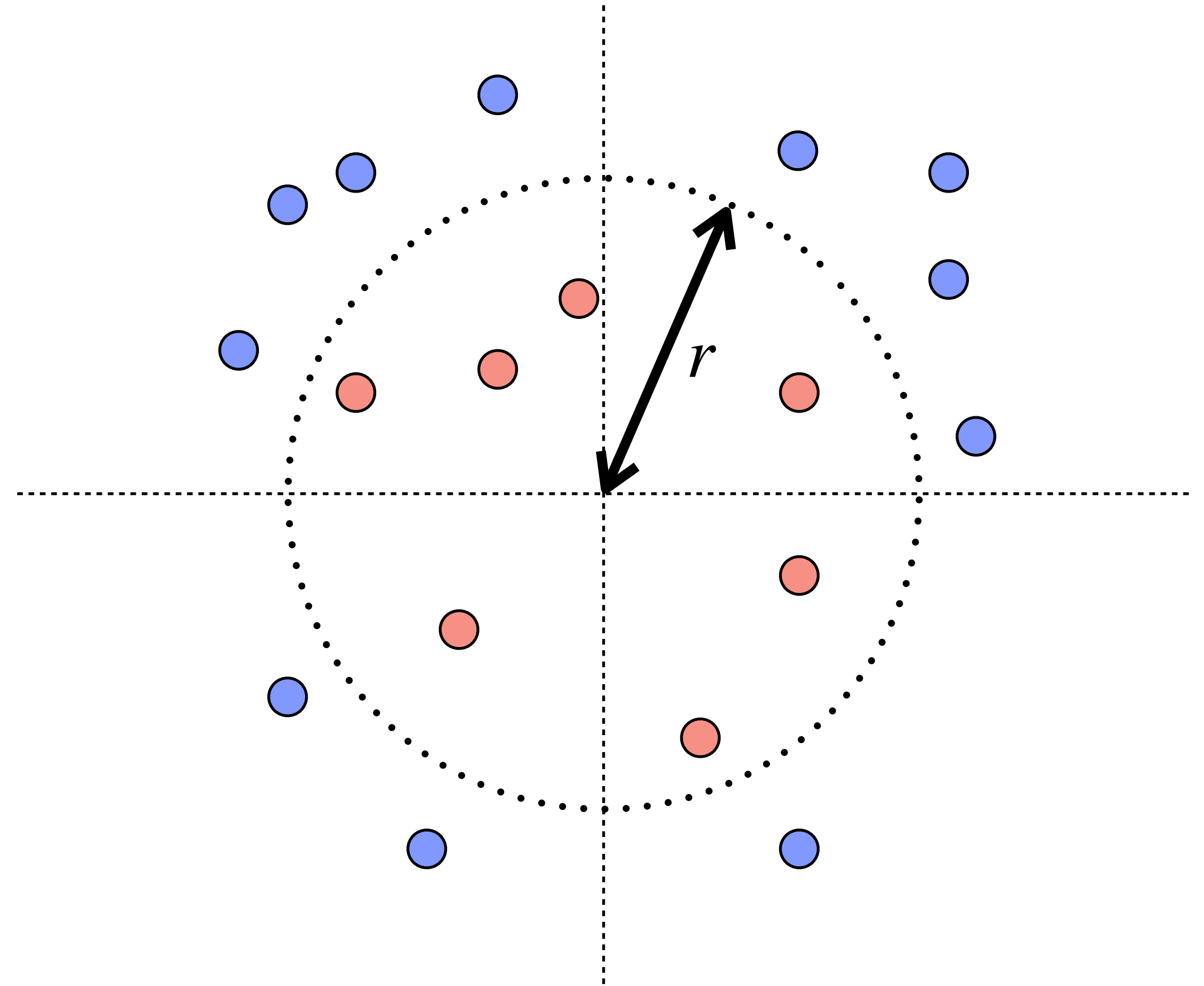
- **Goal** - What are we going to do ?

Binary Classification



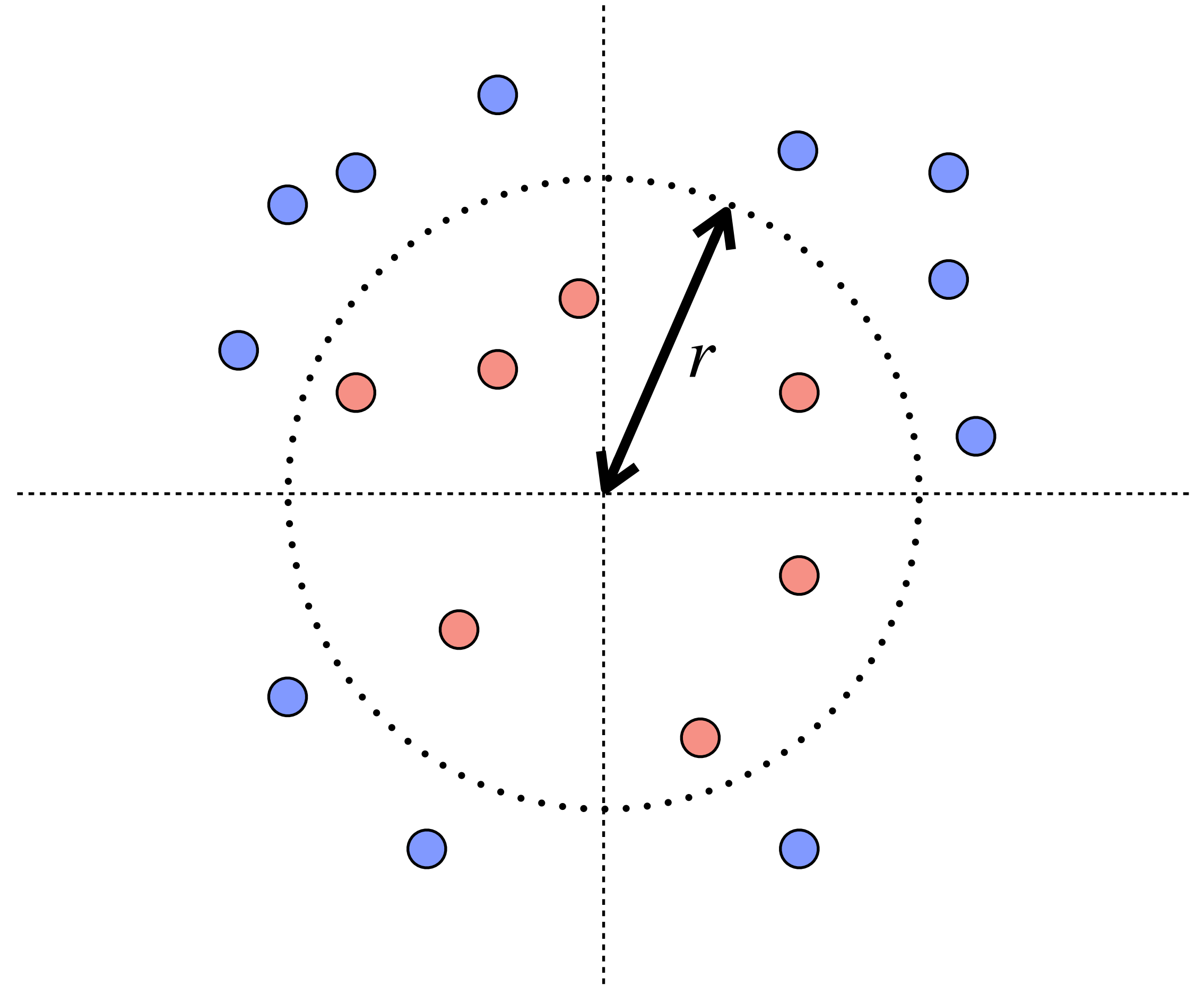
Example Toy Project

- **Goal** - Binary Classification
- **Metric** - Accuracy



Example Toy Project

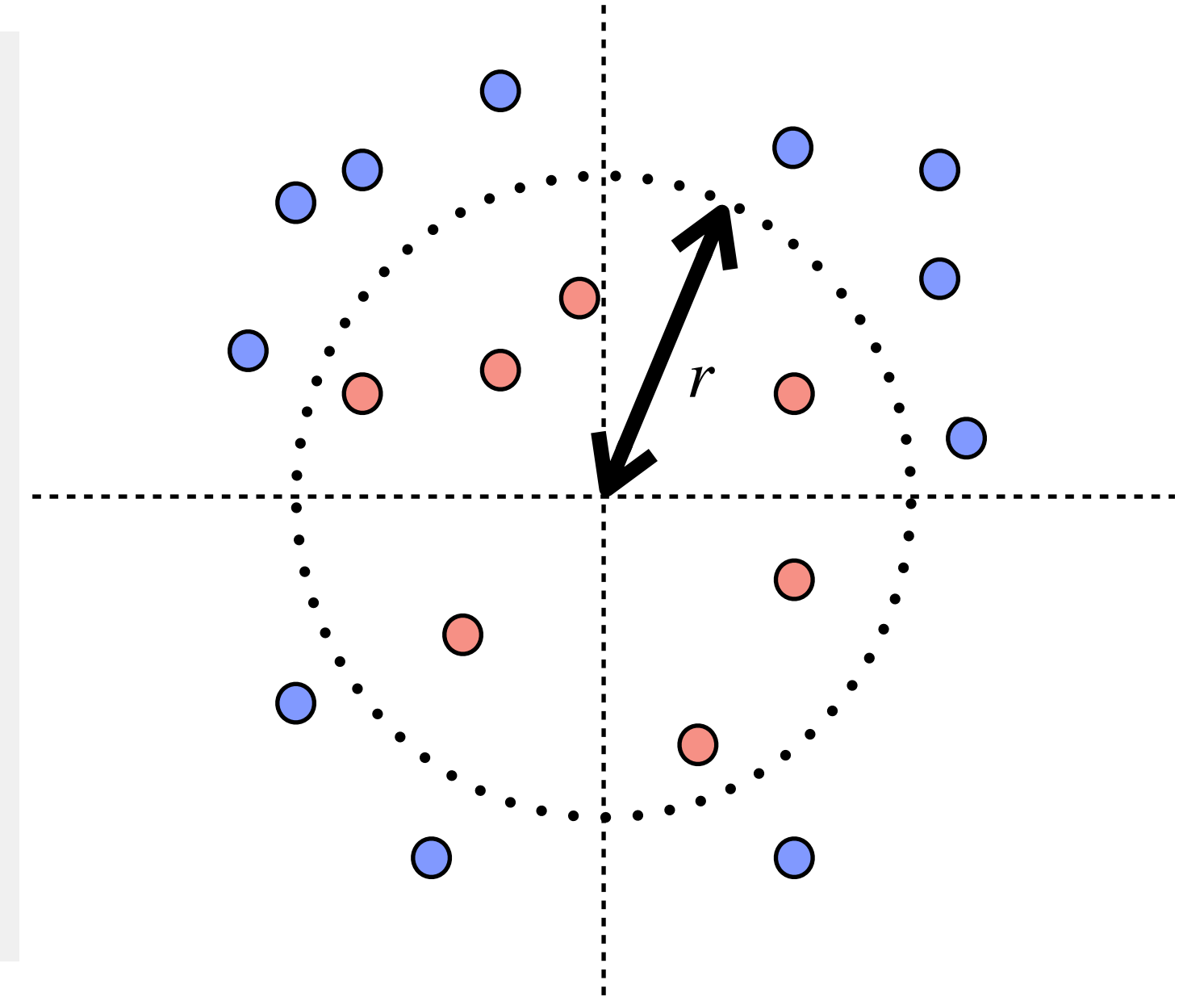
- **Goal** - Binary Classification
- **Metric** - Accuracy
- **Loss** - Binary Cross-Entropy



Example Toy Project

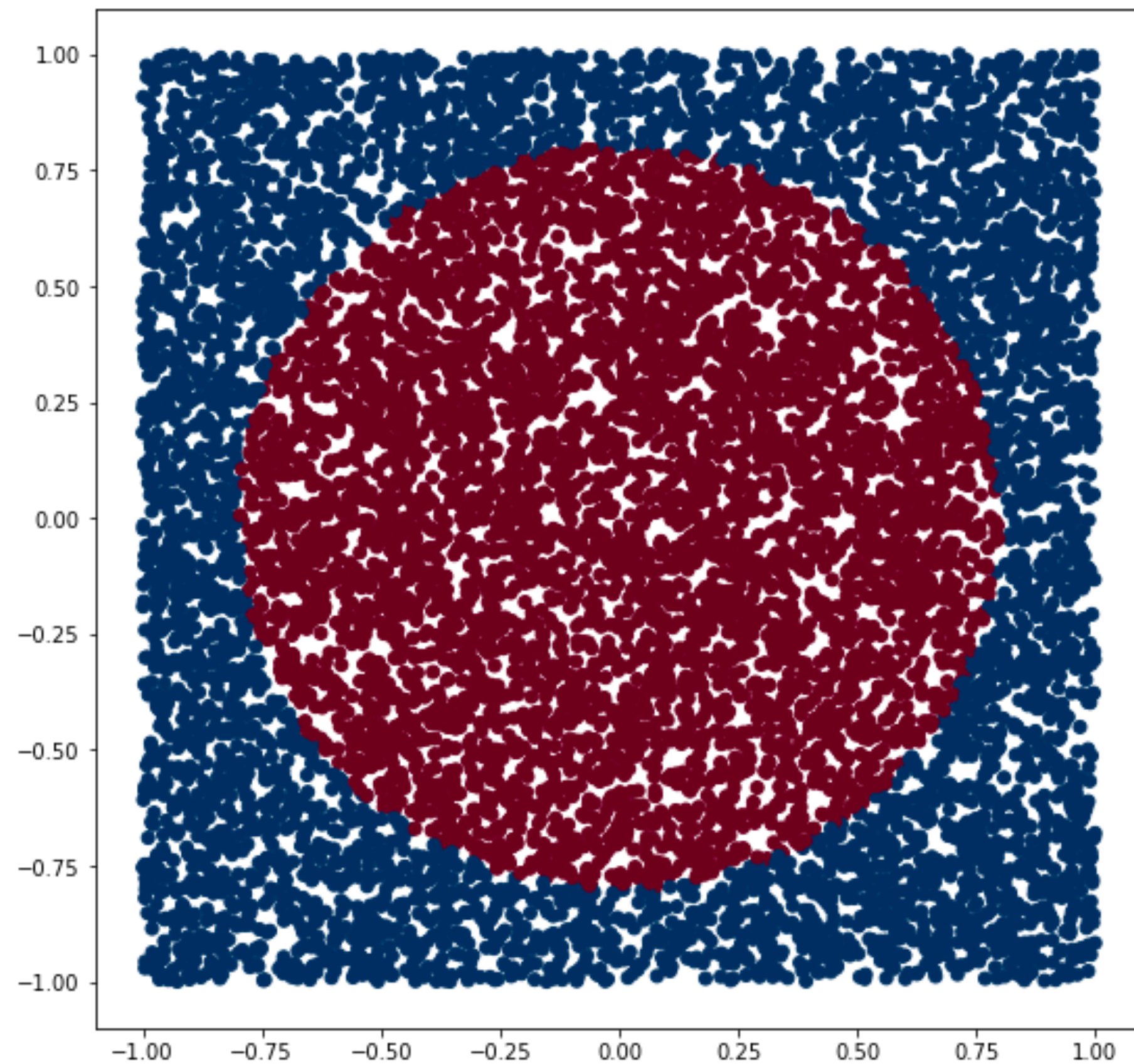
Data

```
def dataset_Circles(m=10, radius=0.7, noise=0.0):  
    # m samples of 2D point  
    X = (np.random.rand(2, m) * 2.0) - 1.0  
    N = (np.random.rand(2, m) - 0.5) * noise  
    # compute radius taking noise into account  
    R = np.sqrt(np.sum((X+N)**2), axis=0, keepdims=True))  
    Y = (R > radius).astype(float)  
    return X, Y
```

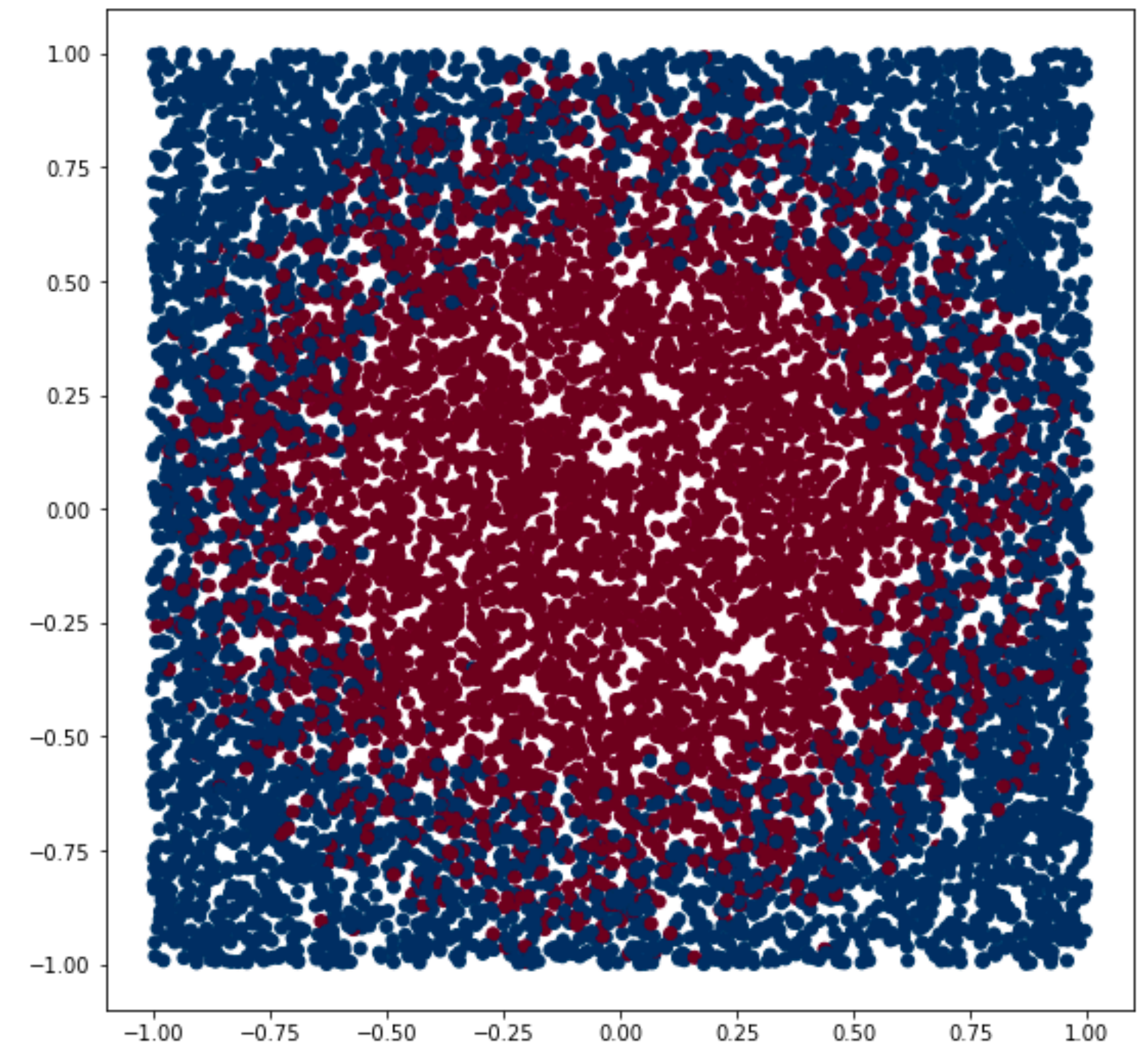


Example Toy Project Data

```
X, Y = dataset_Circles(m=10000, radius=0.8, noise=0.0)
```

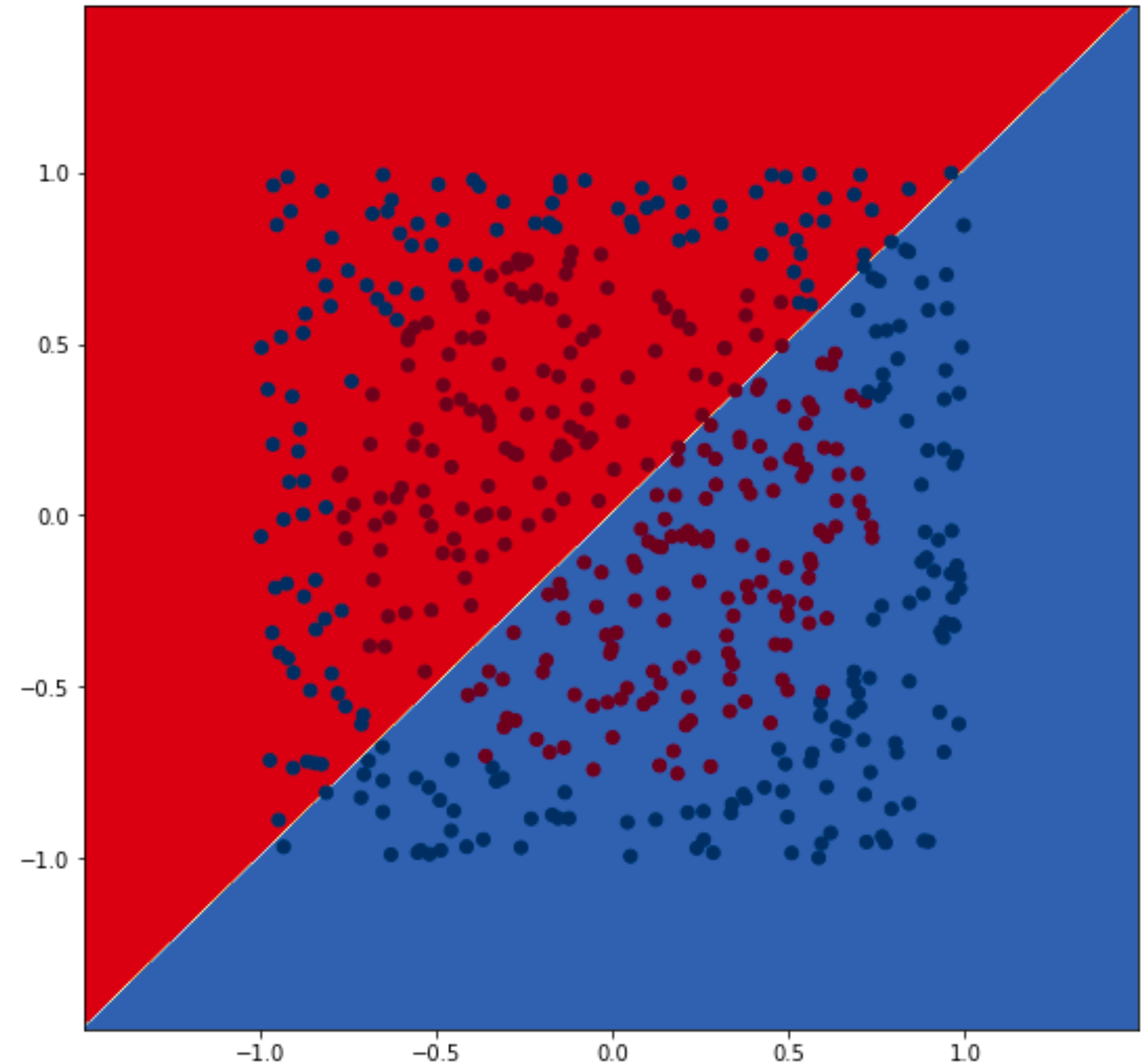


```
X, Y = dataset_Circles(m=10000, radius=0.8, noise=0.5)
```



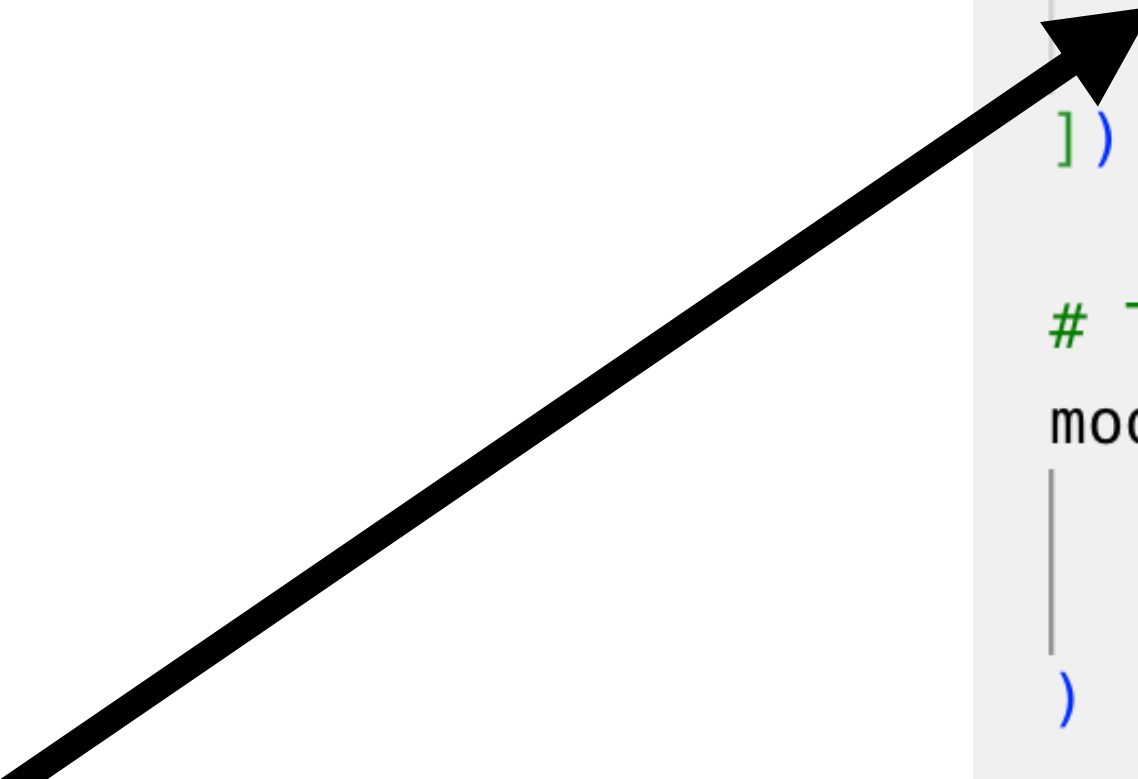
Example Toy Project Visualization

```
def testModel(X):  
    x1 = X[0, None]  
    x2 = X[1, None]  
  
    yhat = (x1 > x2).astype(float)  
    return yhat  
  
# Visualize a small dataset  
X, Y = dataset_Circles(m=500, radius=0.8, noise=0.0)  
draw_DecisionBoundary(X, Y, testModel)
```



Example Toy Project

- **Goal** - Binary Classification
- **Metric** - Accuracy
- **Loss** - Binary Cross-Entropy
- **Model Architecture** - MLP
 - 1 hidden layer - 6 units
 - 1 output layer - 1 unit
- **Learning rate** = 0.01



```
# Training and Validation Dataset
X, Y = dataset_Circles(m=1000)
devX, devY = dataset_Circles(m=200)

nx, m = X.shape
ny, _ = Y.shape

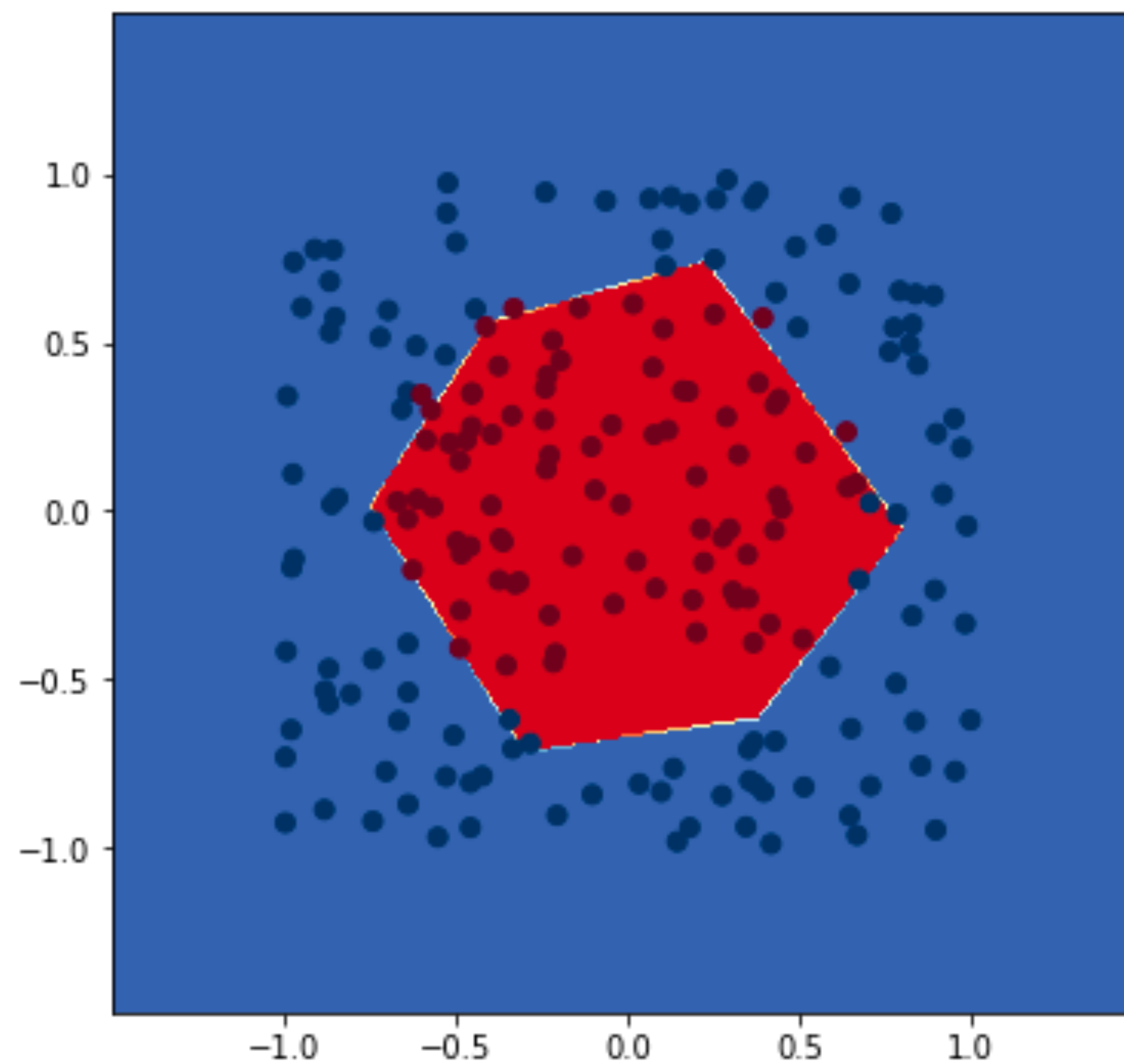
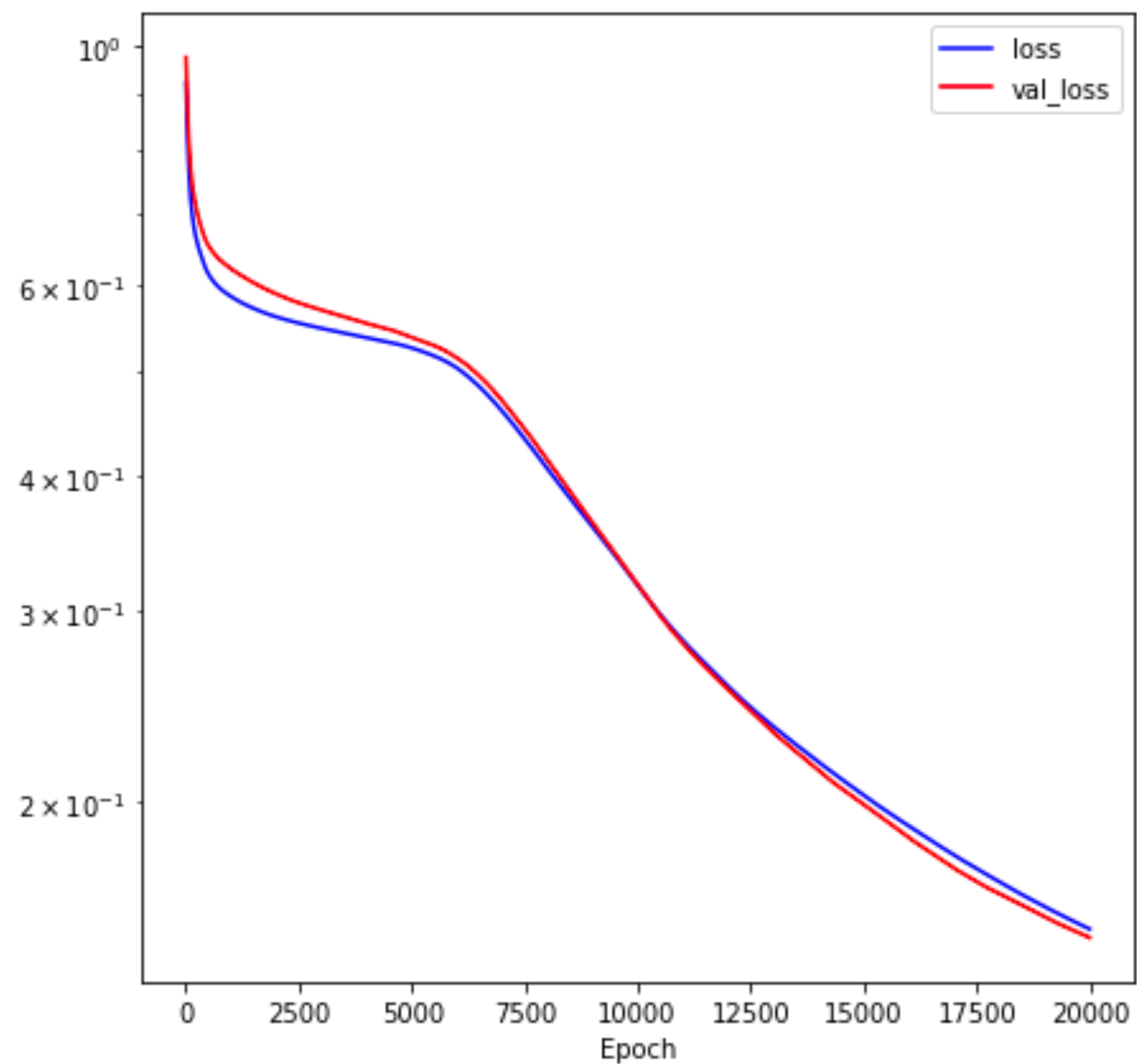
# Model architecture
model = Model([
    Input(nx),
    Dense(6, act='relu'),
    Dense(ny, act='sigmoid')
])

# Train with binary cross-entropy loss
model.initialize(
    loss=CreateLossFunction('bce'),
    optimizer=GradientDescent(0.01)
)

# Train for 20,000 epochs
results = model.train(X, Y, 20000, 0, devX, devY)

# Visualize
draw_TrainingResults(results)
draw_DecisionBoundary(devX, devY, model)
```

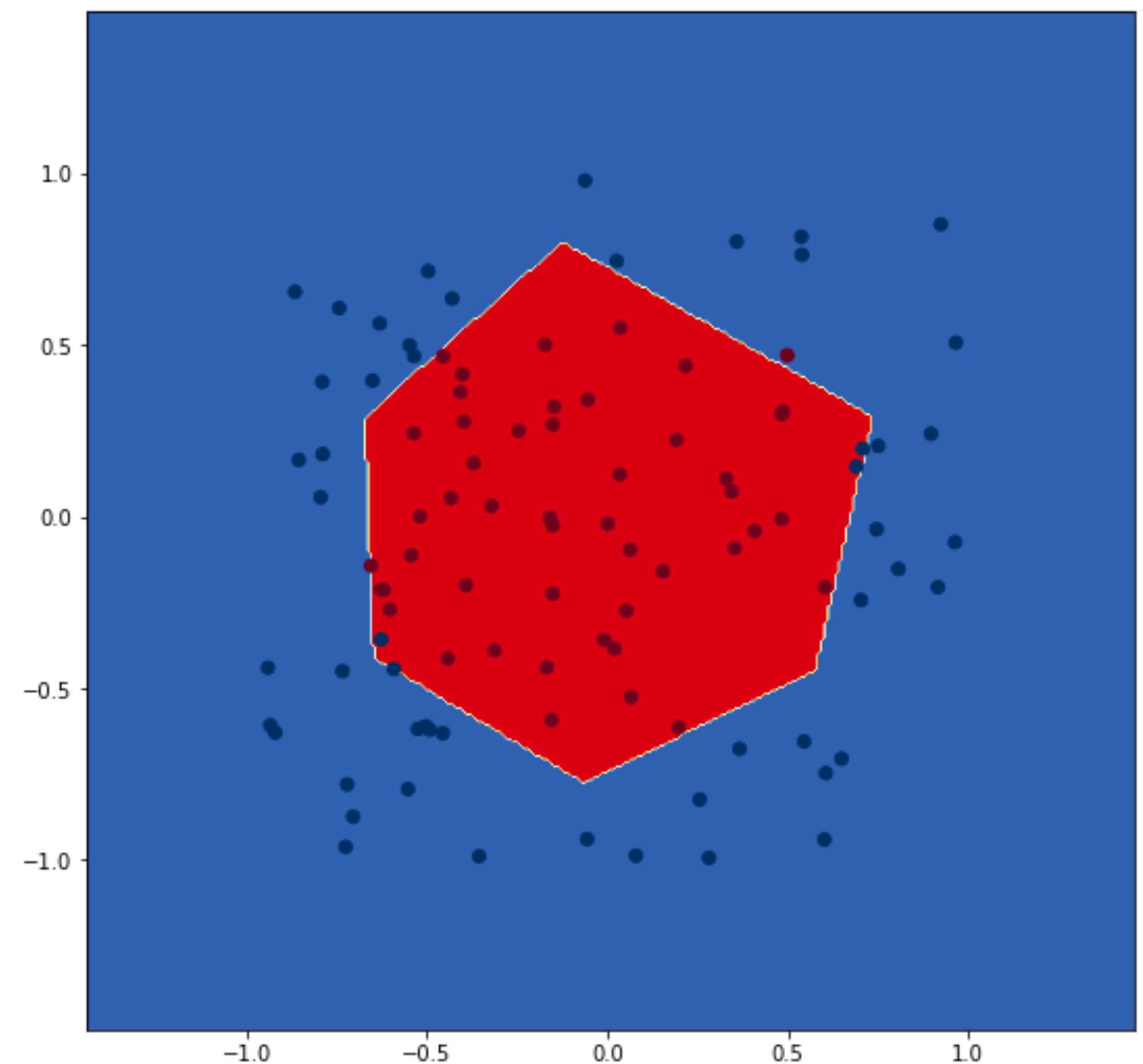
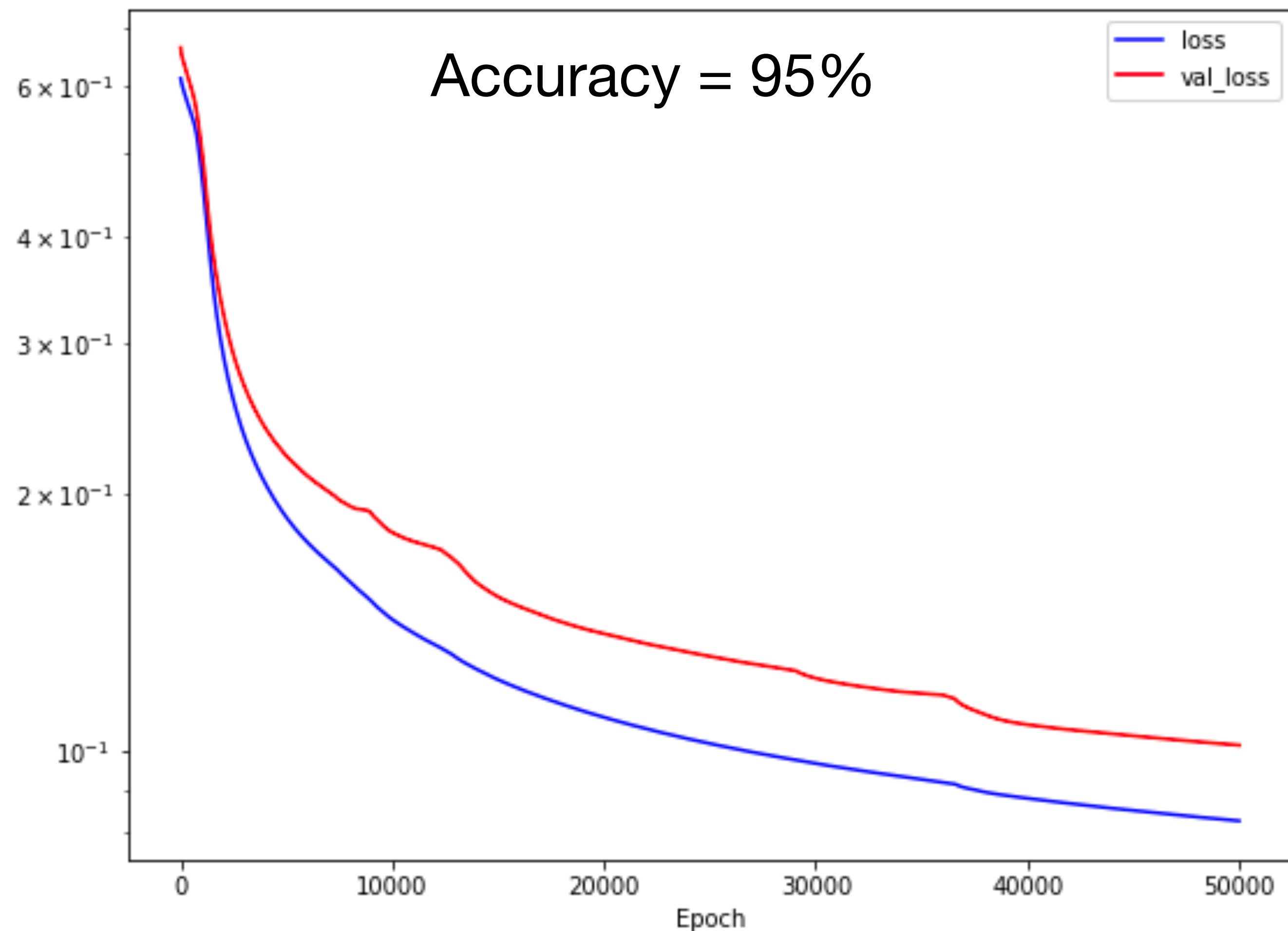
Example Toy Project



Example Toy Project

Underfit

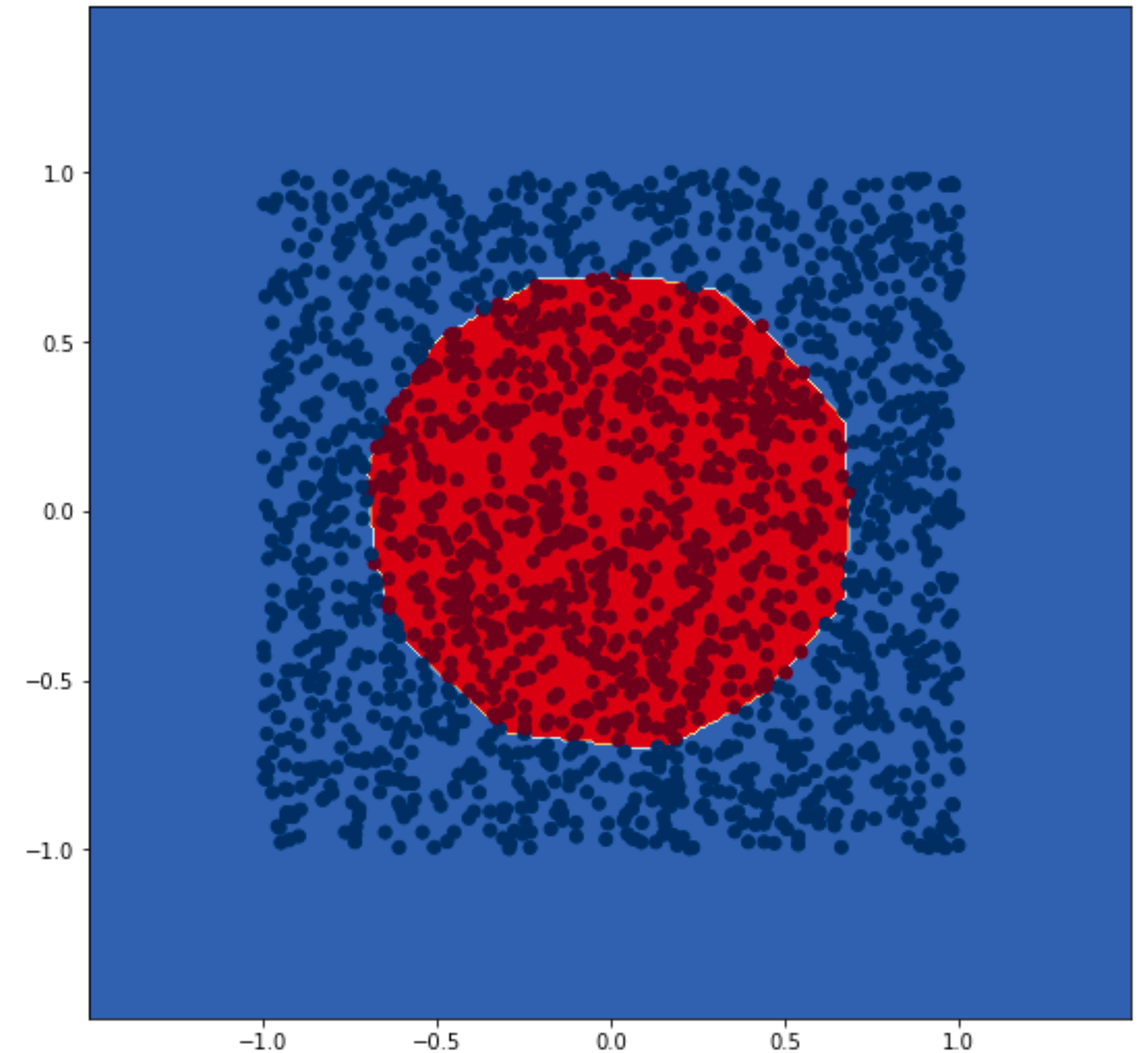
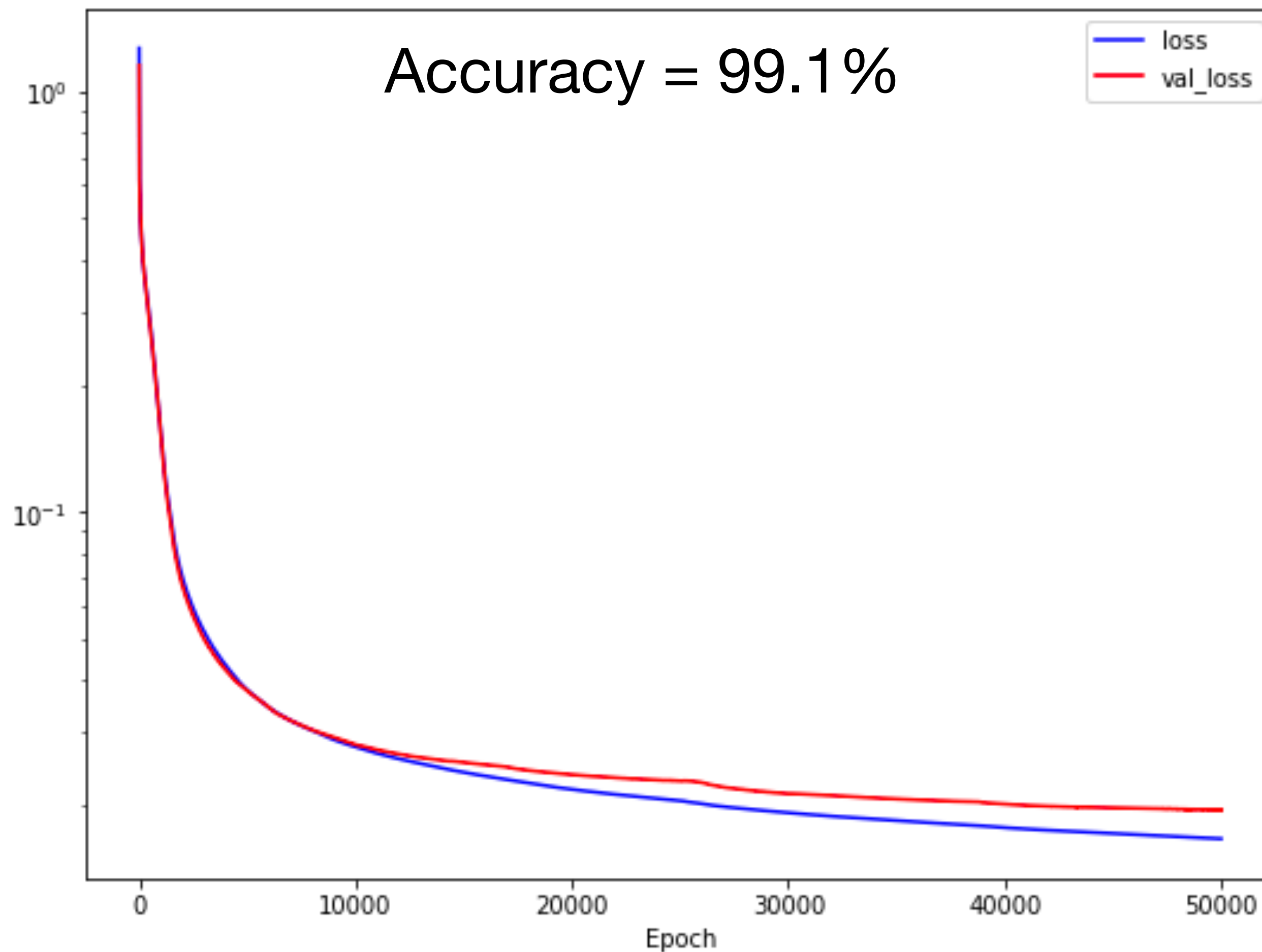
```
layers = [  
    Input(2),  
    Dense(3, act='relu'),  
    Dense(1, act='sigmoid')  
]  
  
# Train underfit scenario  
test_UnderfitOverfit(mTrain=500, mDev=100, noise=0.0, layers=layers)
```



Example Toy Project

“Just Right”

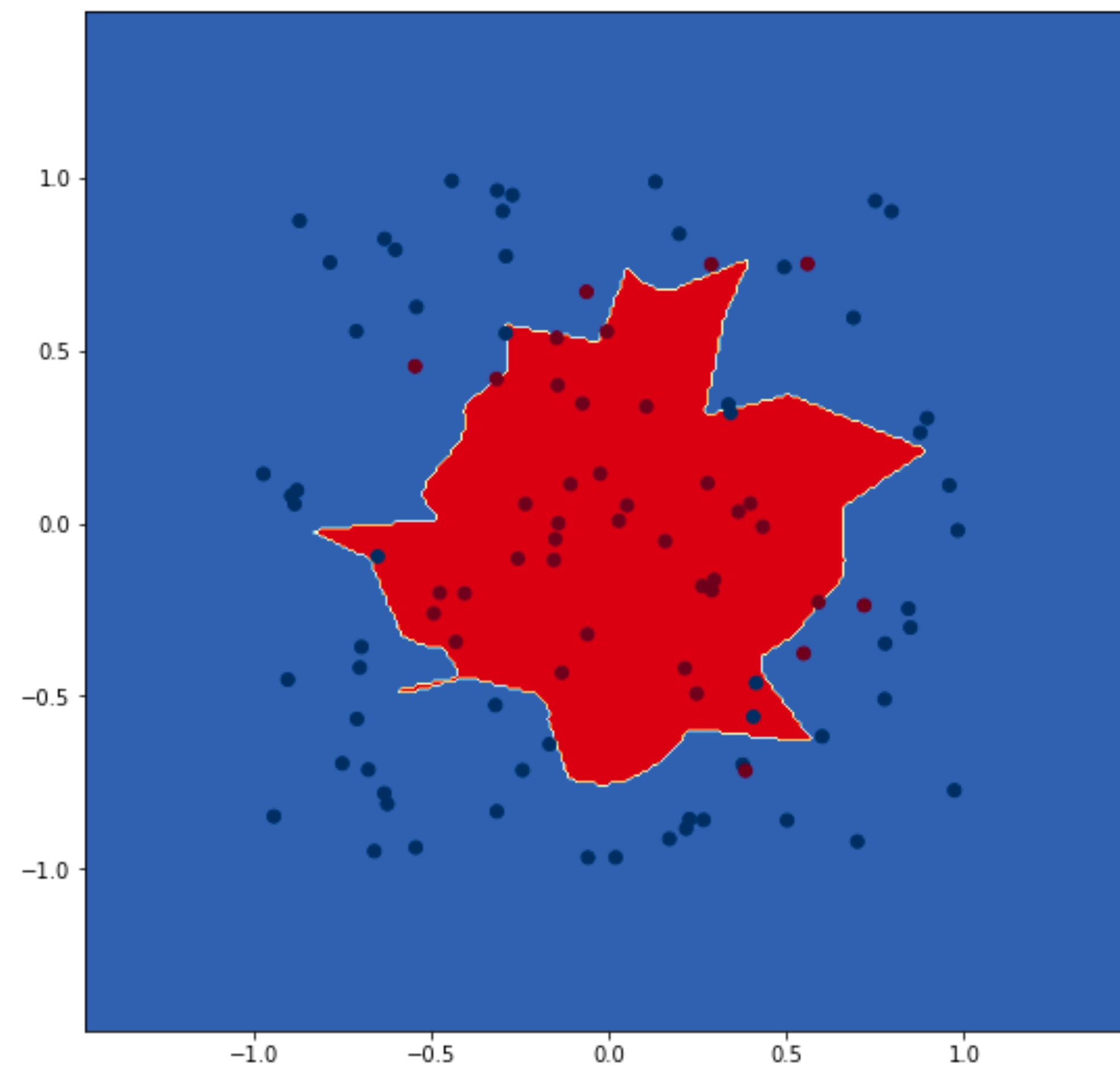
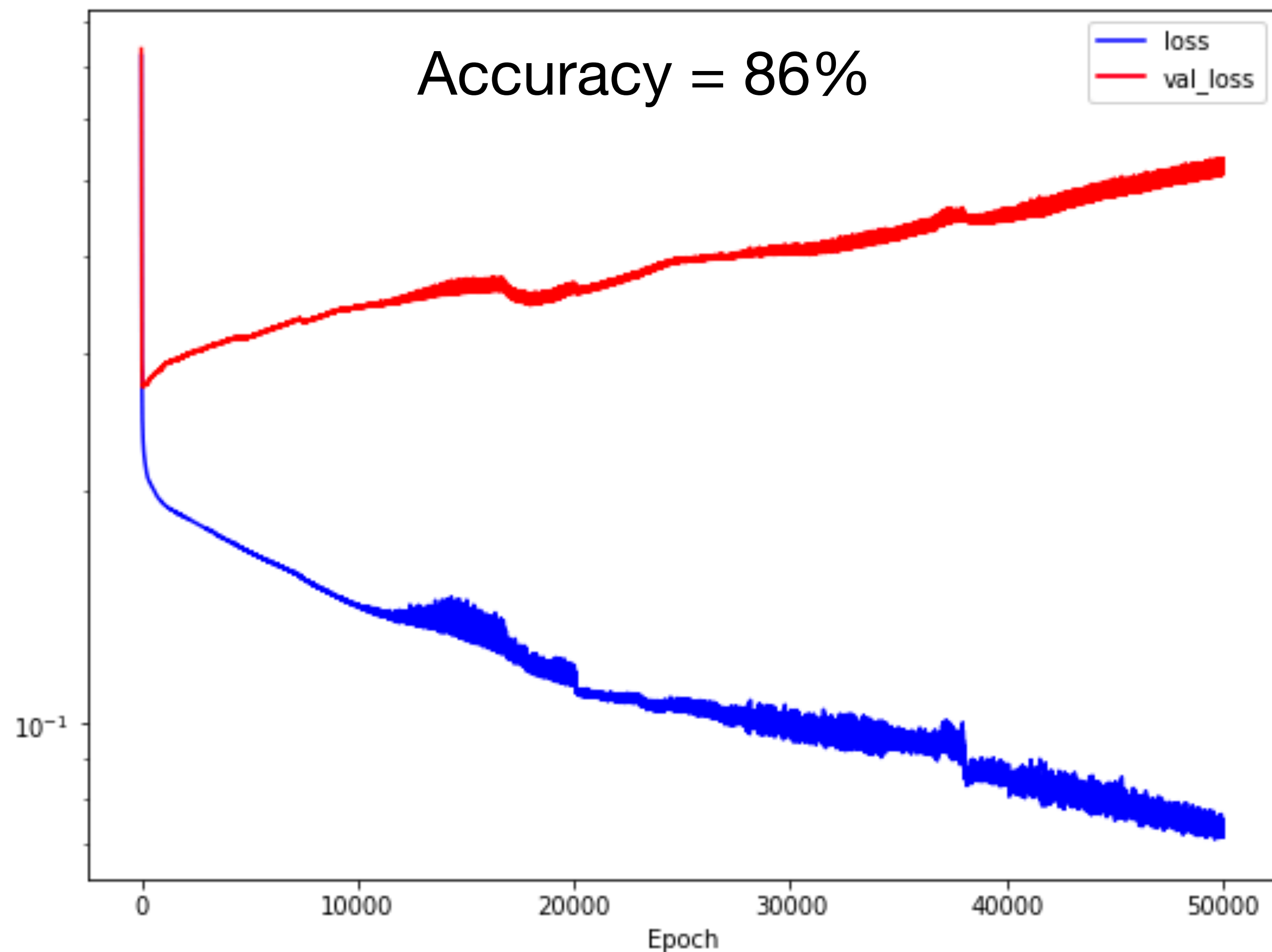
```
layers = [  
    Input(2),  
    Dense(8, act='relu'),  
    Dense(8, act='relu'),  
    Dense(1, act='sigmoid')  
]  
  
# Train just-right scenario  
test_UnderfitOverfit(mTrain=5000, mDev=2000, noise=0.0, layers=layers)
```



Example Toy Project

Overfit

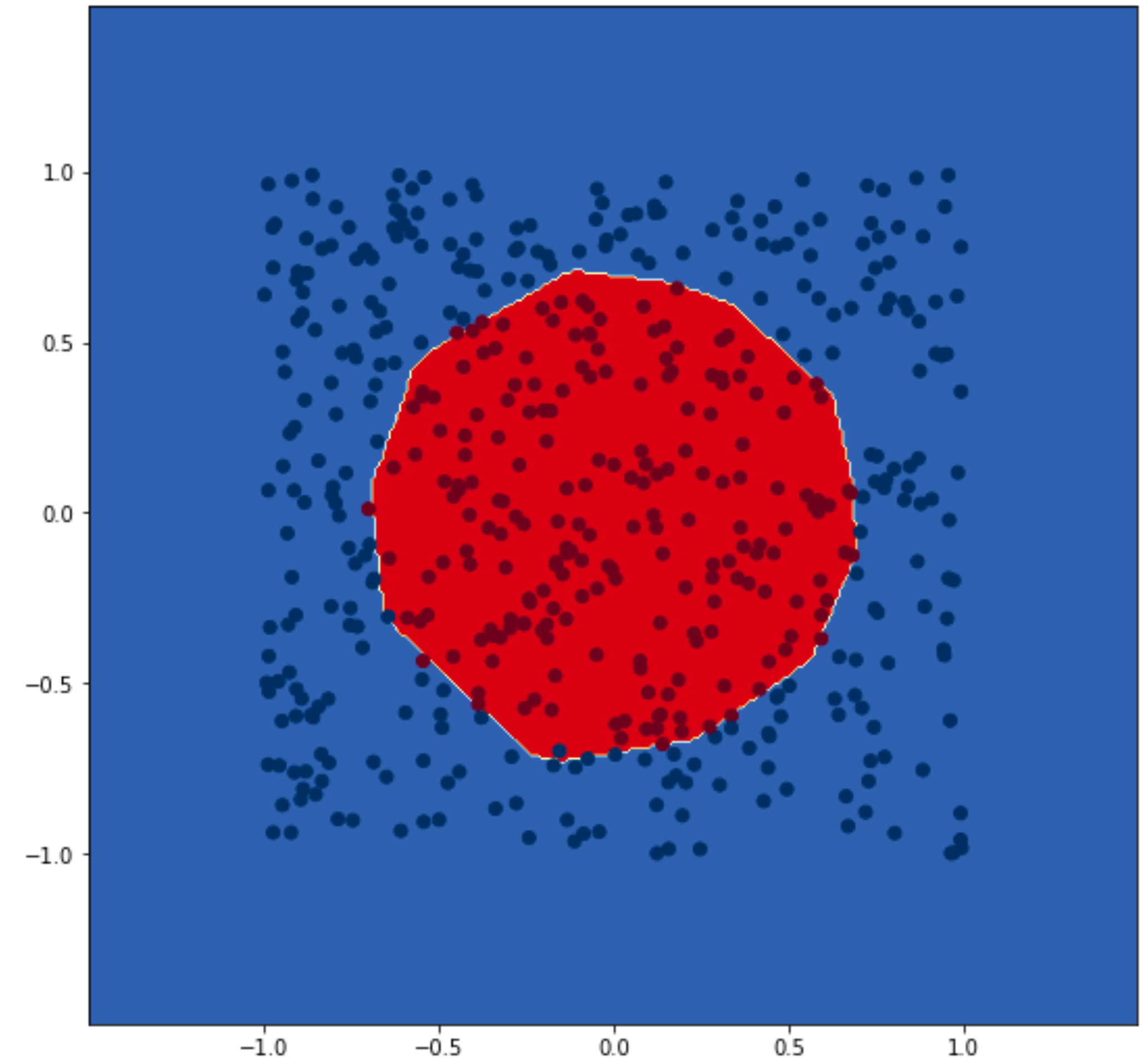
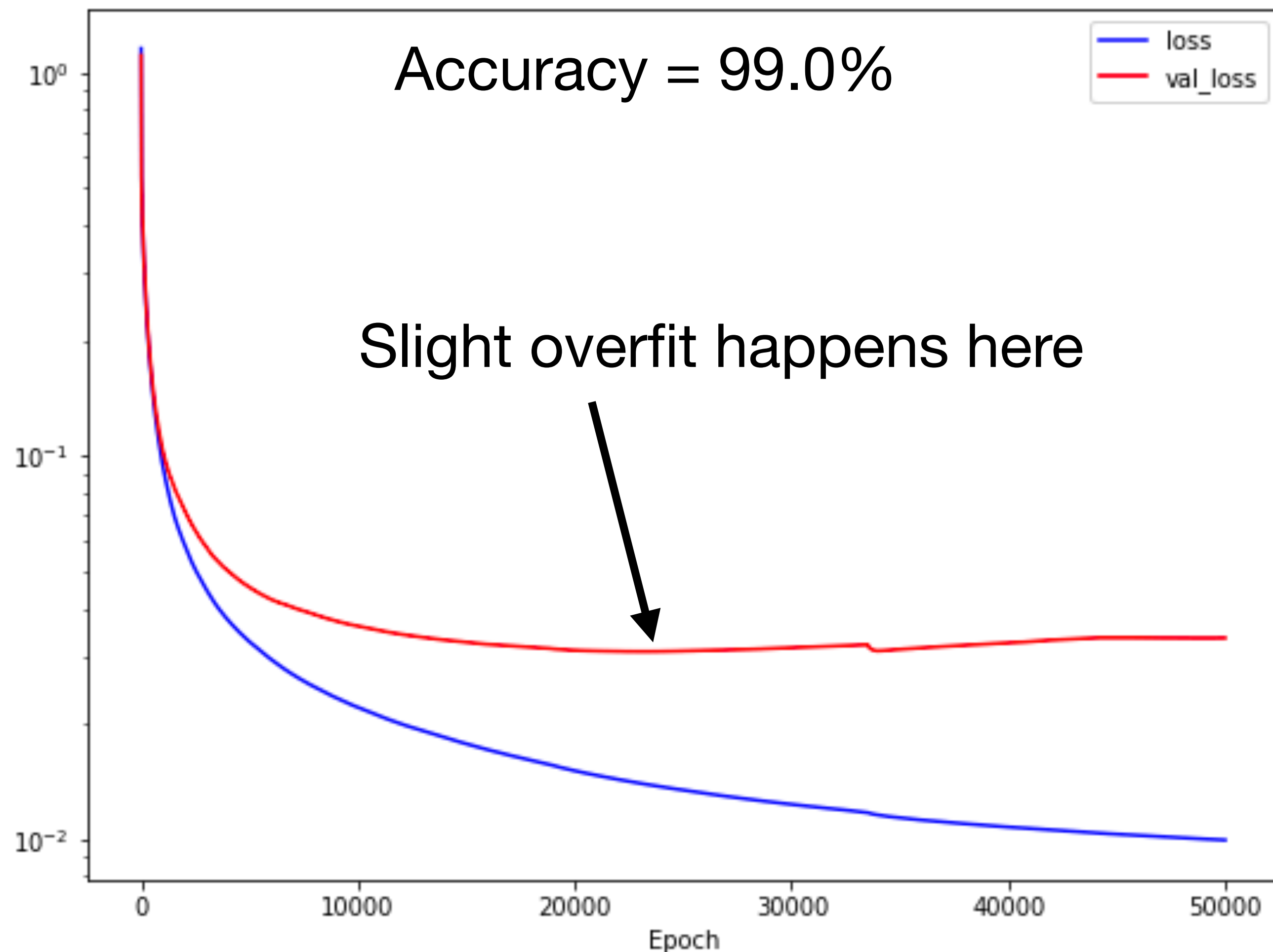
```
layers = [  
    Input(2),  
    Dense(20, act='relu'),  
    Dense(20, act='relu'),  
    Dense(1, act='sigmoid')  
]  
  
# Train overfit scenario  
test_UnderfitOverfit(mTrain=200, mDev=100, noise=0.5, layers=layers)
```



Example Toy Project

“Just Right 2”

```
layers = [  
    Input(2),  
    Dense(10, act='relu'),      # 2 more neurons in hidden  
    Dense(10, act='relu'),      # layers  
    Dense(1, act='sigmoid')  
]  
# Train Overfit-2 - slightly smaller dataset  
test_UnderfitOverfit(mTrain=1000, mDev=500, noise=0.0, layers=layers)
```



Regularization

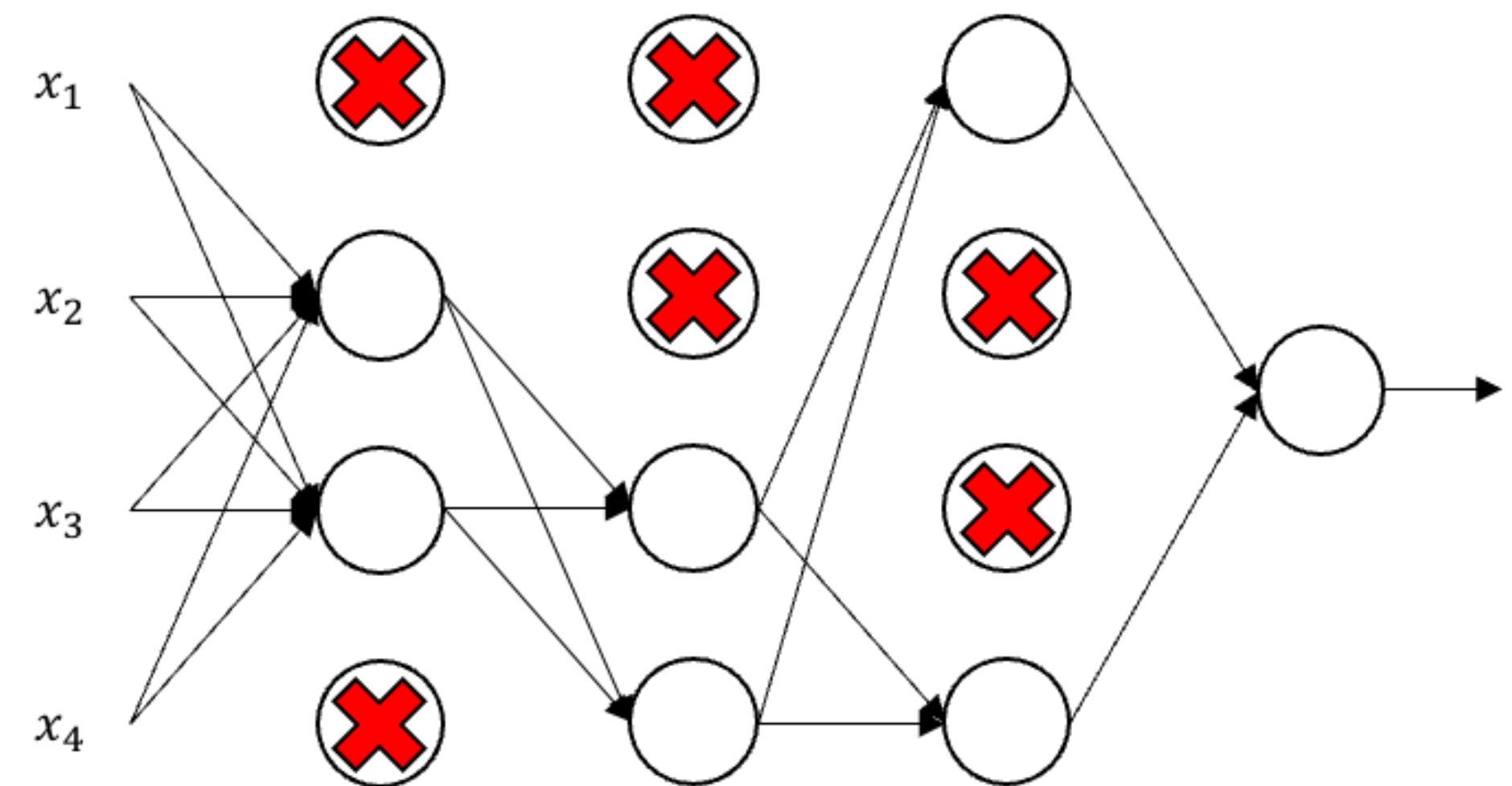
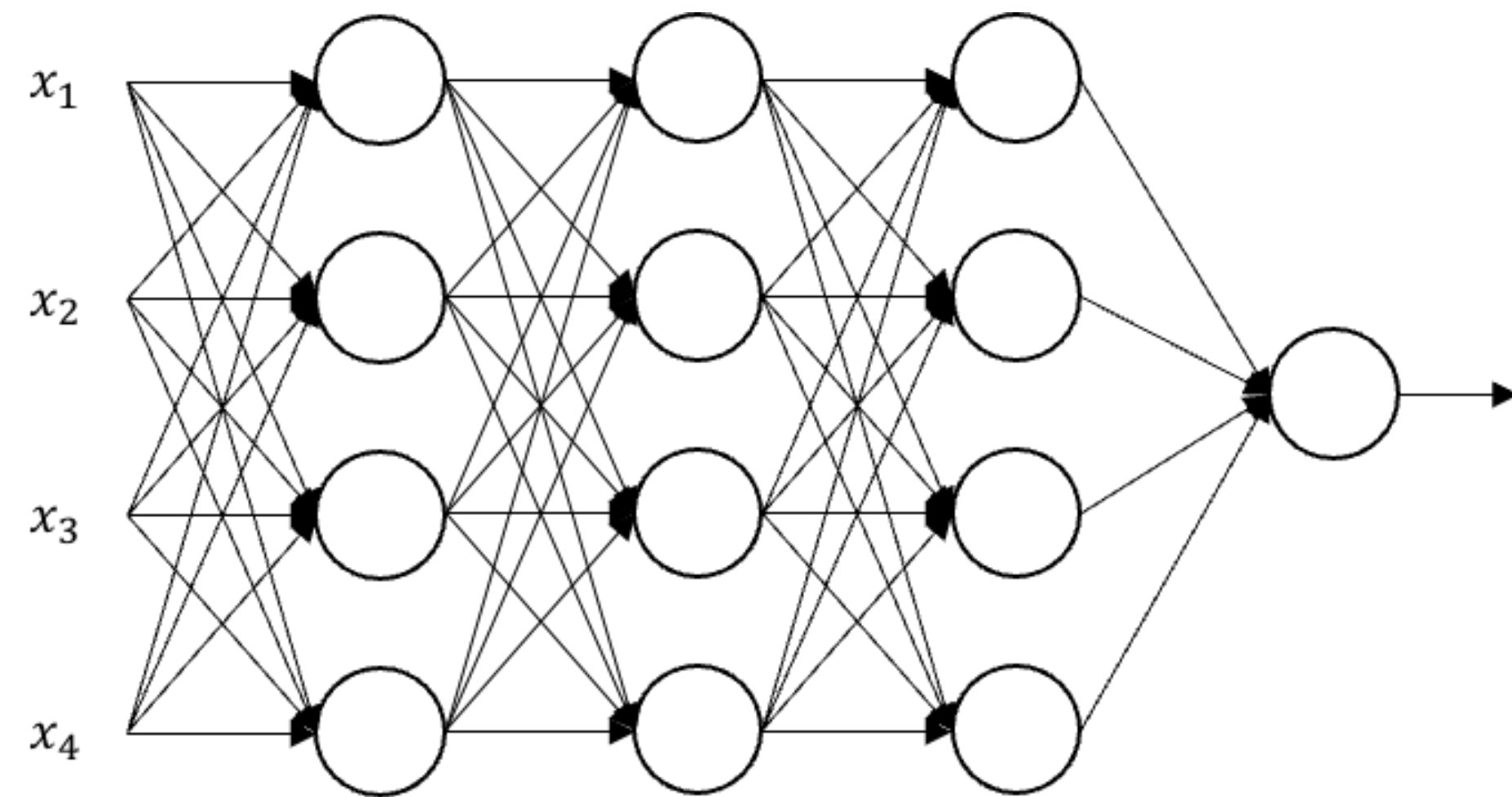
“Don’t memorize the train set”

Regularization

- **Prevent overfitting**
- Improve the generalization performance of a model
- Discourage the model from becoming too complex
- Control the model's ability to fit noise in the training data

Regularization

DropOut



50 % chance, that a neuron will be used

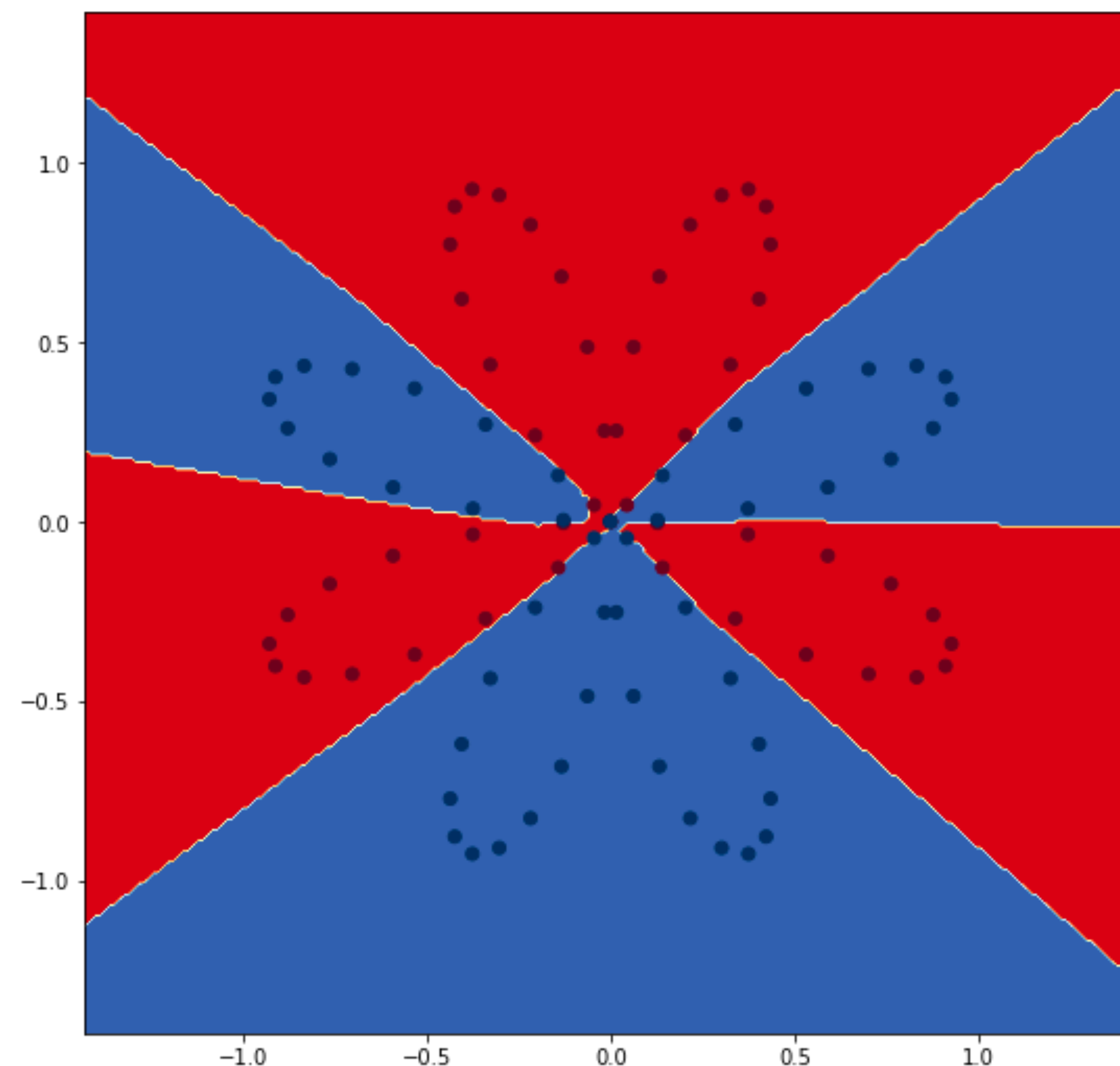
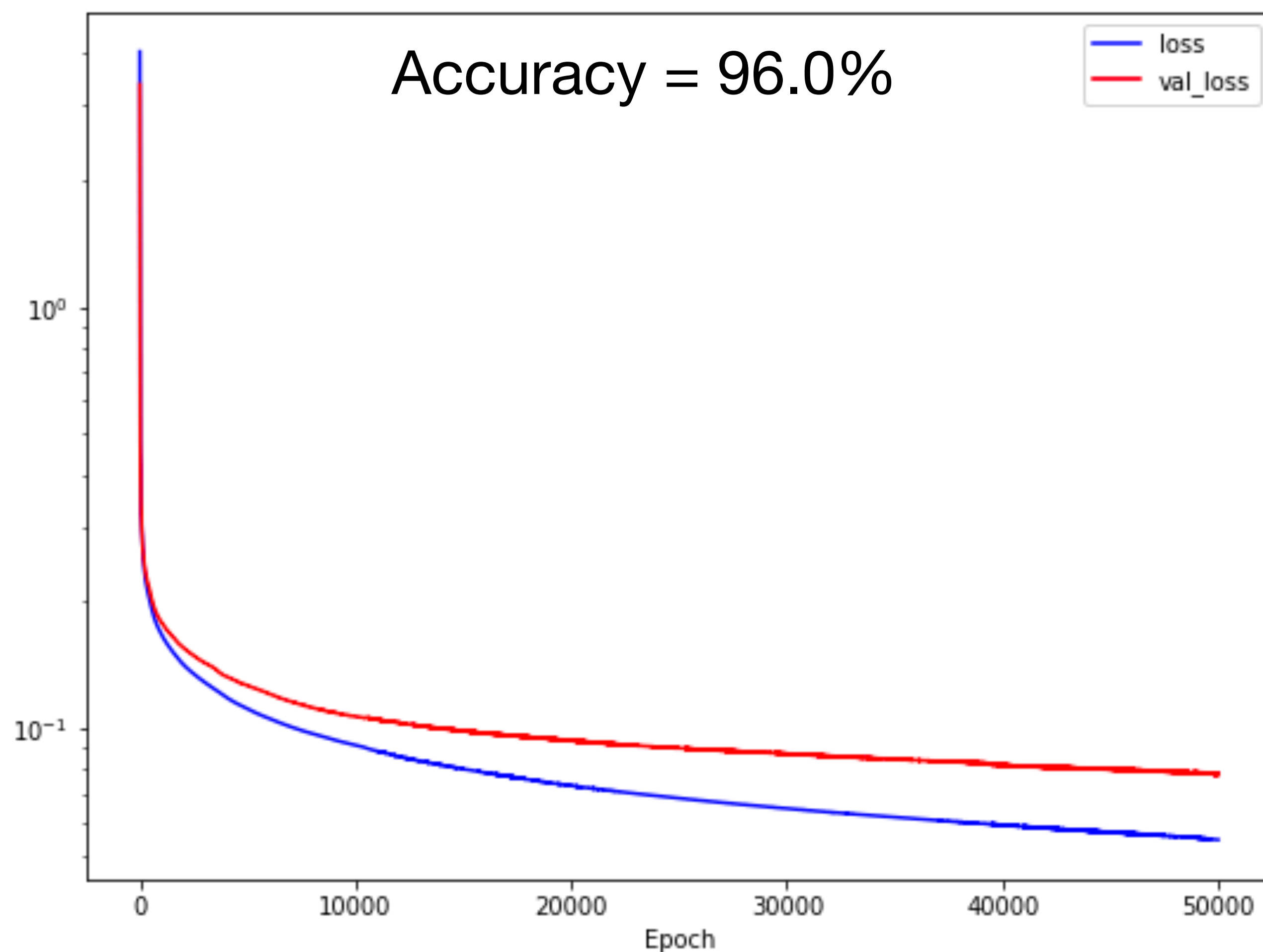
- usually applied to activations; different “dropped” neurons selected each mini-batch

Regularization

DropOut

```
layers = [  
    Input(2),  
    Dense(15, act='relu'),  
    Dense(15, act='relu'),  
    Dense(1, act='sigmoid')  
]  
test_DropOutCase(mTrain=500, mDev=100, noise=0.0, layers=layers)
```

No random noise

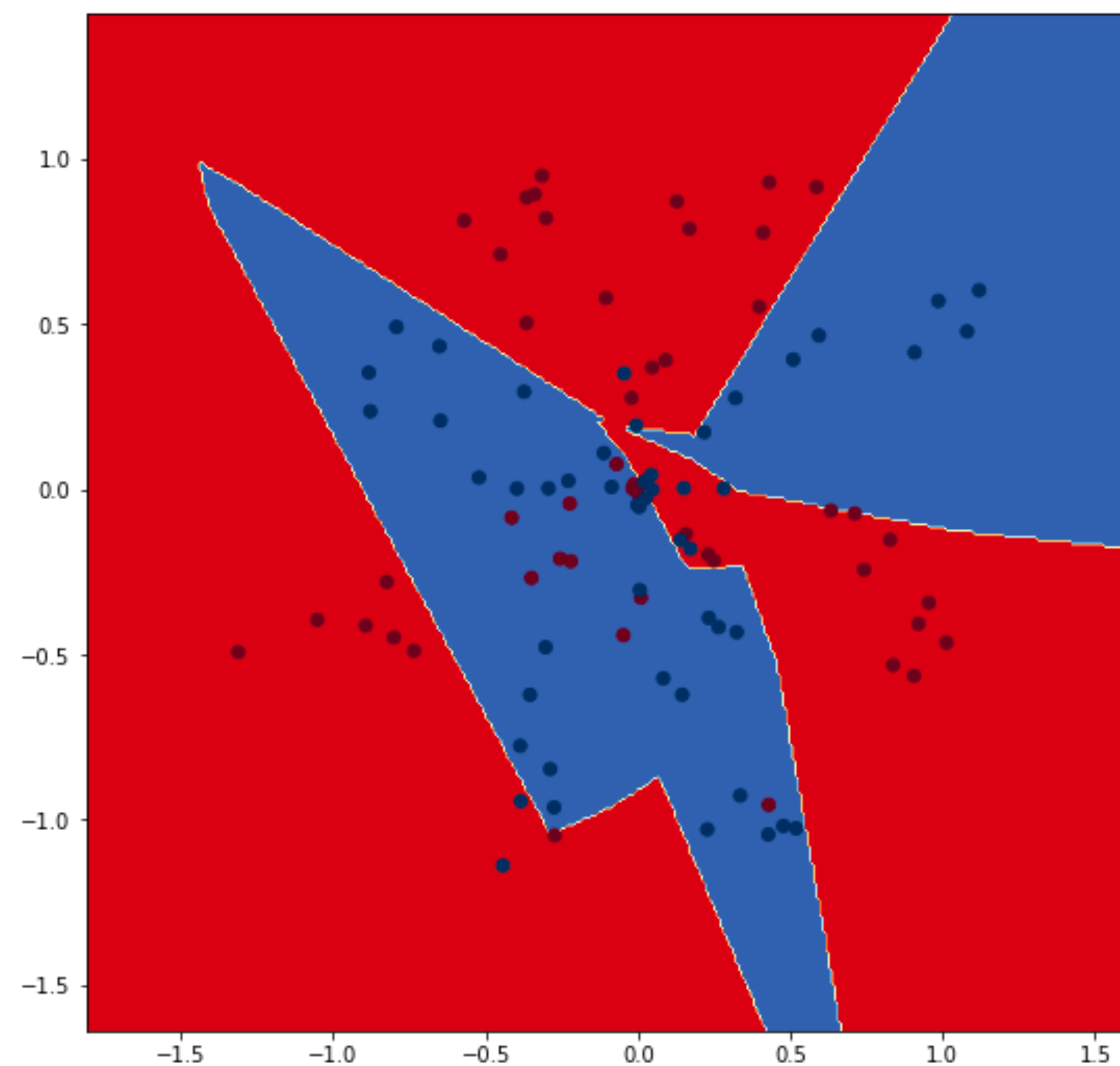
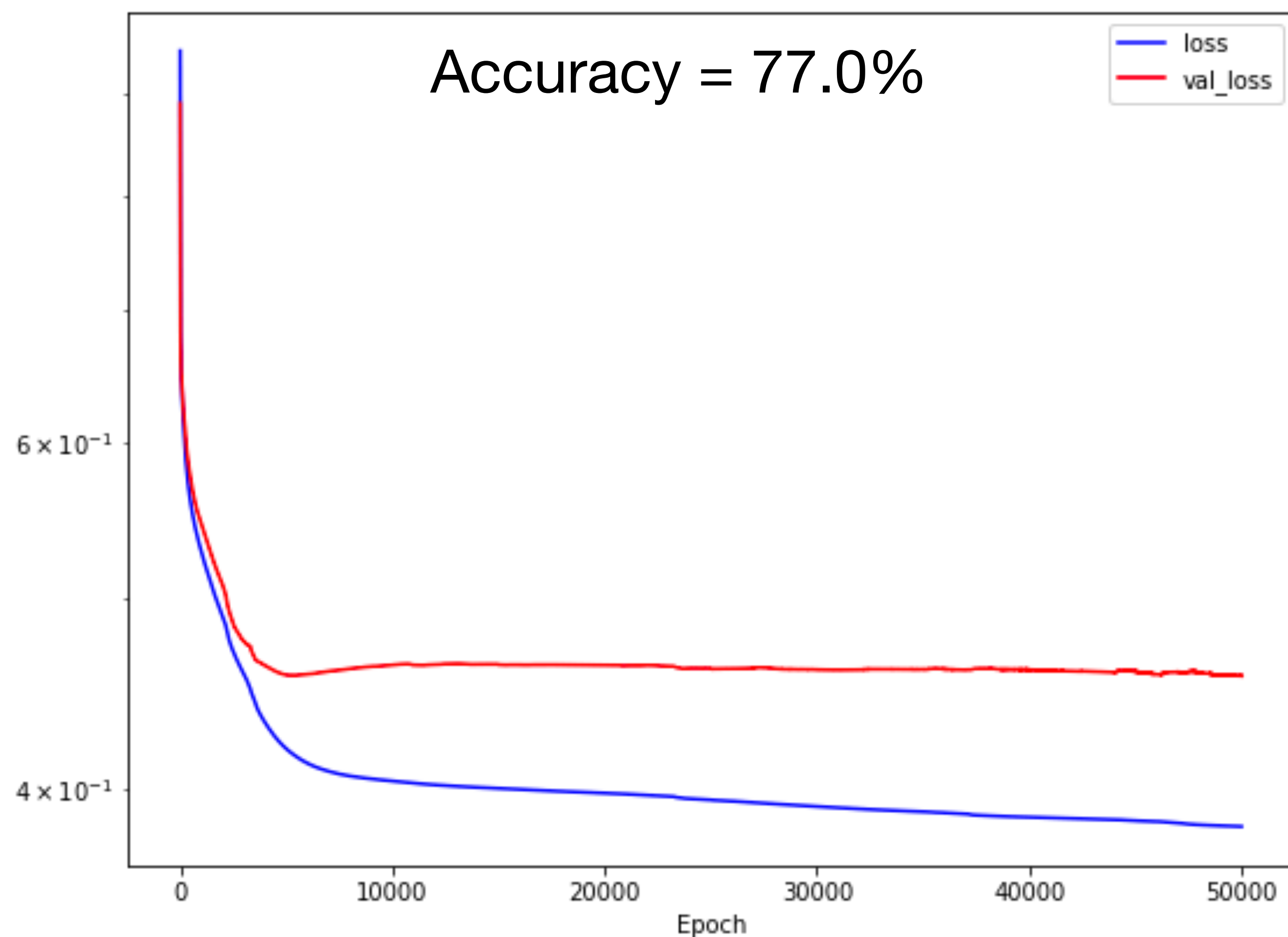


Regularization

DropOut - Poor Fit

```
layers = [  
    Input(2),  
    Dense(5, act='relu'),  
    Dense(5, act='relu'),  
    Dense(1, act='sigmoid')  
]  
test_DropOutCase(mTrain=500, mDev=100, noise=0.2, layers=layers)
```

Noise !!

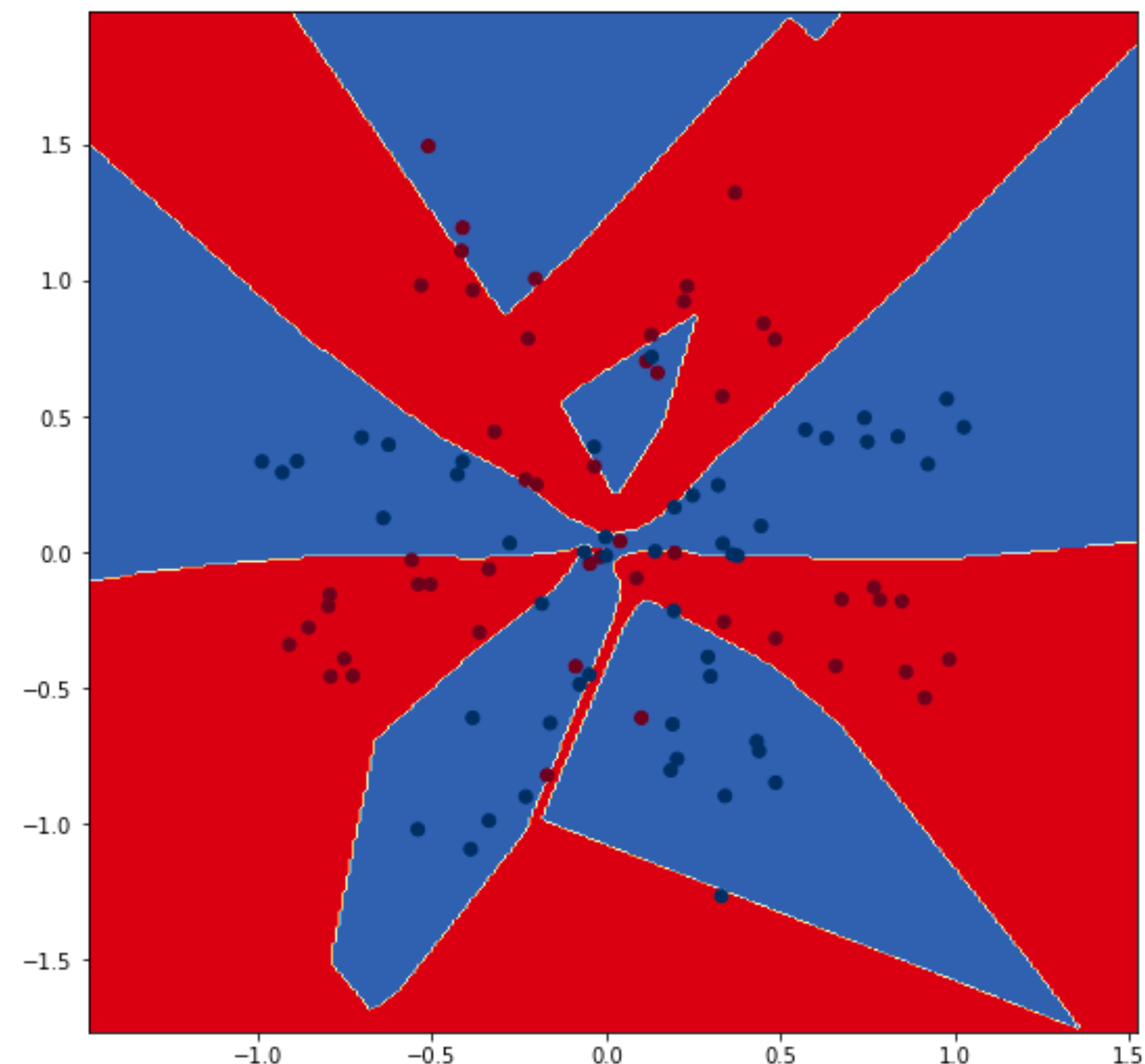
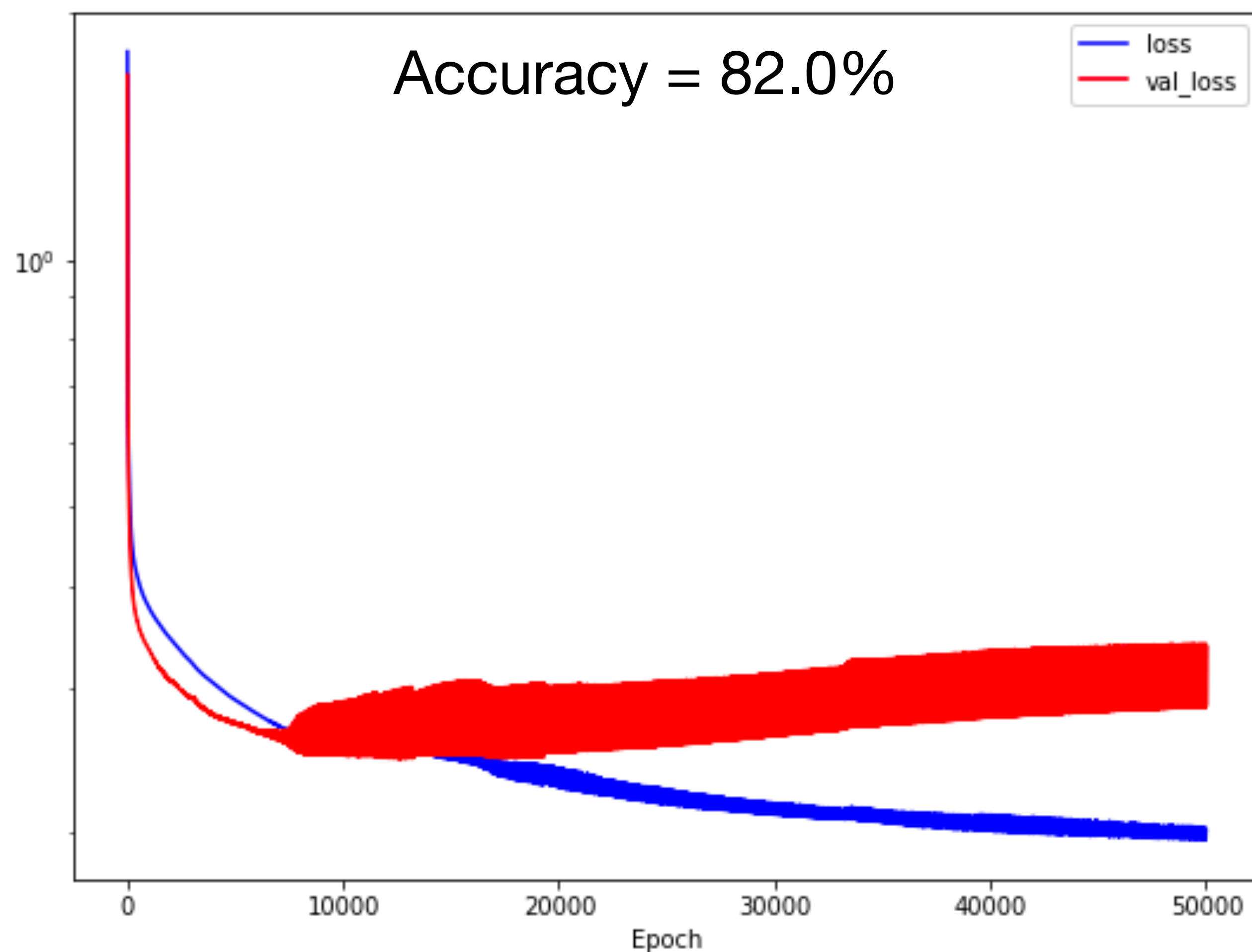


Regularization

DropOut - Stronger Model

```
layers = [  
    Input(2),  
    Dense(15, act='relu'),  
    Dense(15, act='relu'),  
    Dense(1, act='sigmoid')  
]  
test_DropOutCase(mTrain=400, mDev=100, noise=0.2, layers=layers)
```

Noise !!

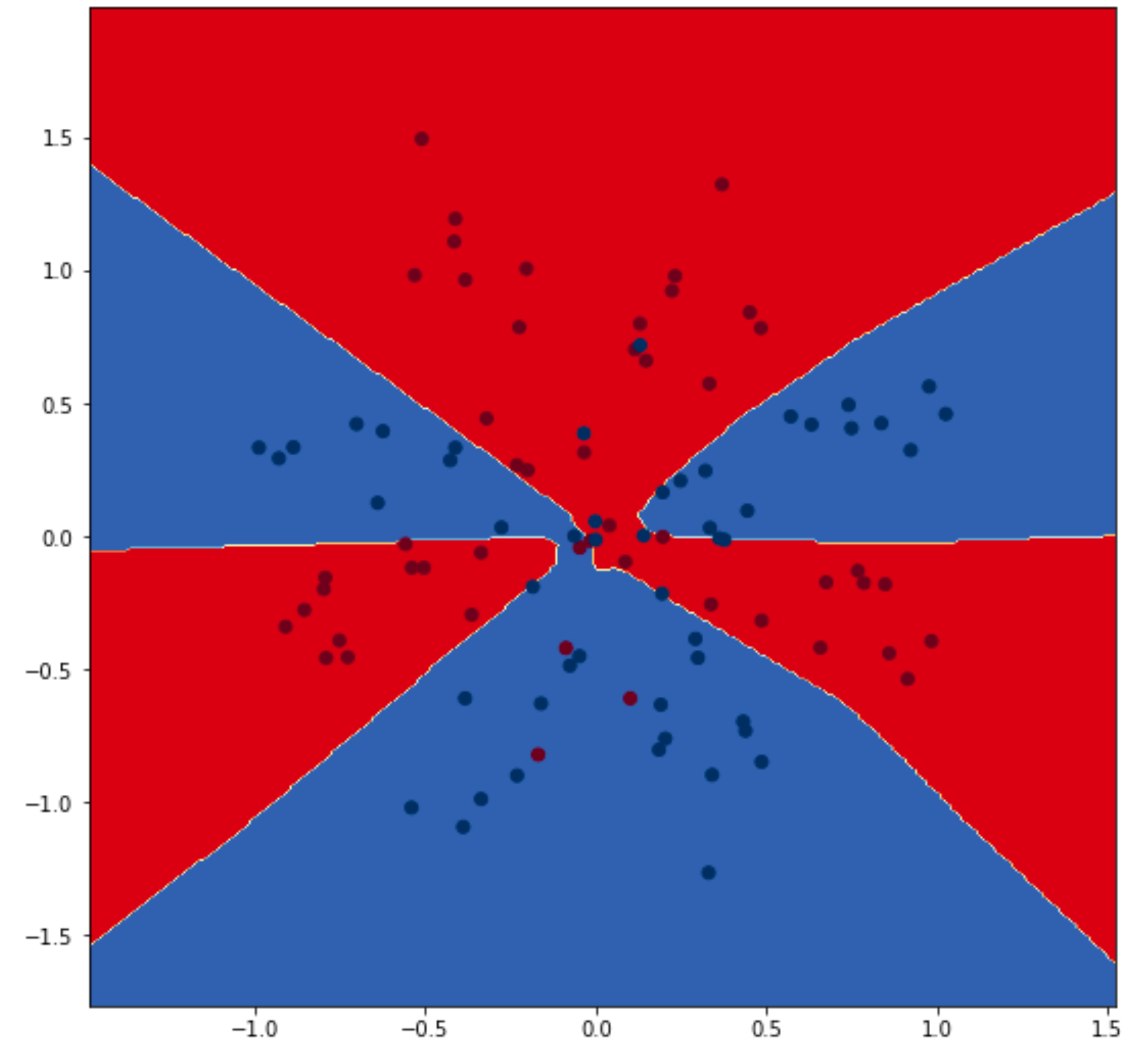
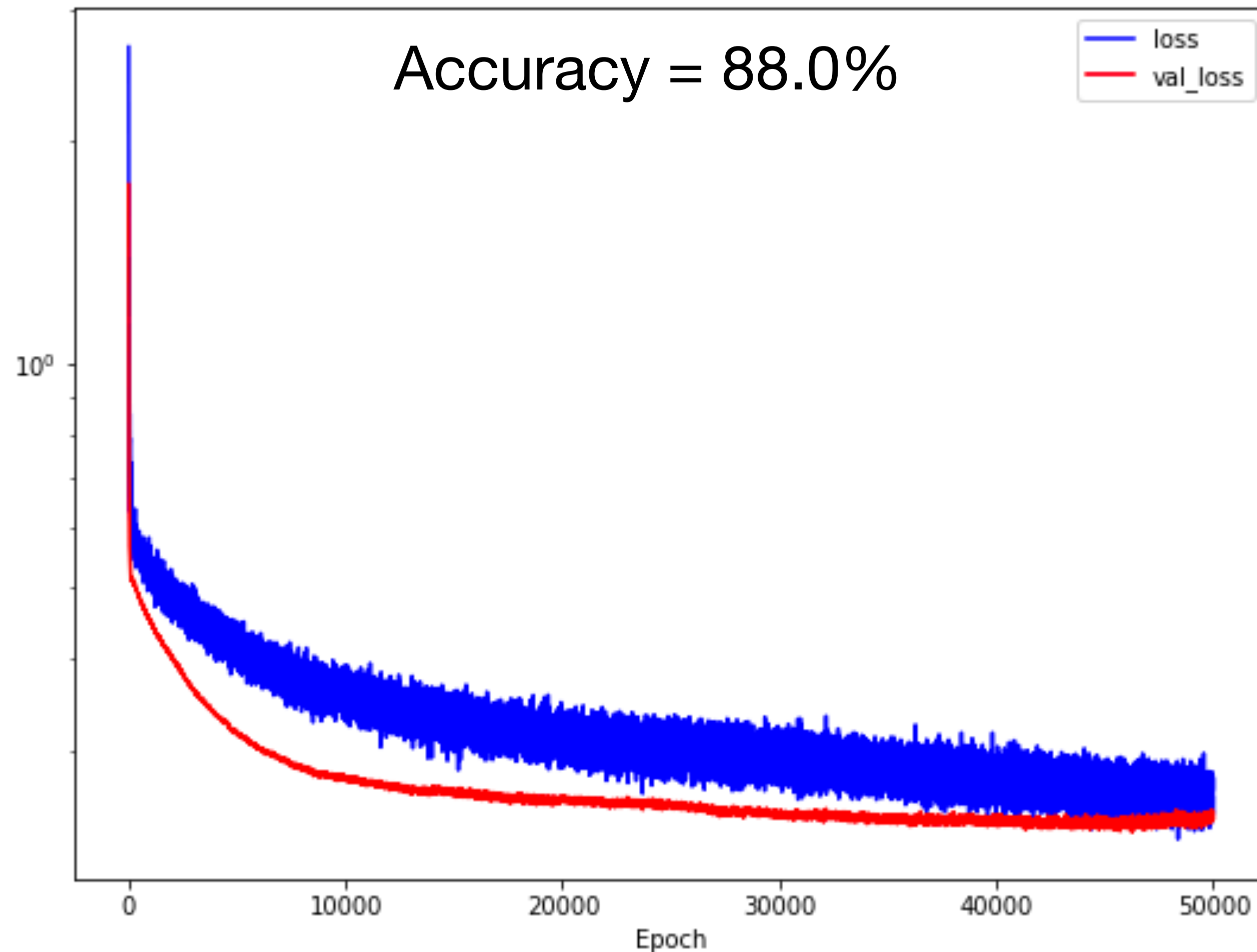


Regularization

DropOut - Stronger Model

```
layers = [  
    Input(2),  
    Dense(15, act='relu'),  
    Dense(15, act='relu'),  
    Dropout(0.4),   
    Dense(1, act='sigmoid')  
]  
test_DropOutCase(mTrain=400, mDev=100, noise=0.2, layers=layers)
```

DropOut !!



Regularization

L1, L2

- Large values of weights - enables model to fit noise in training data
- We want smaller weights

$$\tilde{J}(\Theta, \mathcal{X}) = J(\Theta, \mathcal{X}) + \lambda \Omega(\Theta)$$

original cost function

λ - regularization weight

$\Omega(\Theta)$ - penalty function

Regularization

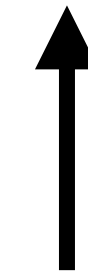
L1, L2

- Large values of weights - enables model to fit noise in training data
- We want smaller weights

$$\tilde{J}(\Theta, \mathcal{X}) = J(\Theta, \mathcal{X}) + \lambda \Omega(\Theta)$$

$$\text{L1 : } \Omega(\Theta) = \sum_i |w_i|$$

$$\text{L2 : } \Omega(\Theta) = \sum_i w_i^2$$



λ - regularization weight

$\Omega(\Theta)$ - penalty function

Regularization

L1, L2

- Large values of weights - enables model to fit noise in training data
- We want smaller weights

$$\tilde{J}(\Theta, \mathcal{X}) = J(\Theta, \mathcal{X}) + \lambda \Omega(\Theta)$$

$$\text{L1} : \Omega(\Theta) = \sum_i |w_i|$$

$$\text{L2} : \Omega(\Theta) = \sum_i w_i^2$$

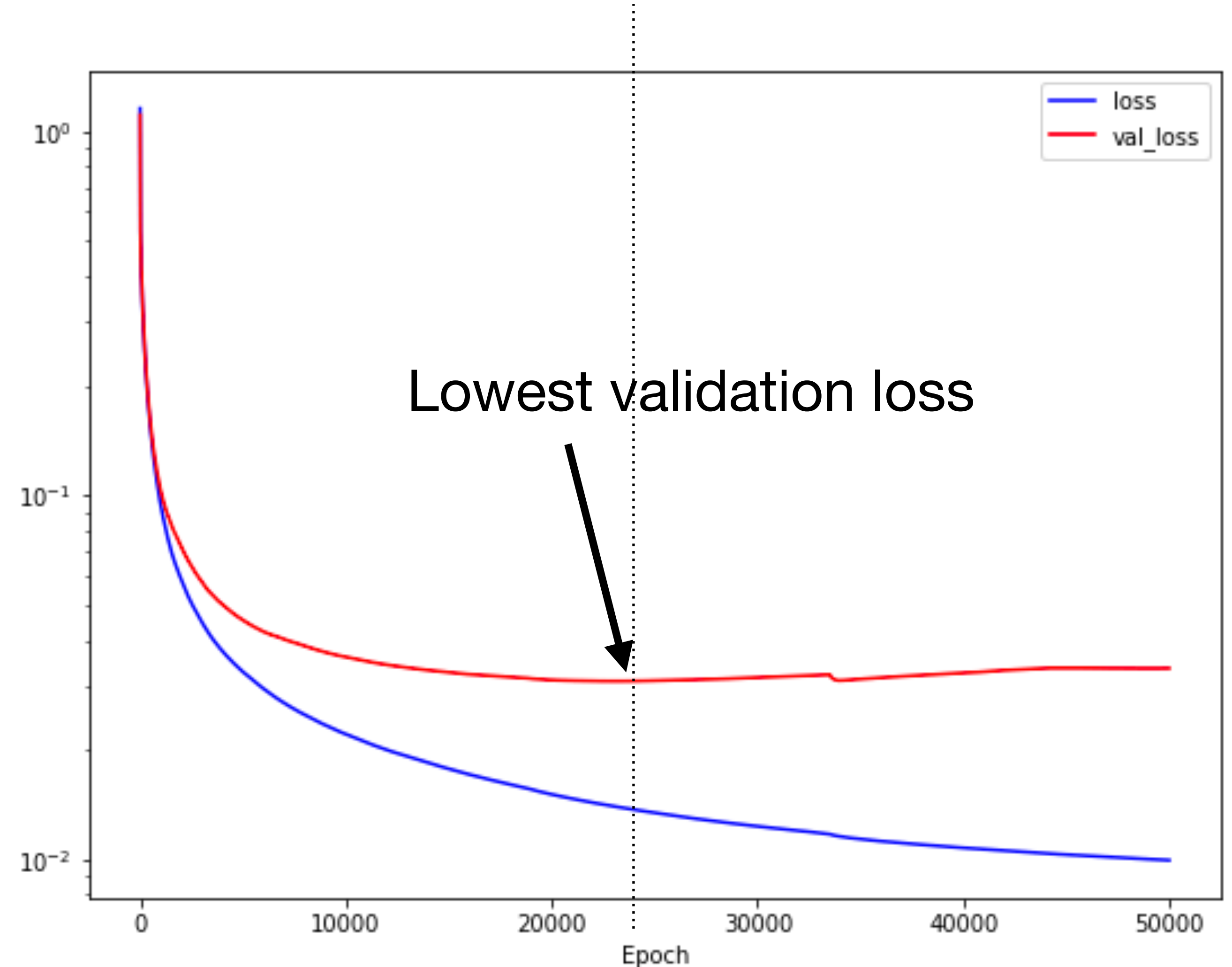
(also called **weight decay** (for SGD))

↑
 λ - regularization weight
 $\Omega(\Theta)$ - penalty function

Regularization

Early Stopping

- Complex model is more likely to overfit
- Training loss is decreasing
- Validation loss is increasing
- Use the model with lowest validation loss



Initialization

“Make gradient flow possible from the start”

Initialization

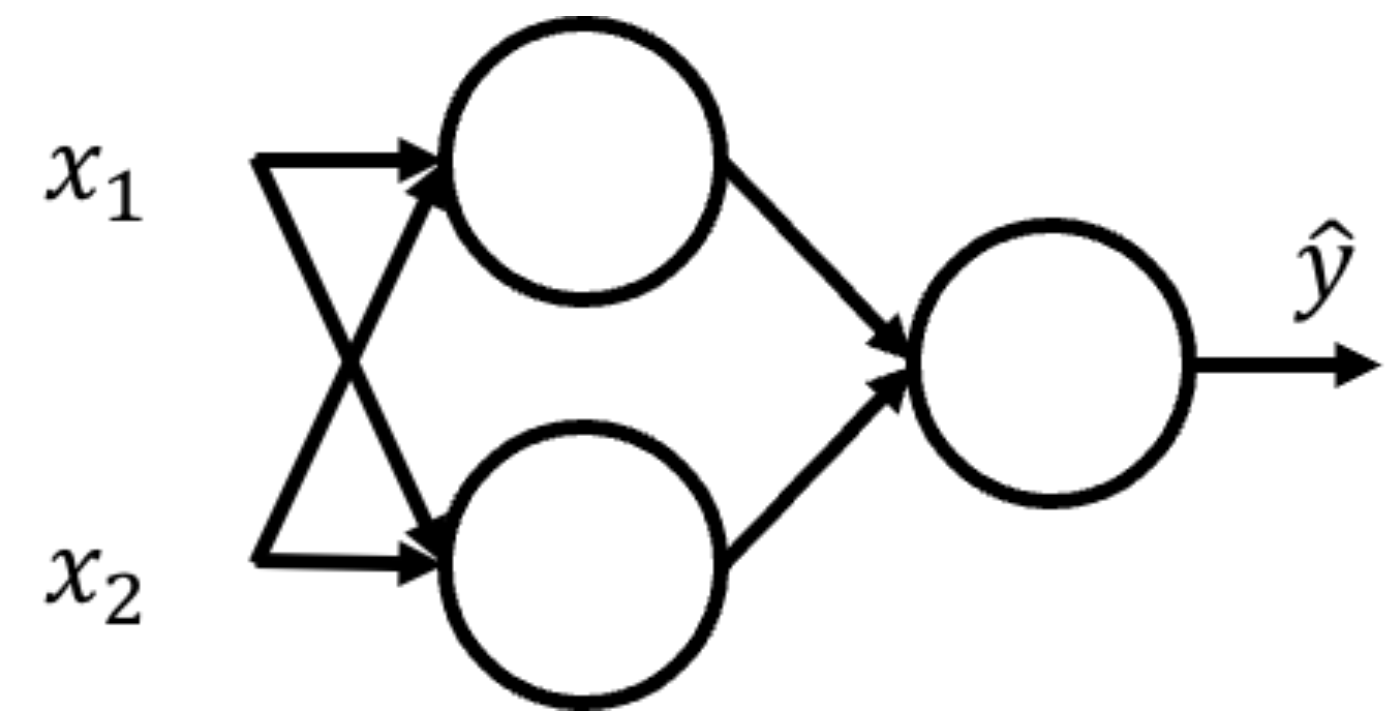
Vanishing & Exploding Gradient Problem

- Consider the following example

Initialization

Vanishing & Exploding Gradient Problem

- Consider the following example
 - ignore bias, only use linear activation function

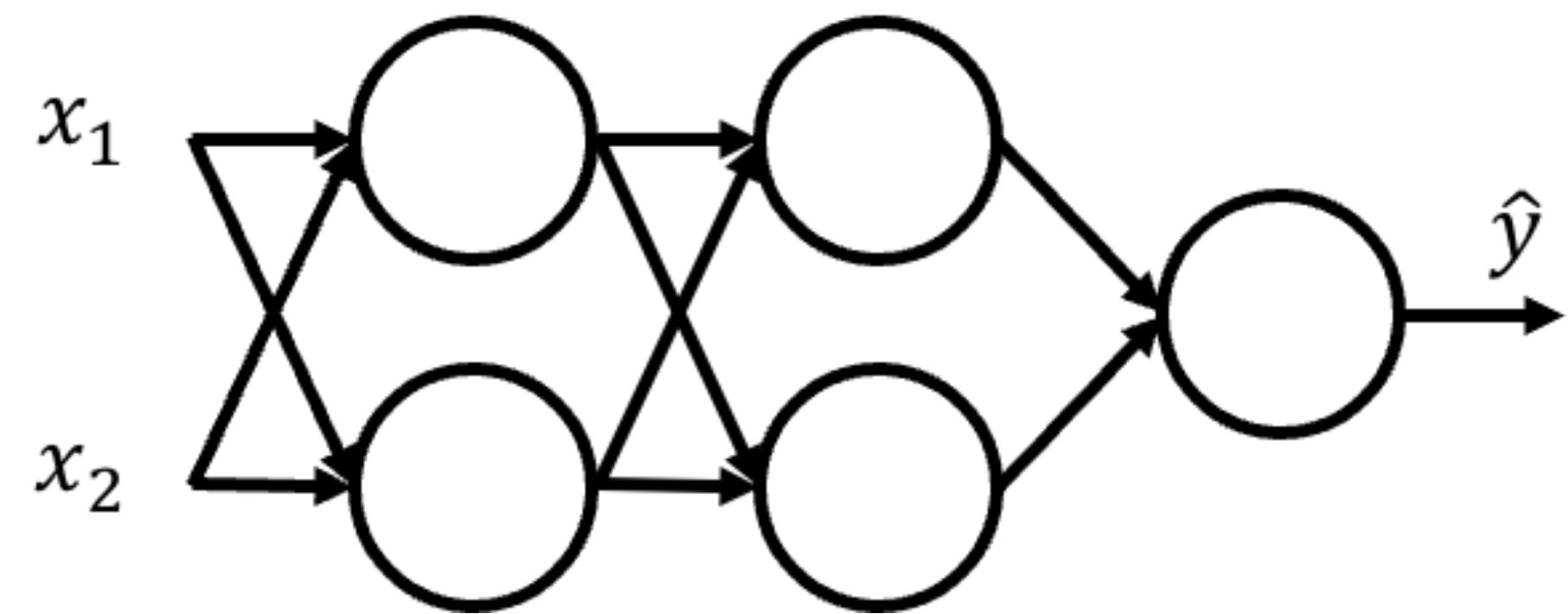


$$\hat{y} = W^{[2]}W^{[1]}x$$

Initialization

Vanishing & Exploding Gradient Problem

- Consider the following example
 - ignore bias, only use linear activation function

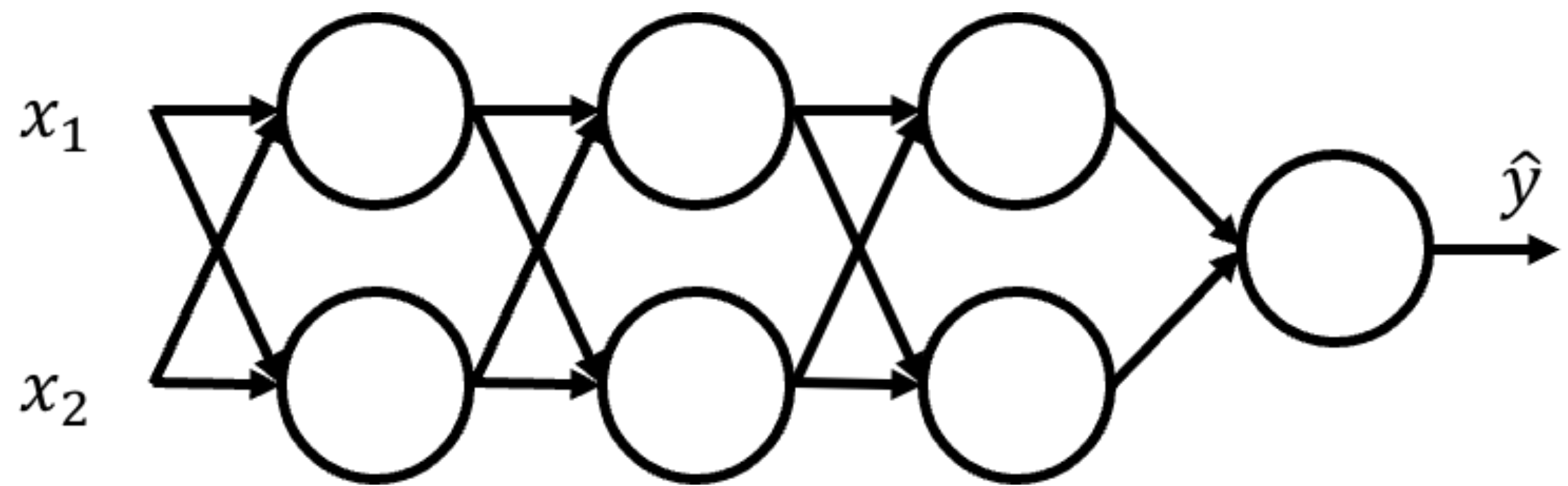


$$\hat{y} = W^{[3]}W^{[2]}W^{[1]}x$$

Initialization

Vanishing & Exploding Gradient Problem

- Consider the following example
 - ignore bias, only use linear activation function

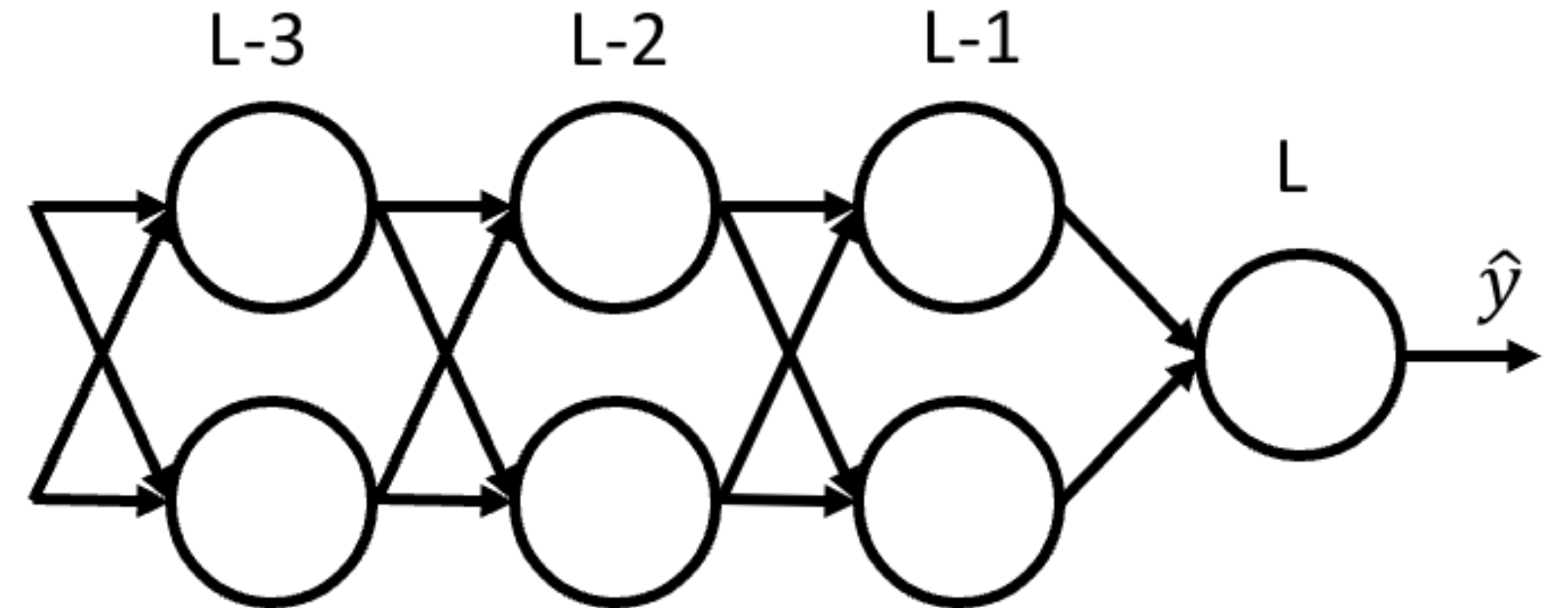
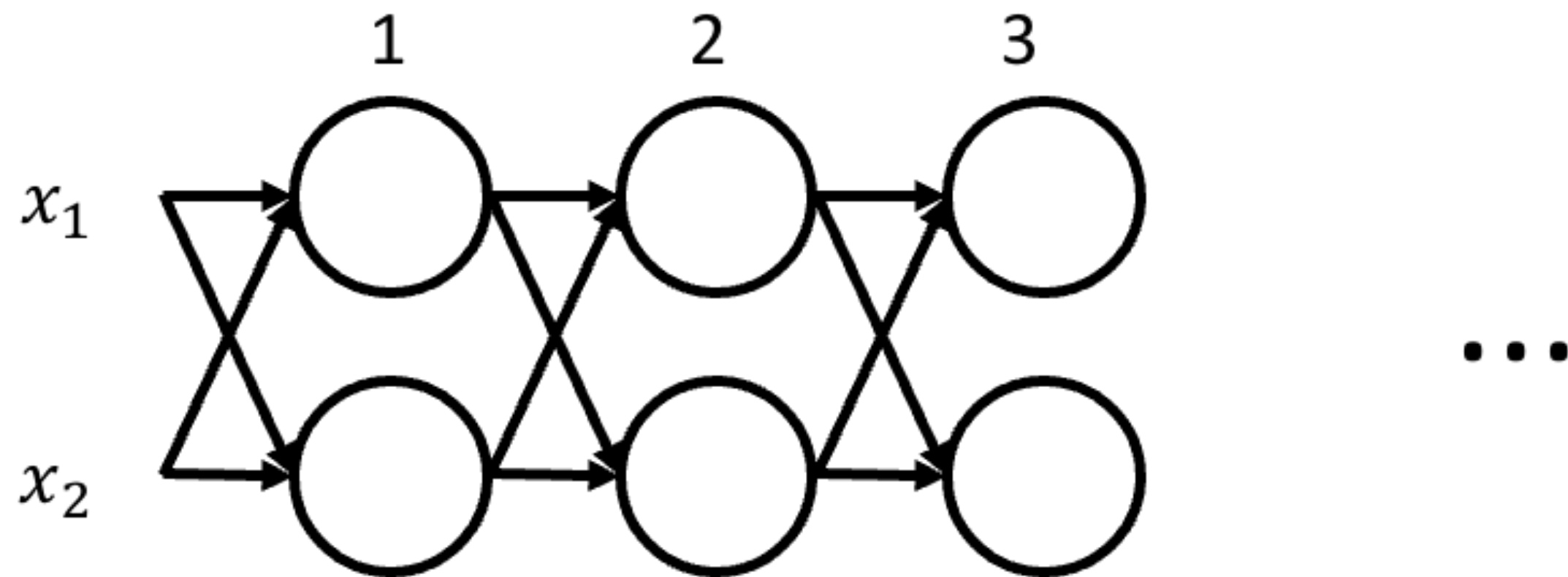


$$\hat{y} = W^{[4]}W^{[3]}W^{[2]}W^{[1]}x$$

Initialization

Vanishing & Exploding Gradient Problem

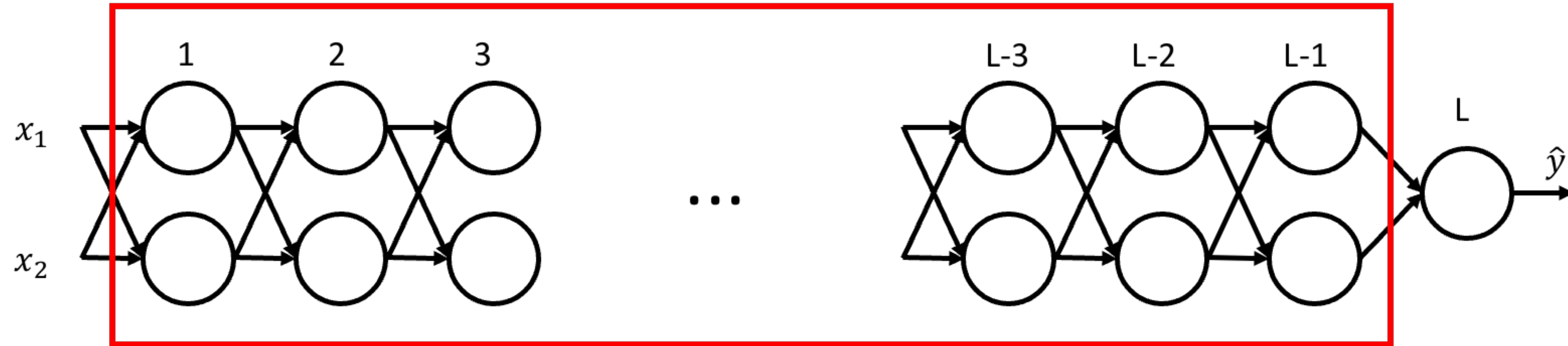
- Consider the following example
 - ignore bias, only use linear activation function



$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

Initialization

Vanishing & Exploding Gradient Problem



$L = 21$

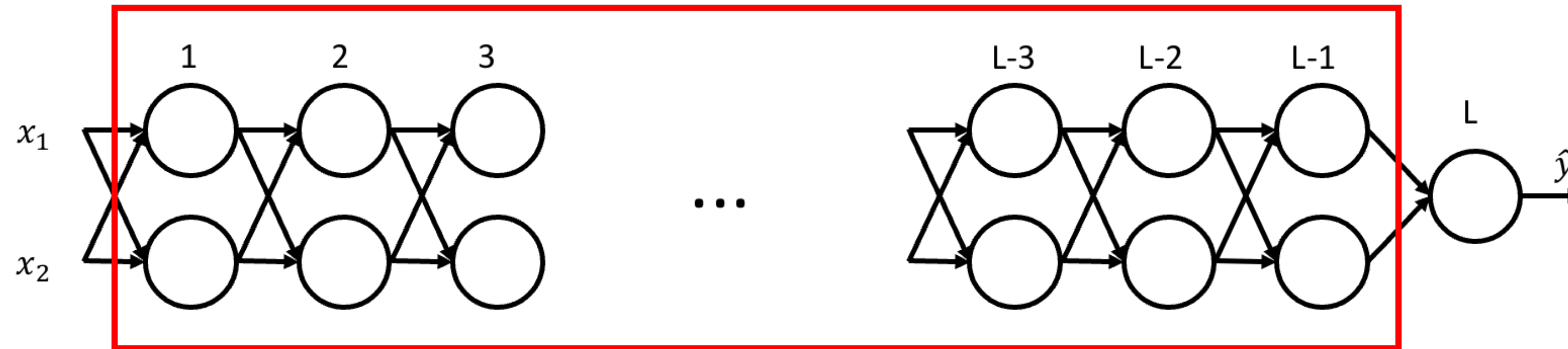
$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

$$\hat{y} = W^{[L]} W^{L-1} x$$

W is raised to the number of layers

Initialization

Vanishing & Exploding Gradient Problem



$$L = 21$$

$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

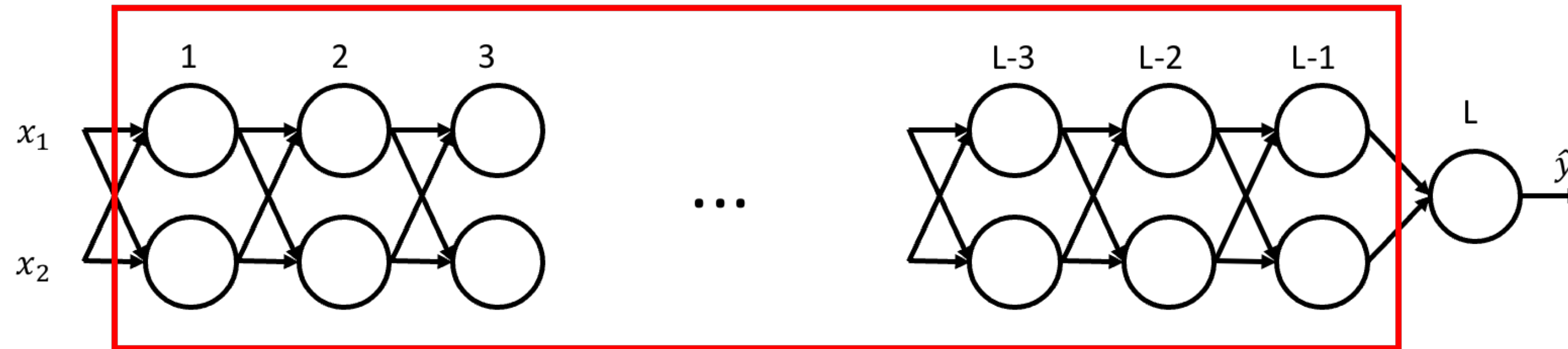
$$\hat{y} = W^{[L]} W^{L-1} x$$

$$W = \begin{pmatrix} 1.5 & 0 \\ 0 & 1.5 \end{pmatrix}$$

W is raised to the number of layers

Initialization

Vanishing & Exploding Gradient Problem



$L = 21$

$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

$$\hat{y} = W^{[L]} W^{L-1} x$$

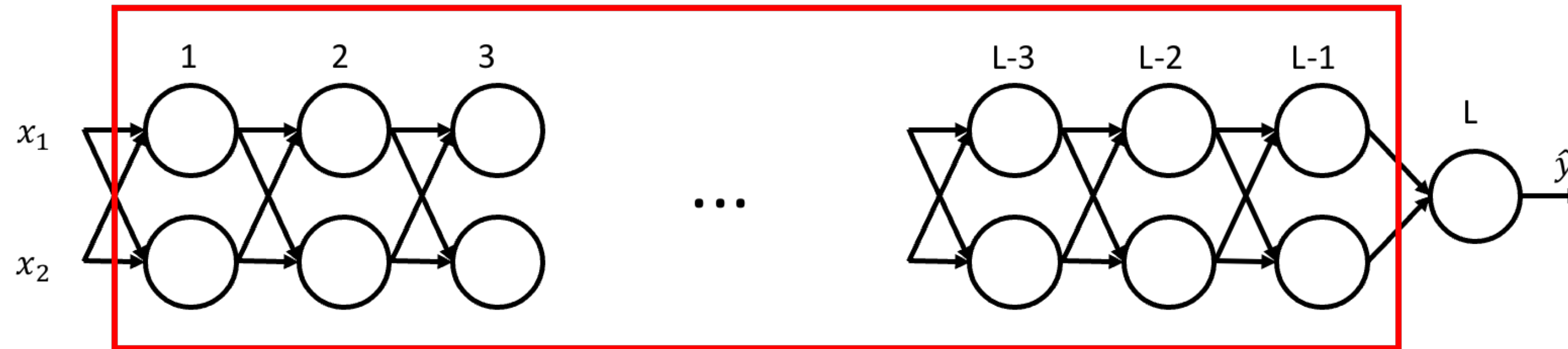
W is raised to the number of layers

$$W = \begin{pmatrix} 1.5 & 0 \\ 0 & 1.5 \end{pmatrix}$$

$$W^2 = \begin{pmatrix} 2.25 & 0 \\ 0 & 2.25 \end{pmatrix}$$

Initialization

Vanishing & Exploding Gradient Problem



$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

$$\hat{y} = W^{[L]} W^{L-1} x$$

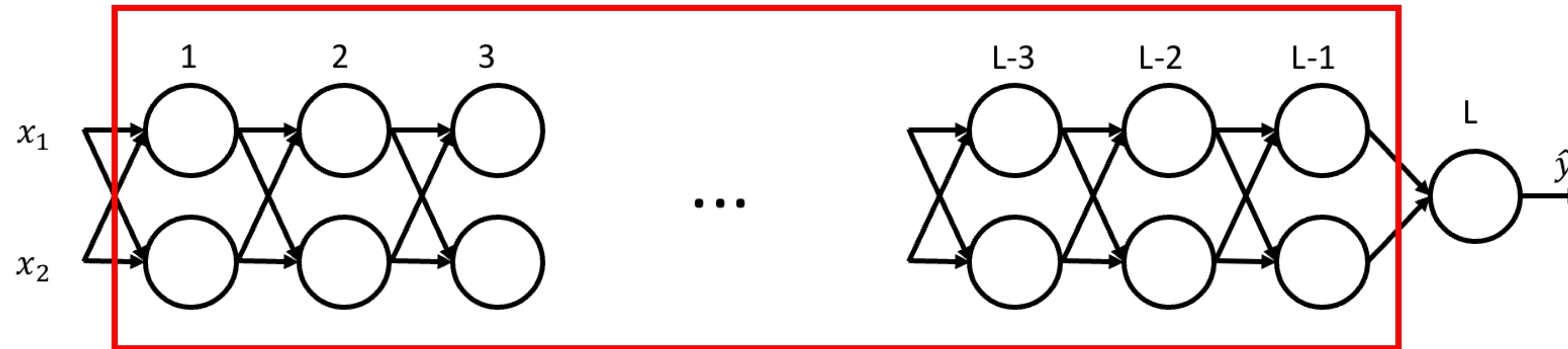
W is raised to the number of layers

$$W = \begin{pmatrix} 1.5 & 0 \\ 0 & 1.5 \end{pmatrix}$$

$$W^3 = \begin{pmatrix} 3.75 & 0 \\ 0 & 3.75 \end{pmatrix}$$

Initialization

Vanishing & Exploding Gradient Problem



$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

$$\hat{y} = W^{[L]} W^{L-1} x$$

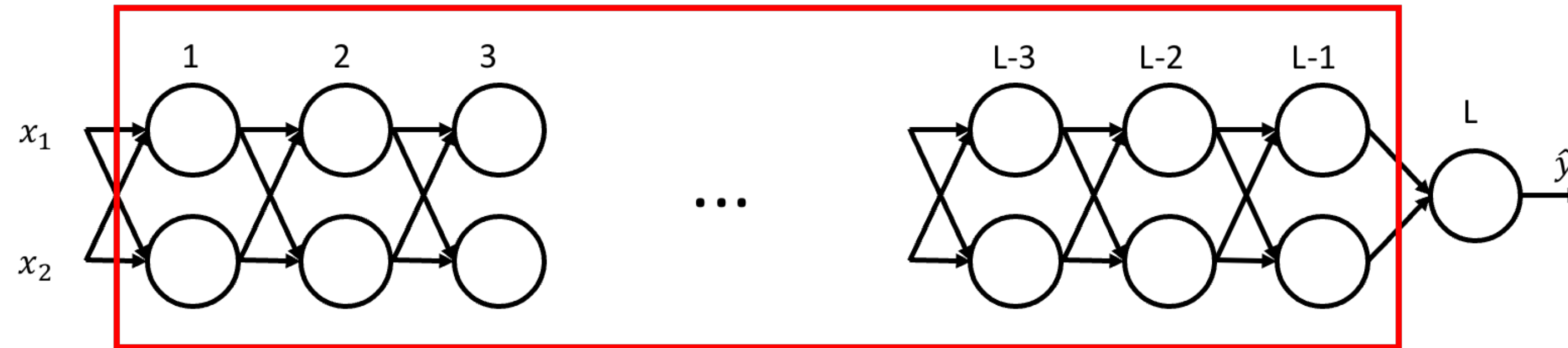
W is raised to the number of layers

$$W = \begin{pmatrix} 1.5 & 0 \\ 0 & 1.5 \end{pmatrix}$$

$$W^{20} = \begin{pmatrix} 3325.26 & 0 \\ 0 & 3325.26 \end{pmatrix}$$

Initialization

Vanishing & Exploding Gradient Problem



$L = 21$

$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

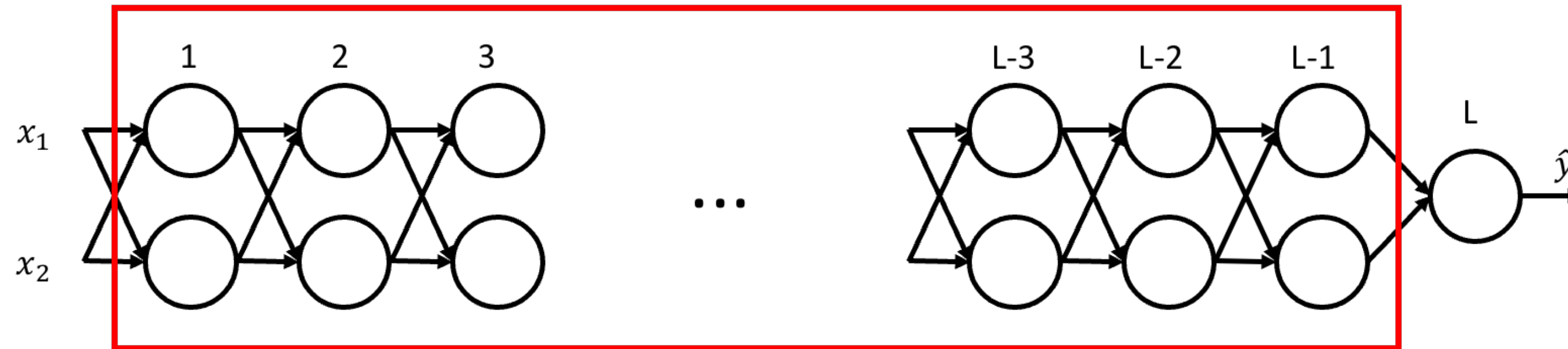
$$\hat{y} = W^{[L]} W^{L-1} x$$

$$W = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

W is raised to the number of layers

Initialization

Vanishing & Exploding Gradient Problem



$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

$$\hat{y} = W^{[L]} W^{L-1} x$$

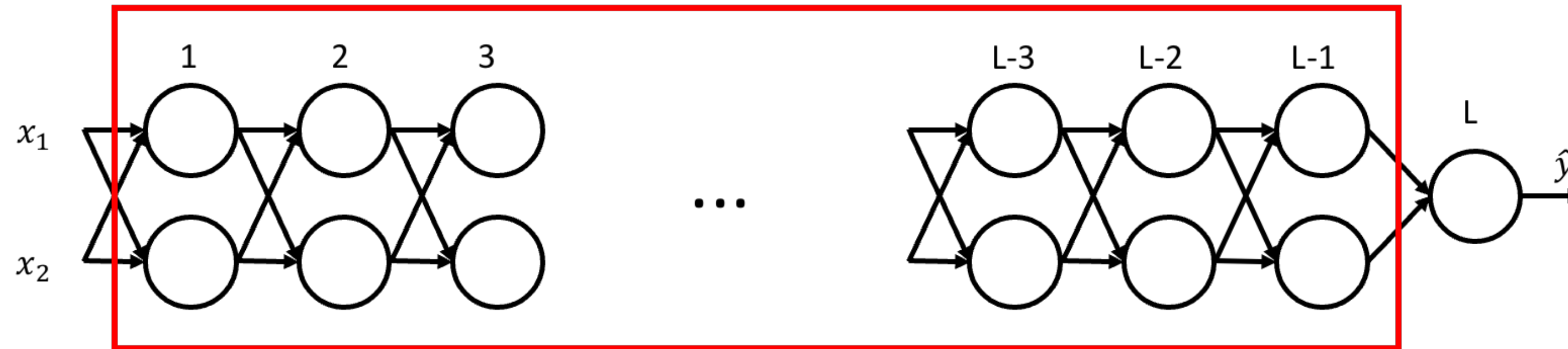
W is raised to the number of layers

$$W = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

$$W^2 = \begin{pmatrix} 0.25 & 0 \\ 0 & 0.25 \end{pmatrix}$$

Initialization

Vanishing & Exploding Gradient Problem



$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

$$\hat{y} = W^{[L]} W^{L-1} x$$

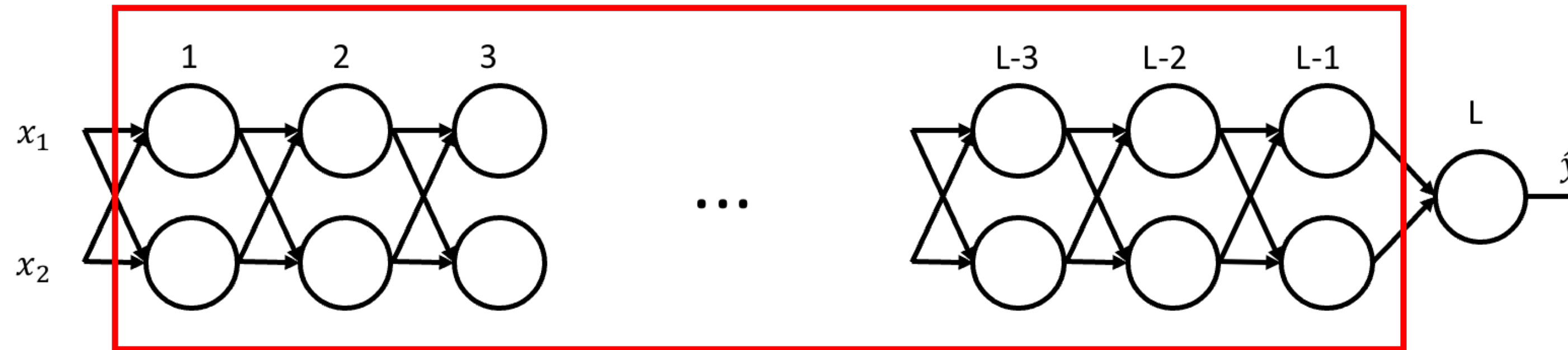
W is raised to the number of layers

$$W = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

$$W^3 = \begin{pmatrix} 0.125 & 0 \\ 0 & 0.125 \end{pmatrix}$$

Initialization

Vanishing & Exploding Gradient Problem



$$\hat{y} = W^{[L]} \dots W^{[4]} W^{[3]} W^{[2]} W^{[1]} x$$

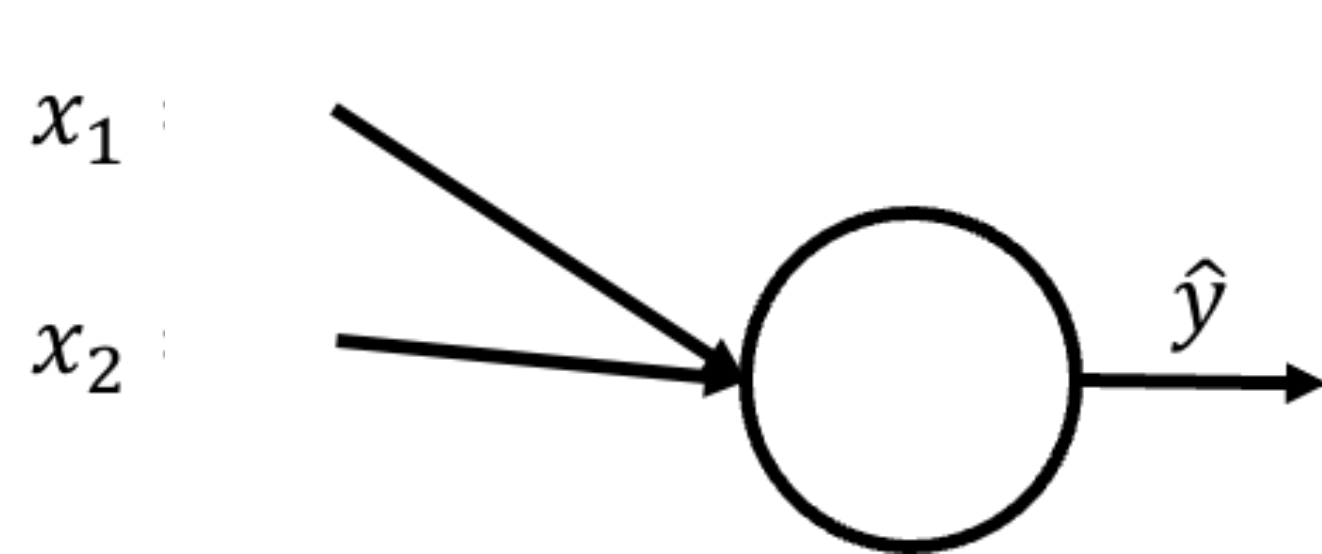
$$\hat{y} = W^{[L]} W^{L-1} x$$

$$W = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

W is raised to the number of layers

$$W^{20} = \begin{pmatrix} 0.000000095 & 0 \\ 0 & 0.000000095 \end{pmatrix}$$

Initialization

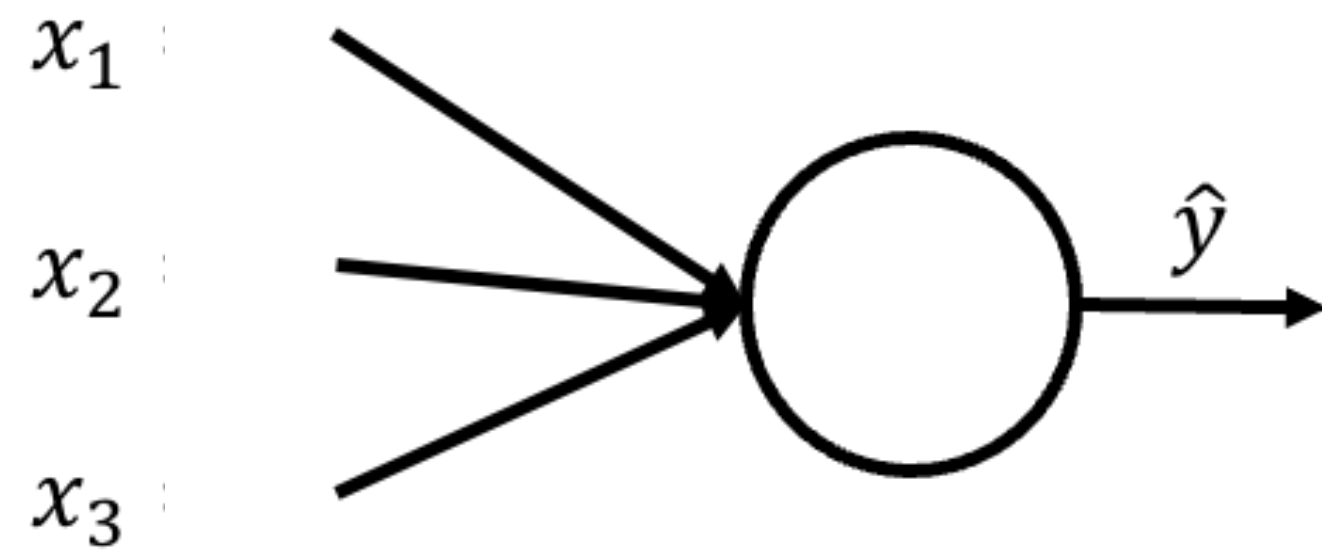


$$b = 0$$

$$Var(w_i) = 1$$

$$z = w_1x_1 + w_2x_2$$

Initialization

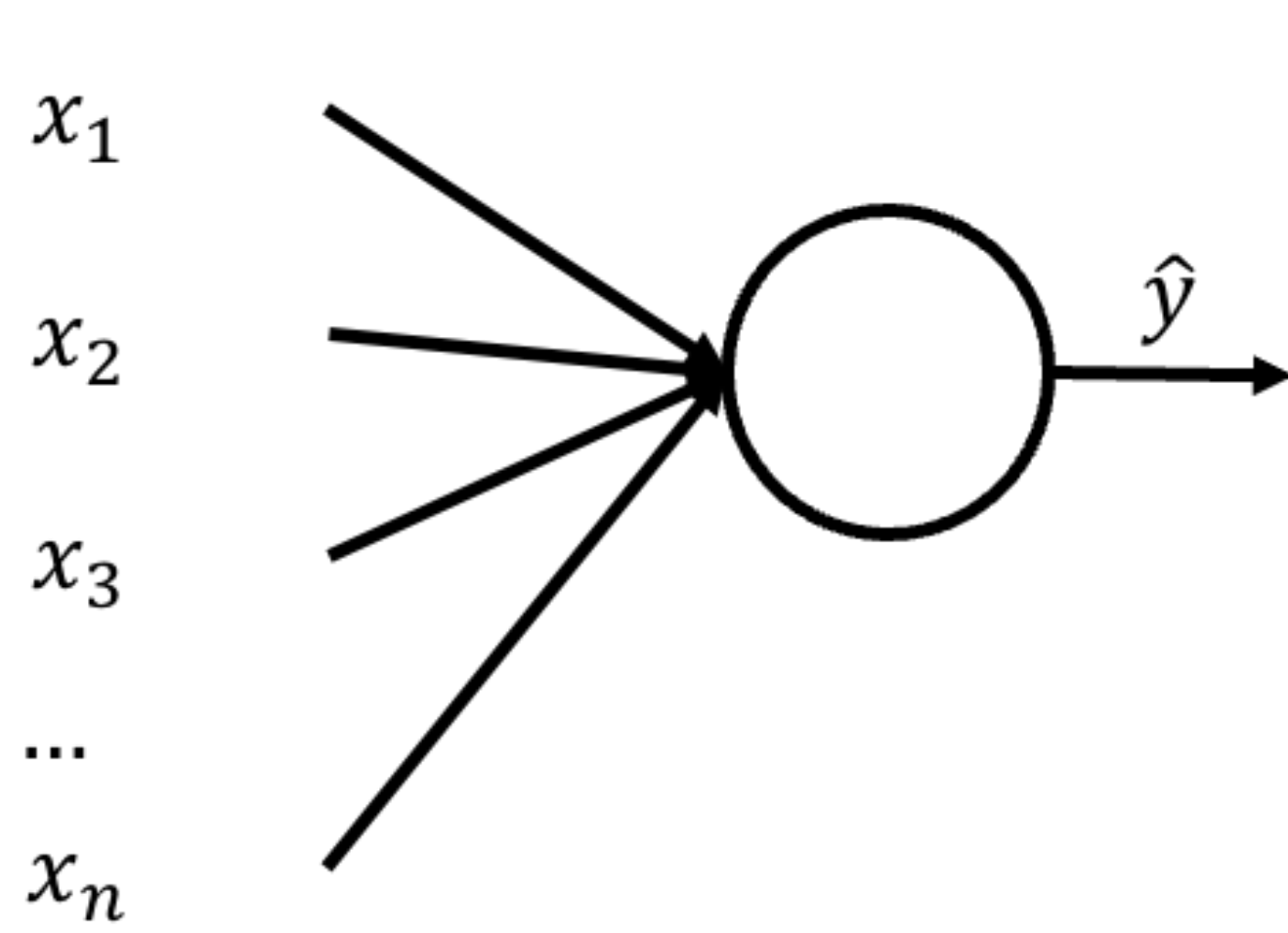


$$b = 0$$

$$Var(w_i) = 1$$

$$z = w_1x_1 + w_2x_2 + w_3x_3$$

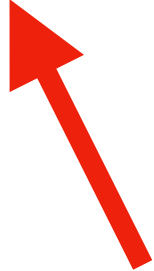
Initialization



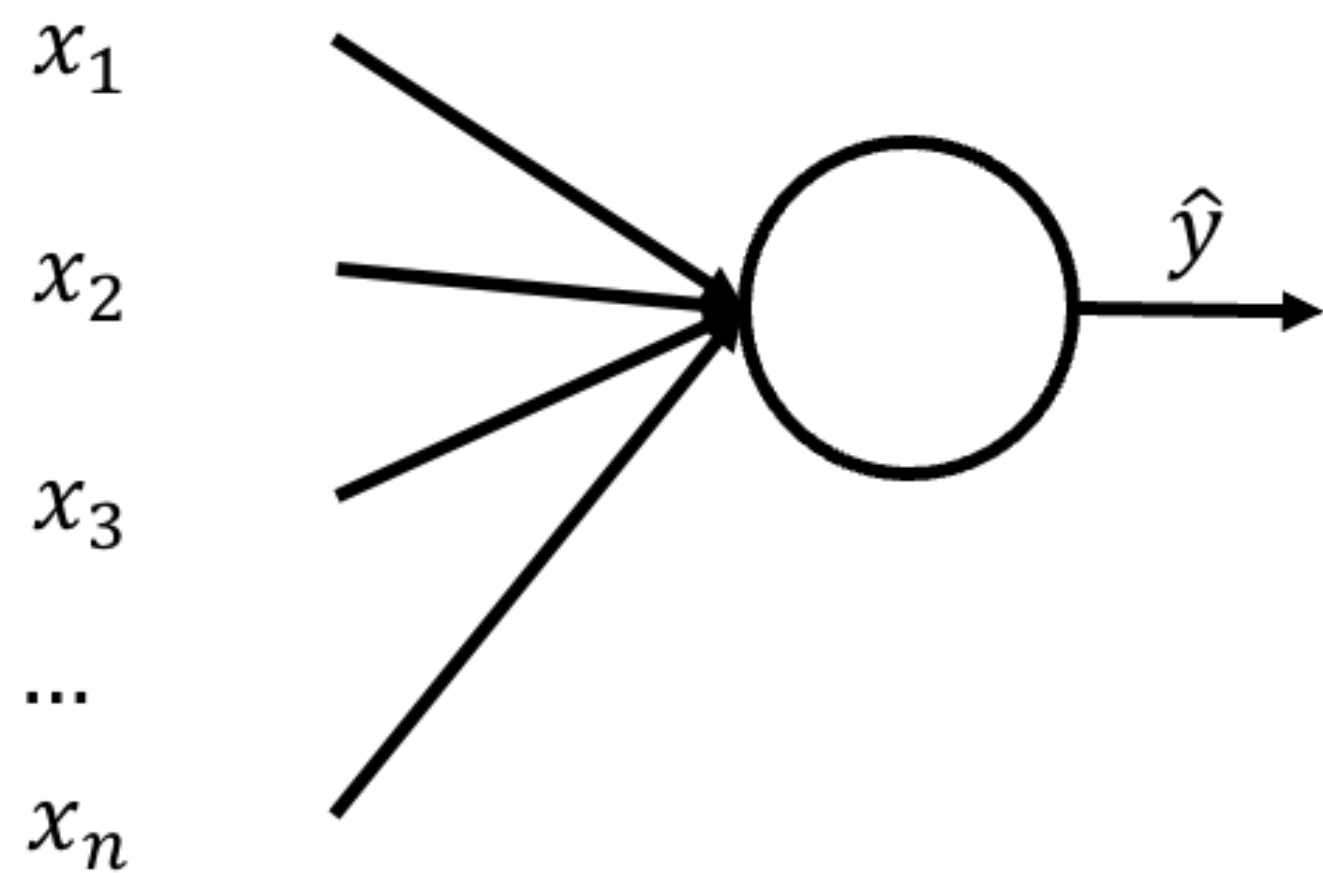
$$b = 0$$

$$Var(w_i) = 1$$

$$z = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n$$

 $Var(z) = n Var(w_i) Var(x_i) = n Var(x_i)$

Initialization



$$b = 0$$

$$\text{Var}(w_i) = \frac{1}{n}$$

$$z = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n$$

$$\text{Var}(z) = n \text{Var}(w_i) \text{Var}(x_i) = \textcolor{red}{n} \frac{1}{n} \text{Var}(x_i) = \text{Var}(x_i)$$

Initialization

- Xavier Glorot Initialization

$$\sigma = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

Sigmoid activation

- Kaiming He Initialization

$$\sigma = \sqrt{\frac{4}{n_{in} + n_{out}}}$$

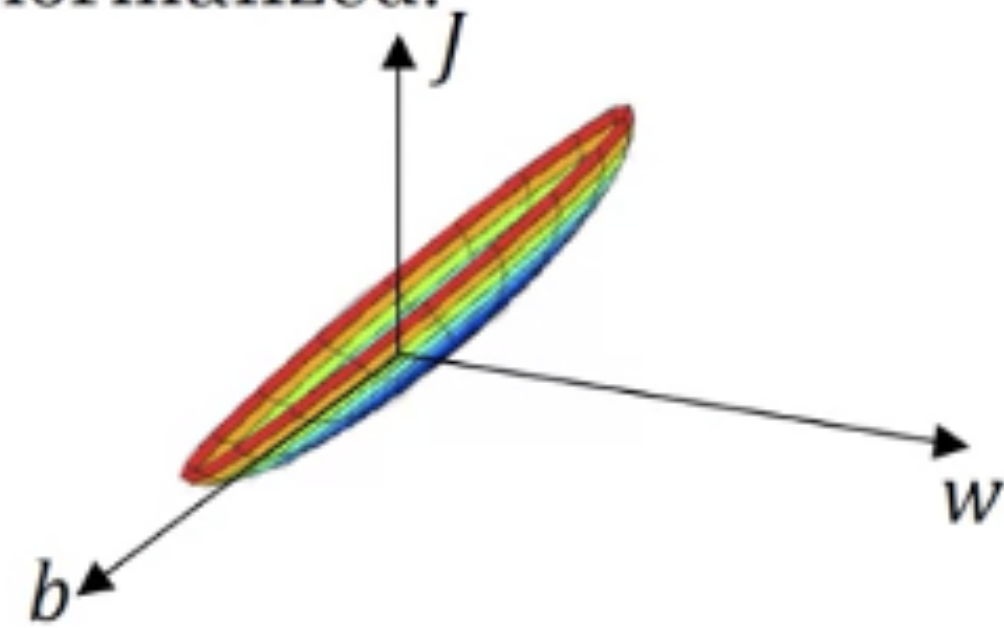
ReLU activation

Normalization

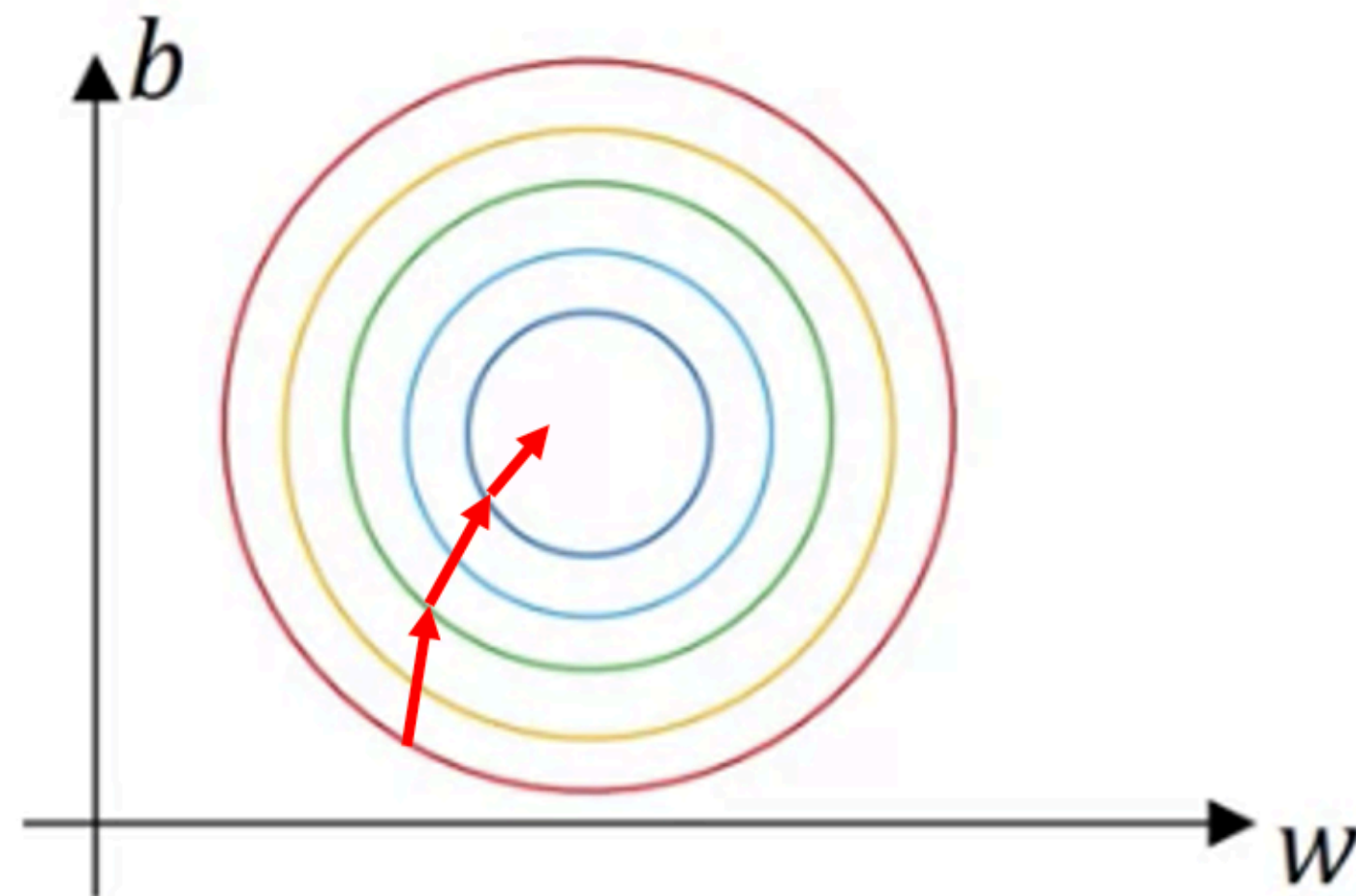
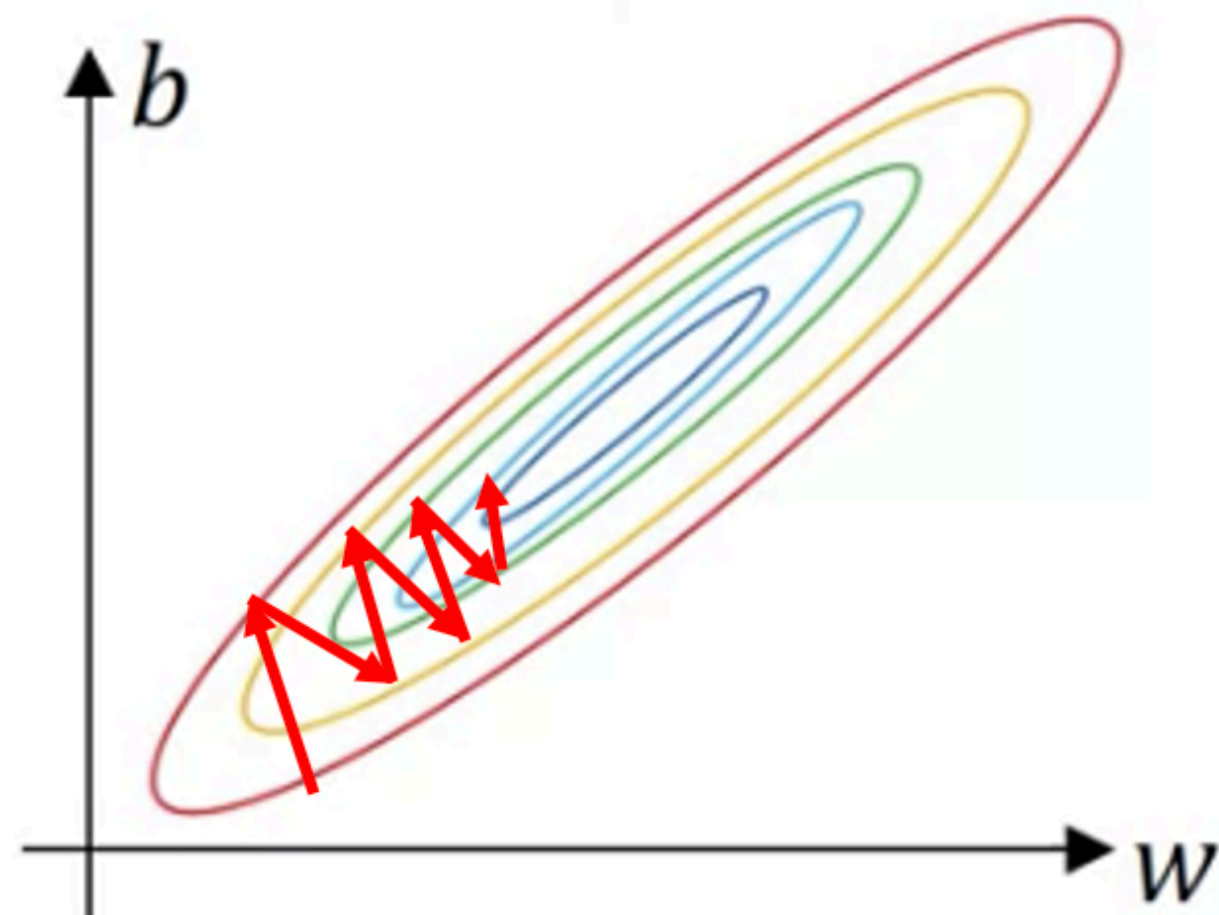
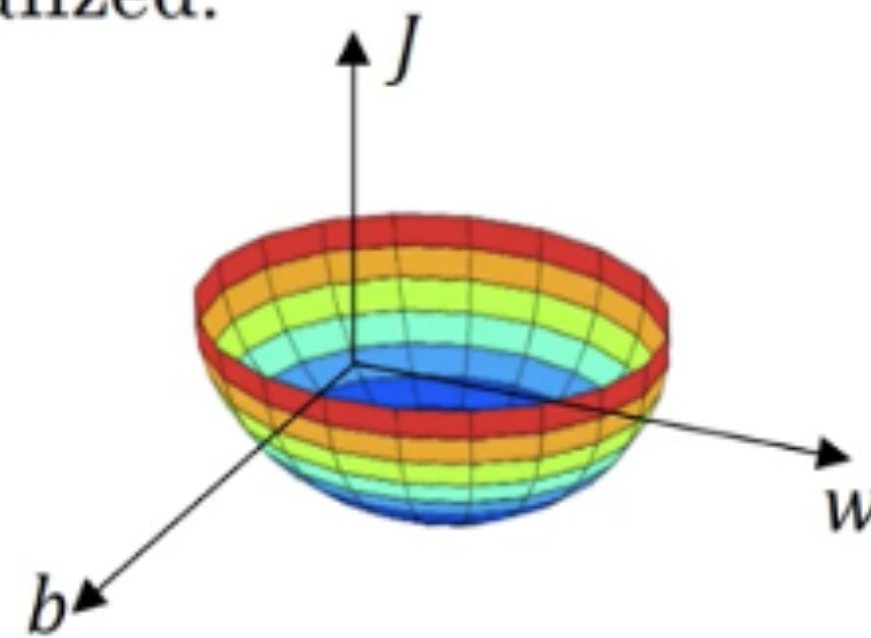
“Keep scales of inputs and activations consistent (for stable and faster training)”

Normalization

Unnormalized:



Normalized:



Normalization

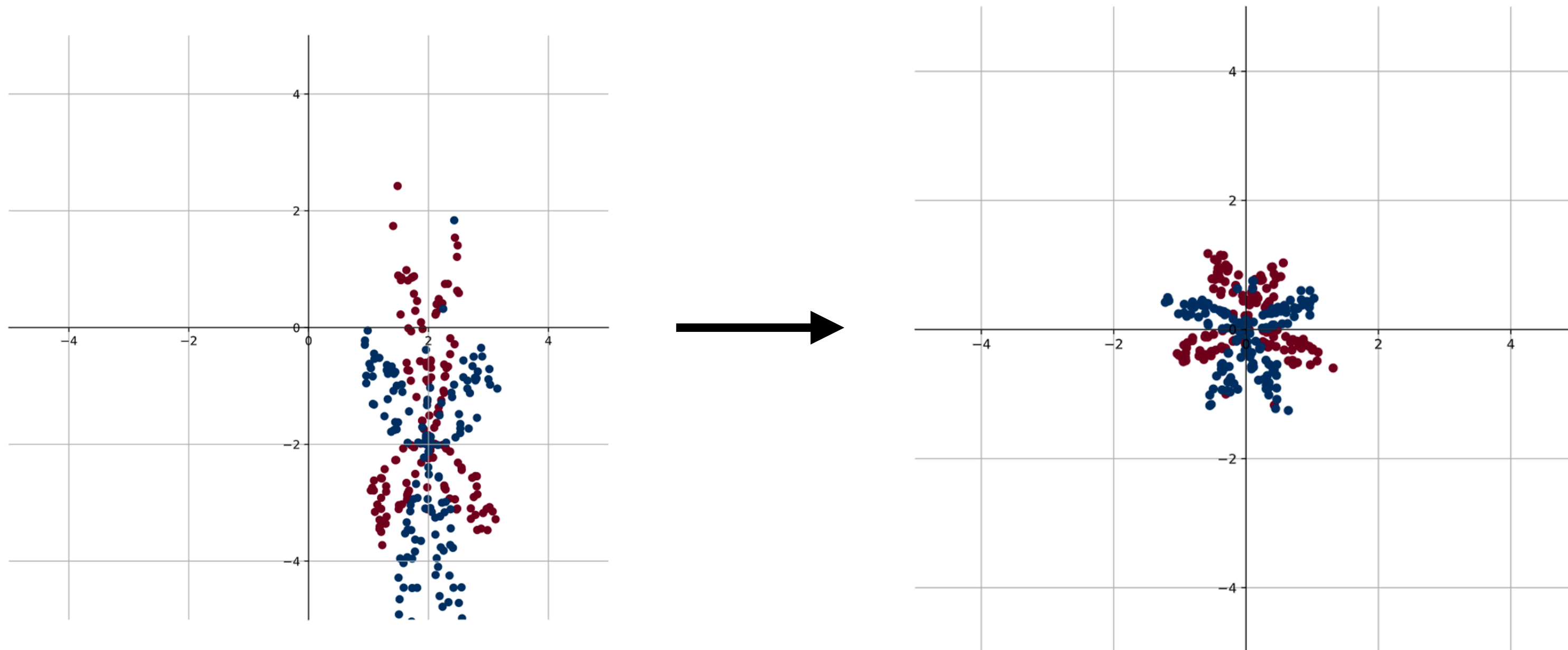
Input Normalization

- Mean
- Variance

$$\mu = E(X) = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$\sigma^2 = Var(X) = E[(X - \mu)^2]$$

$$\tilde{x} = \frac{x - \mu}{\sigma}$$



Normalization

Input Normalization

- **Single sample** - has **value**

Normalization

Input Normalization

- Single sample - has value
- **Set of samples** (Dataset) - can be thought of as **random variable**
 - Mean
 - Variance
 - Covariance
 - ...

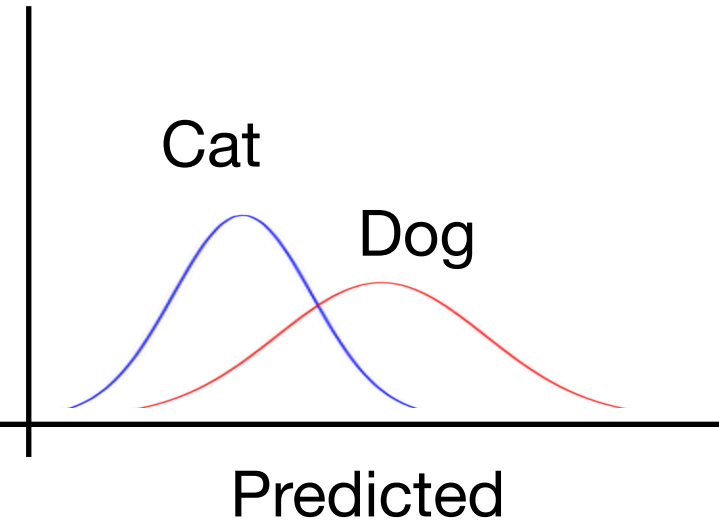
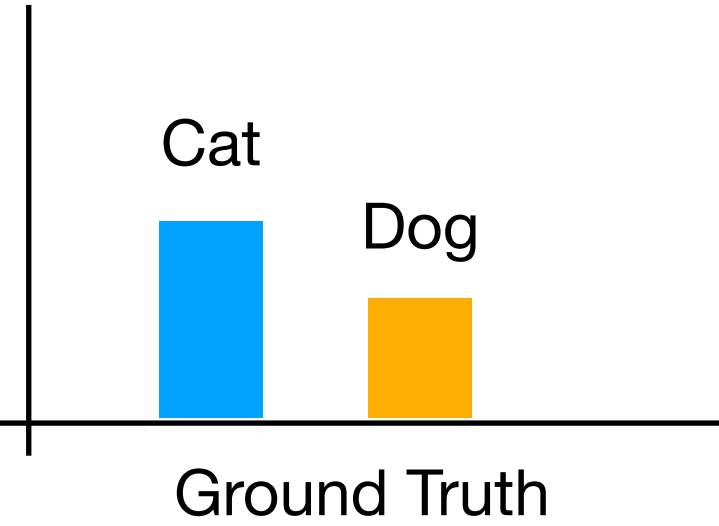
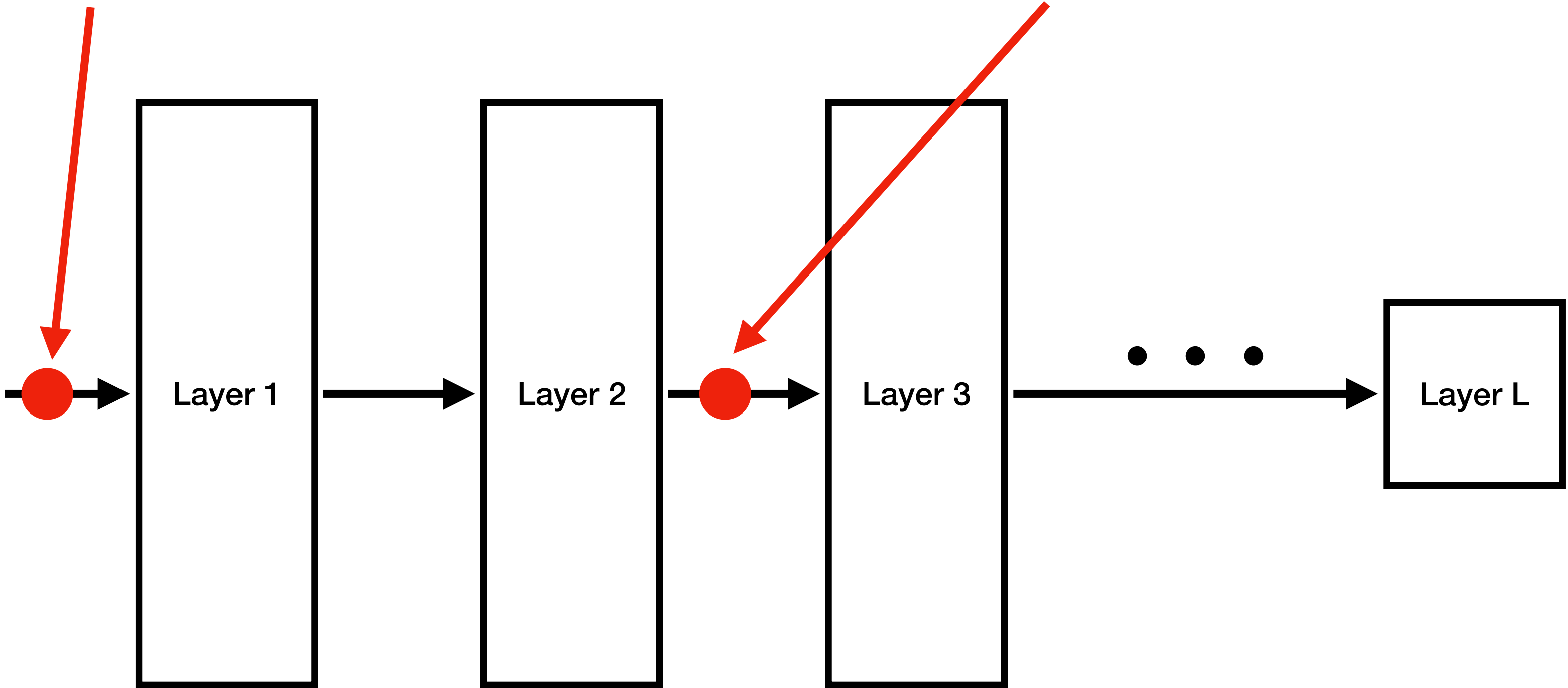
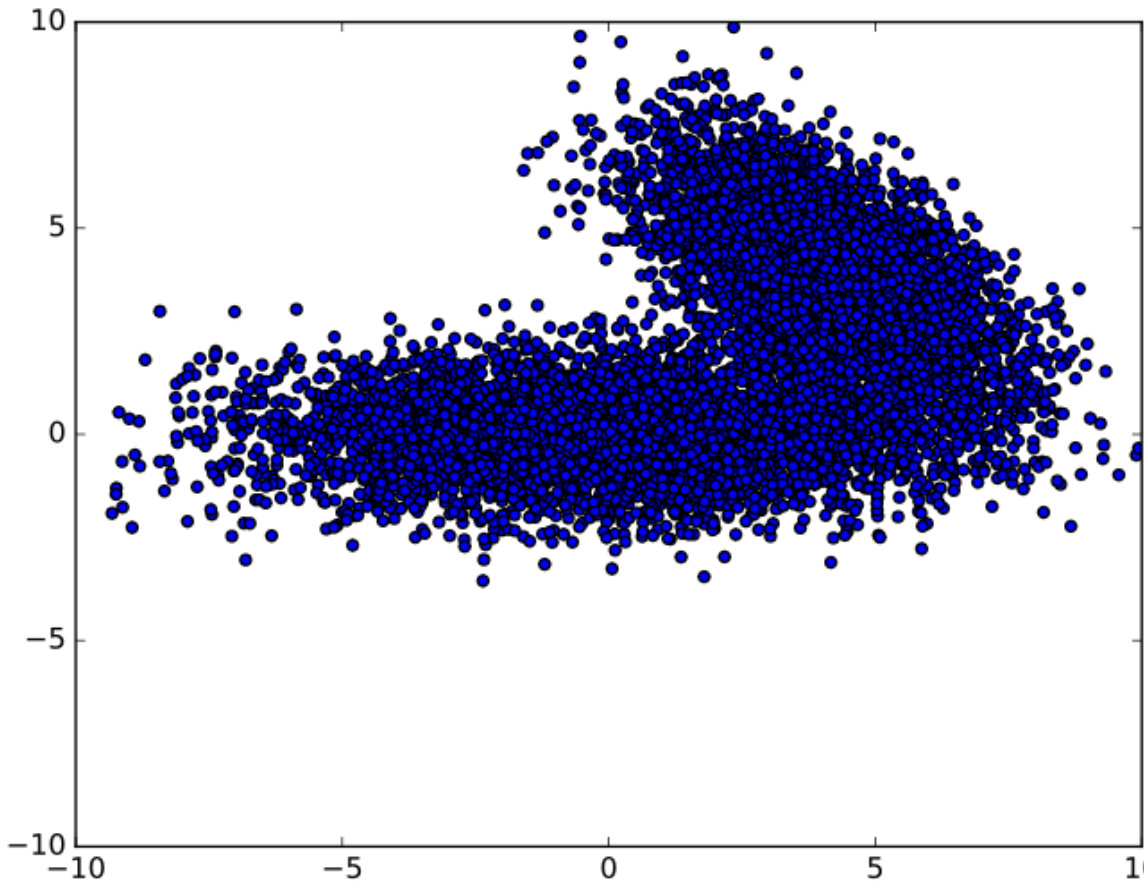
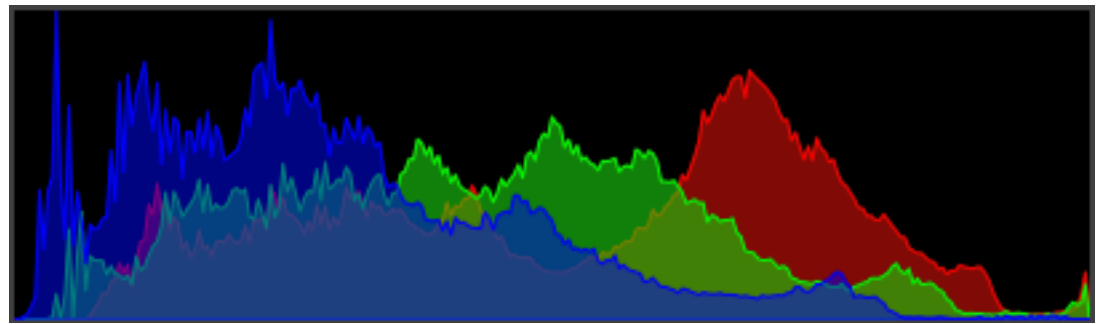
Normalization

Input Normalization

- Single sample - has value
- Set of samples (Dataset) - can be thought of as random variable
 - Mean
 - Variance
 - Covariance
 - ...
- Always compute μ_{train} , σ_{train} on the training dataset, then **normalize all splits (training, validation, test, ...) using the same μ_{train} , σ_{train} values**
=> **prevents data leakage**

Normalization

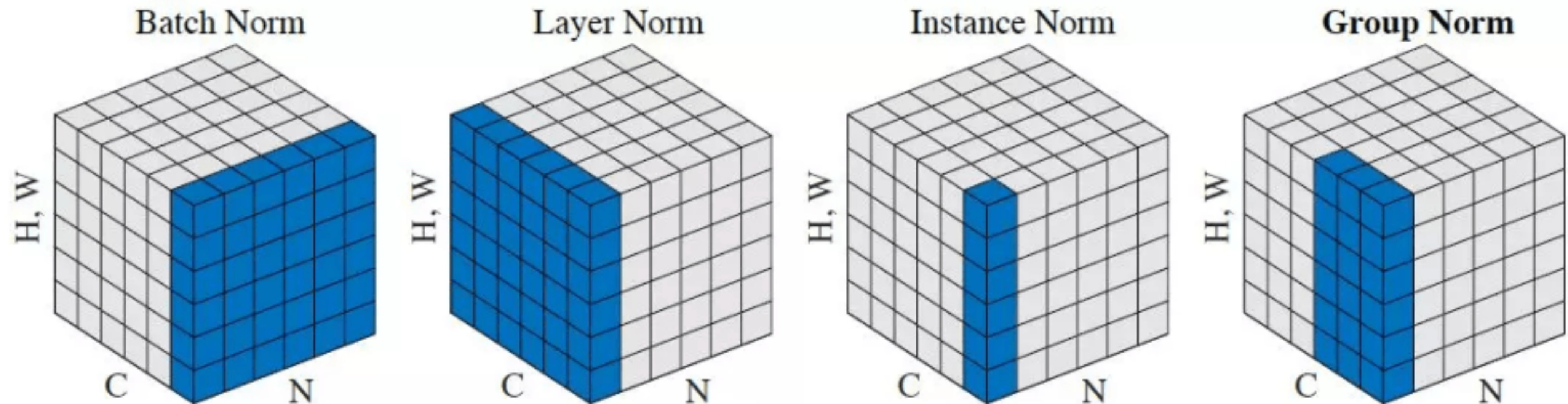
Normalization Inside the Network



Normalization

Normalization Inside the Network

normalizes hidden-layer activations to stabilize training



N = batch size, C = channels / feature maps, H, W = spatial dimensions

Normalization

Normalization Inside the Network

- Normalization can also have learnable parameters (γ, β)

$$y = \gamma \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} + \beta$$

Small value
for numerical stability

Normalization

Normalization Inside the Network

- Normalization can also have learnable parameters
 - γ, β are learned (like weights), via backprop
- Training - we track the μ, σ values
- Validation - we must **use the same values** to transform data